OPERATOR DOCUMENTATION

Complete Operator Guide

Every architecture brief, setup handbook and field guide for the InfraneX BT mining platform — merged into one navigable document.

7	153	3
DOCUMENTS	PAGES OF CONTENT	PARTS

01 Platform Architecture — MVP Overview
02 Architecture Documentation v2
03 CPU Miner Setup Guide
04 GPU Miner Setup Guide
05 SN6 Laptop CPU Operator Guide
06 Deployment & Hosting Guide
07 DevOps Engine — Miner Operations Field Guide

Contents

Three parts · seven documents · bookmarked in the PDF sidebar

PART I — PLATFORM & ARCHITECTURE

01 Platform Architecture — MVP Overview ..... pp. 3–52 **50 pp**

The full platform brief: intelligence dashboards, mining operations, and the MVP system design.

02 Architecture Documentation v2 ..... pp. 53–67 **15 pp**

Formal reference for the current system: modules, data flows and integrations.

PART II — MINER SETUP

03 CPU Miner Setup Guide ..... pp. 68–90 **23 pp**

Operator handbook — from an empty dashboard to a running CPU miner.

04 GPU Miner Setup Guide ..... pp. 91–117 **27 pp**

Operator handbook for GPU-backed subnets — hardware, hosting and registration.

05 SN6 Laptop CPU Operator Guide ..... pp. 118–130 **13 pp**

Compact laptop-focused walkthrough for the SN6 CPU operator track.

PART III — OPERATIONS

06 Deployment & Hosting Guide ..... pp. 131–142 **12 pp**

Verified DigitalOcean hosting recipes and deployment paths.

07 DevOps Engine — Miner Operations Field Guide ..... pp. 143–155 **13 pp**

GPU miner operations under the DevOps engine: monitoring, triggers and defense.

Infranex BT

Bittensor Intelligence & Mining Operations Platform

> *A production-ready platform for discovering, analyzing, and deploying Bittensor mining operations with GPU infrastructure automation.*

Overview

Infranex BT automates the complete Bittensor mining lifecycle - from subnet discovery to miner deployment, monitoring, and optimization.

```
BITTENSOR NETWORK
        ?
        ?
INFRANEX INTELLIGENCE ENGINE
        ?
        ?
SUBNET ANALYZER (Emissions, Competition, Requirements, GPU, Risk, ROI)
        ?
        ?
GPU RECOMMENDATION ("Use H100 80GB", "Provider: X")
        ?
        ?
USER (Register miner -> Rent GPU -> Connect hotkey)
        ?
        ?
GPU PROVIDER ENGINE (Connect GPU, Detect hardware, Check environment, Configure GPU, Install miner)
        ?
        ?
DEPLOYMENT ENGINE (Subnet config, Dependencies, Docker, Models, Miner config)
        ?
        ?
PRE-FLIGHT ENGINE (GPU, CUDA, Miner, Hotkey, UID, Subnet, Network)
        ?
        ?
MINER RUNNING
        ?
        ?
MONITORING ENGINE (GPU + Miner + Incentive, Rewards + Cost + ROI)
        ?
        ?
OPTIMIZATION ENGINE (Keep / Optimize / Switch)
```

MVP Architecture

Layer 1 - Intelligence Engine

Purpose: Find the best subnets to mine on.

Component	Description
Bittensor Network Client	Connect to subtensor RPC, fetch metagraphs, neurons, emissions
Subnet Analyzer	Score subnets on emissions, competition, requirements, GPU needs, risk, ROI
3-Pillar Scoring Model	Utility (30%), Technical (35%), Economics (35%)
Decision Engine	RUN (>=75) / WATCH (40-74) / AVOID (<40)

Status: ? Built

Layer 2 - GPU Recommendation

Purpose: Match the best GPU to the best subnet.

Component	Description
GPU Catalog	Available GPU types, VRAM, CUDA compute capability
Provider Comparison	Price, availability, reliability across RunPod, Vast.ai, TensorDock, E2E
Recommendation Engine	"Use H100 80GB from Provider X - estimated $X/hr, Y% ROI"

Status: ?? Partial (mock mode only)

Layer 3 - User Onboarding

Purpose: Get the user from registration to miner setup.

Component	Description
Register Miner	User creates miner profile, connects wallet
Rent GPU	User selects GPU from recommendation or catalog
Connect Hotkey	User provides Bittensor hotkey for the miner

Status: ?? Partial (auth exists, no miner/hotkey UI flow)

Layer 4 - GPU Provider Engine

Purpose: Provision and configure real GPU infrastructure.

Component	Description
Connect GPU	SSH/API connection to rented GPU server
Detect Hardware	GPU model, VRAM, CUDA version, driver version
Check Environment	OS, Docker, network, disk space

Configure GPU

Install CUDA drivers, set up environment

Install Miner

Clone subnet repo, install dependencies, configure

Status: ?? Partial (provider adapters exist, no real execution)

Layer 5 - Deployment Engine

Purpose: Deploy the miner to the configured GPU.

Component	Description
Subnet Configuration	Fetch subnet-specific config (hotkey, netuid, etc.)
Dependencies	Install Python packages, models, data
Docker	Containerize and start the miner
Models	Download required ML models
Miner Configuration	Write config files, set environment variables

Status: ?? Partial (state machine exists, no real Docker/SSH execution)

Layer 6 - Pre-Flight Engine

Purpose: Validate everything before miner launch.

Check	Description
GPU	Correct GPU type detected, VRAM sufficient
CUDA	CUDA toolkit installed, version compatible
Miner	Miner binary/script present and executable
Hotkey	Hotkey is valid and registered on-chain
UID	Miner UID is available in the subnet
Subnet	Subnet is active and accepting miners
Network	Subtensor connectivity confirmed

Status: ? Not built

Layer 7 - Miner Lifecycle

Purpose: Start, stop, restart miners.

Component	Description
Start Miner	Launch miner process in background/screen/tmux
Stop Miner	Graceful shutdown with cleanup
Restart Miner	Stop + start with config reload
Status Check	Is the miner process alive and responding

Status: ? Not built

Layer 8 - Monitoring Engine

Purpose: Track miner performance and earnings in real-time.

Component	Description
GPU Monitoring	Utilization, temperature, memory, power
Miner Monitoring	Process health, logs, uptime
Incentive Tracking	Emissions received, weights set by validators
Reward Tracking	TAO earned, USD equivalent, historical
Cost Tracking	GPU rental cost, total spend
ROI Calculation	Revenue - Cost, trend, comparison to projections

Status: ?? Partial (framework exists, no real telemetry)

Layer 9 - Optimization Engine

Purpose: Maximize ROI through continuous optimization.

Component	Description
Keep	Continue mining if ROI is positive and trending up
Optimize	Adjust config, switch to better GPU, tune parameters
Switch	Migrate to a different subnet if opportunity score changes
Auto-Switch	Automated migration when ROI drops below threshold

Status: ?? Partial (optimizer logic exists, no real switching)

Technology Stack

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS, Radix UI, Recharts
Backend	FastAPI, Python 3.11+, Pydantic v2, SQLAlchemy, Alembic
Database	Supabase PostgreSQL (RLS, migrations)
Auth	Supabase Auth (JWT, OAuth)
Workers	Python async workers, Vercel Cron
GPU Providers	RunPod, Vast.ai, TensorDock, E2E Networks
Bittensor	Official Python SDK (bittensor>=9.0)
Deployment	Vercel (frontend + backend), Supabase (database)

Phase-wise Build Plan

Phase 1 - Foundation (Week 1-2)

Goal: Working frontend + backend + database + auth.

Task	Status
Next.js 16 frontend scaffold with App Router	?
FastAPI backend scaffold with CORS	?
Supabase project setup (PostgreSQL)	?
27+ database tables (subnets, metrics, emissions, GPUs, deployments, miners)	?
SQLAlchemy ORM models for all tables	?
Alembic migration setup	?
Supabase Auth (login, logout, callback)	?
Frontend layout (sidebar, header, dashboard shell)	?
Environment variable configuration (.env)	?

Deliverable: User can log in, see empty dashboard, API returns health checks.

Phase 2 - Data Ingestion (Week 3-4)

Goal: Bittensor network data flowing into the database.

Task	Status
Bittensor SDK client (dual-mode: mock/production)	?
Subnet scanner worker (fetch all subnets, metrics)	?
Market data worker (TAO price, FX rates)	?
GitHub analyzer worker (repo analysis, requirements extraction)	?
Vercel Cron configuration (scanner: 5min, market: 1hr, scoring: 15min)	?
Database seed data for testing	?

Deliverable: Subnets, metrics, and market data populate in the database automatically.

Phase 3 - Intelligence Engine (Week 5-6)

Goal: Automated subnet scoring and opportunity detection.

Task	Status
3-pillar scoring model (Utility 30%, Technical 35%, Economics 35%)	?
Opportunity score calculation (0-100)	?
Decision engine (RUN/WATCH/AVOID)	?
Strategy engine (portfolio constraints)	?
GPU matching service (catalog ranking)	?
Profitability projection (revenue/cost/ROI)	?
Change detection worker (subnets, emissions, competition)	?

Deliverable: Subnets scored, opportunities ranked, GPU recommendations generated.

Phase 4 - Frontend Dashboard (Week 7-8)

Goal: Users can browse and analyze subnets.

Task	Status
Dashboard page (metrics, charts, recent activity)	?
Subnets list page (sortable, filterable table)	?
Subnet detail page (metrics, history, score breakdown)	?
Opportunities page (ranked opportunities with scores)	?
Analytics page (trends, charts)	?
Settings page	?
React Query data fetching hooks	?
Recharts visualizations (revenue, trends)	?

Deliverable: Users can browse subnets, view scores, analyze opportunities.

Phase 5 - GPU Provider Integration (Week 9-10)

Goal: Real GPU providers connected, users can rent GPUs.

Task	Status	Priority
Provider adapter interface (standardized)	?	-
RunPod adapter (real API)	?? Mock	P0
Vast.ai adapter (real API)	?? Mock	P0
TensorDock adapter (real API)	?? Mock	P1

E2E Networks adapter (real API)

?? Mock

P1

GPU catalog page (browse available GPUs)

?? Partial

P0

GPU recommendation UI ("Best GPU for this subnet")

?

P0

Provider comparison page

?

P1

GPU rental flow (select -> rent -> confirm)

?

P0

Deliverable: Users can browse GPUs, get recommendations, and rent real GPUs.

Phase 6 - User Miner Flow (Week 11-12)

Goal: Users can register miners and connect hotkeys.

Task	Status	Priority
Miner registration page	?	P0
Hotkey connection flow	?	P0
Wallet integration (Bittensor wallet)	?	P0
Miner profile page	?	P0
Miner list page (user's miners)	?	P0
Hotkey validation (on-chain check)	?	P0

Deliverable: Users can create miners, provide hotkeys, and manage their miner portfolio.

Phase 7 - GPU Provider Engine (Week 13-14)

Goal: Real GPU detection, configuration, and miner installation.

Task	Status	Priority
SSH connection to GPU server	?	P0
Hardware detection (GPU model, VRAM, CUDA)	?	P0
Environment check (OS, Docker, disk, network)	?	P0
CUDA driver installation	?	P0
Miner software installation	?	P0
Subnet dependency installation	?	P0
Model download and setup	?	P0

Deliverable: Automated GPU server setup from bare machine to miner-ready.

Phase 8 - Deployment Engine (Week 15-16)

Goal: Deploy miners to GPU servers.

Task	Status	Priority
Deployment state machine (real execution)	?? Partial	P0

Docker containerization of miners

?

P0

Miner startup with screen/tmux

?

P0

Config file generation

?

P0

Environment variable injection

?

P0

Deployment approval flow (UI)

?? Partial

P0

Deployment status page

?? Partial

P0

Deliverable: One-click deployment from opportunity to running miner.

Phase 9 - Pre-Flight Engine (Week 17)

Goal: Validate everything before miner launch.

Task	Status	Priority
GPU validation (correct type, sufficient VRAM)	?	P0
CUDA validation (version compatibility)	?	P0
Miner validation (binary exists, executable)	?	P0
Hotkey validation (registered on-chain)	?	P0
UID validation (available in subnet)	?	P0
Subnet validation (active, accepting miners)	?	P0
Network validation (subtensor connectivity)	?	P0
Pre-flight UI (checklist with pass/fail)	?	P0

Deliverable: Green-light checklist before every deployment.

Phase 10 - Monitoring Engine (Week 18-19)

Goal: Real-time monitoring of running miners.

Task	Status	Priority
GPU telemetry collection (utilization, temp, memory, power)	?	P0
Miner process health monitoring	?	P0
Emissions tracking (on-chain rewards)	?	P0
Reward calculation (TAO earned, USD equivalent)	?	P0
Cost tracking (GPU rental spend)	?	P0
ROI calculation (revenue - cost)	?	P0
Monitoring dashboard (real-time charts)	?? Partial	P0
Alert system (miner down, GPU hot, low rewards)	?	P1
SSE streaming for live updates	?? Partial	P1

Deliverable: Live dashboard showing GPU health, earnings, and costs.

Phase 11 - Optimization Engine (Week 20-21)

Goal: Automatic optimization and subnet switching.

Task	Status	Priority
ROI threshold monitoring	?	P0
Config optimization suggestions	?	P1
GPU upgrade/downgrade recommendations	?	P1
Subnet switch recommendations	?	P0
Auto-switch execution (with approval)	?	P2
A/B testing (run on 2 subnets, compare)	?	P2
Learning engine (feedback loop, weight adaptation)	?	P1

Deliverable: System recommends and executes optimizations to maximize ROI.

Phase 12 - Production Hardening (Week 22-24)

Goal: Security, reliability, and production readiness.

Task	Status	Priority
3-level approval gates (L1/L2/L3)	?	P0
Audit logging	?	P0
Error handling and retry logic	?? Partial	P0
Rate limiting and abuse protection	?	P0
Comprehensive test suite (>80% coverage)	?	P0
Load testing	?	P1
Security audit	?	P0
Documentation (API, architecture, deployment)	?? Partial	P1
CI/CD pipeline (GitHub Actions)	?	P0
Monitoring and alerting (Sentry, Datadog)	?	P1

Deliverable: Production-ready platform with security, tests, and CI/CD.

Current Build Status

```
????????????????????????????????  55% Complete

Phase  1-4:  ????????????????????  100%  (Foundation, Data, Intelligence, UI)
Phase  5-6:  ????????????????????   60%  (GPU Providers, User Miner Flow)
Phase  7-8:  ????????????????????   40%  (GPU Engine, Deployment)
Phase  9:    ????????????????????    0%  (Pre-Flight)
Phase 10:    ????????????????????   20%  (Monitoring)
Phase 11:    ????????????????????   20%  (Optimization)
Phase 12:    ????????????????????   40%  (Production Hardening)
```

Project Structure

```
infranex-bt/
??? frontend/                 # Next.js 16 App
?   ??? app/                  # App Router pages
?   ?   ??? (auth)/          # Auth pages (login, callback, logout)
?   ?   ??? dashboard/       # Dashboard page
?   ?   ??? subnets/         # Subnets list + detail
?   ?   ??? miners/          # Miners list + detail
?   ?   ??? opportunities/   # Opportunities page
?   ?   ??? analytics/       # Analytics page
?   ?   ??? settings/        # Settings page
?   ??? components/
?   ?   ??? ui/              # Radix UI components
?   ?   ??? layout/          # Sidebar, Header, DashboardLayout
?   ?   ??? cards/           # MetricCard, OpportunityCard, ChangesCard
?   ?   ??? tables/          # OpportunityTable
?   ?   ??? charts/          # RevenueChart, TrendChart
?   ??? hooks/               # useAuth, useSubnets, useOpportunities, etc.
?   ??? lib/                 # API client, Supabase, utils
?   ??? types/               # TypeScript interfaces
?
??? backend/                  # FastAPI App
?   ??? app/
?   ?   ??? api/             # API routes (16 modules)
?   ?   ??? core/            # Config, DB, Auth, Logging
?   ?   ??? models/          # SQLAlchemy ORM (35+ models)
?   ?   ??? schemas/         # Pydantic validation
?   ?   ??? services/        # Business logic (scoring, GPU, profitability)
?   ?   ??? workers/         # Background workers (scanner, scoring, market)
?   ?   ??? intelligence/    # v2.0 3-pillar scoring engine
?   ?   ??? strategy/        # Strategy engine (RUN/WATCH/AVOID)
?   ?   ??? orchestrator/    # Miner lifecycle state machine
?   ?   ??? optimizer/       # Cost optimization, subnet switching
?   ?   ??? recovery/        # Health monitoring, auto-recovery
?   ?   ??? learning/        # Feedback loop, weight adaptation
?   ?   ??? providers/       # GPU provider adapters (Mock, RunPod, Vast.ai, etc.)
```

```
?   ?   ??? deployment/      # Deployment state machine
?   ?   ??? approval/        # L1/L2/L3 approval gates
?   ?   ??? clients/         # Bittensor SDK client, GitHub fetcher
?   ??? tests/               # Pytest tests
?   ??? alembic/             # Database migrations
?   ??? requirements.txt
?
??? database/
?   ??? migrations/          # SQL migrations (001_initial, 002_approval, 003_learning)
?
??? docs/
?   ??? architecture/        # Architecture docs, scoring docs
?
??? .vercel/                 # Vercel deployment config
??? vercel.json              # Root Vercel config (API rewrites)
??? README.md
```

Quick Start

1. Clone & Configure

```
git clone <repository-url>
cd infranex-bt

cp .env.example .env
# Edit .env with your Supabase, Bittensor, and Redis credentials
```

2. Frontend

```
cd frontend
npm install
npm run dev        # http://localhost:3000
```

3. Backend

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000  # http://localhost:8000/docs
```

4. Database

```
psql $SUPABASE_URL -f database/migrations/001_initial_schema.sql
psql $SUPABASE_URL -f database/migrations/002_approval_audit_drift.sql
psql $SUPABASE_URL -f database/migrations/003_learning_engine.sql
cd backend && alembic stamp head
```

Environment Variables

Variable	Required	Description
`SUPABASE_URL`	Yes	Supabase project URL
`SUPABASE_ANON_KEY`	Yes	Supabase anonymous key
`SUPABASE_SERVICE_ROLE_KEY`	Yes	Supabase service role key
`BITTENSOR_NETWORK`	Yes	`testnet` or `finney`
`BITTENSOR_RPC_ENDPOINT`	Yes	Subtensor RPC WebSocket
`REDIS_URL`	Yes	Redis/Valkey connection
`SECRET_KEY`	Yes	JWT signing secret
`DEPLOYMENT_MODE`	Yes	`mock` or `production`
`GITHUB_TOKEN`	No	GitHub PAT for repo analysis
`RUNPOD_API_KEY`	No	RunPod API key
`VASTAI_API_KEY`	No	Vast.ai API key
`TENSORDOCK_API_KEY`	No	TensorDock API key

API Endpoints

Method	Endpoint	Description
GET	`/health`	Health check
GET	`/api/subnets`	List subnets
GET	`/api/subnets/{netuid}`	Subnet detail
GET	`/api/subnets/{netuid}/score`	Opportunity score
GET	`/api/opportunities`	List opportunities
GET	`/api/gpus`	GPU catalog
POST	`/api/deployments/preview`	Preview deployment
POST	`/api/deployments/approve`	Approve & deploy
GET	`/api/miners`	List miners
GET	`/api/miners/{id}/health`	Miner health
GET	`/api/rewards`	Reward history
GET	`/api/profitability`	Profitability reports
POST	`/api/approvals/{id}/approve`	Approve action

Security

Principle	Implementation
No Coldkey Storage	Coldkeys never stored in DB, logs, or env
Hotkey Handling	Provided at deployment time only, encrypted in transit
Approval Gates	L1 (auto) / L2 (confirm) / L3 (mandatory)
Audit Logging	All actions logged with user, timestamp, decision
RLS	Row-level security on all user/data tables
No Auto-Spend	Zero automatic financial transactions

Development Commands

```
# Frontend
cd frontend
npm run dev          # Start dev server
npm run build        # Production build
npm run typecheck    # TypeScript check
npm run lint         # ESLint

# Backend
cd backend
uvicorn app.main:app --reload      # Dev server
pytest tests/ -v                   # Run tests
ruff check app/                    # Lint

# Database
psql $DATABASE_URL -f database/migrations/001_initial_schema.sql
cd backend && alembic upgrade head
```

License

Private - Internal Operations Platform

Built for Bittensor Mining Operations

Infranex BT

Architecture Documentation

v2.0 - 3-Pillar Scoring Model

Bittensor Intelligence & Mining Operations Platform

Last updated: 2026-08-29

Private - Internal Operations Platform

Table of Contents

1. System Architecture

2. v2.0 Scoring Engine - 3-Pillar Model

3. Decision Engine - RUN / WATCH / AVOID

4. Scoring API Reference

5. Migration Guide

6. Module Inventory

7. Technology Stack

8. Security Principles

1. System Architecture

Overview

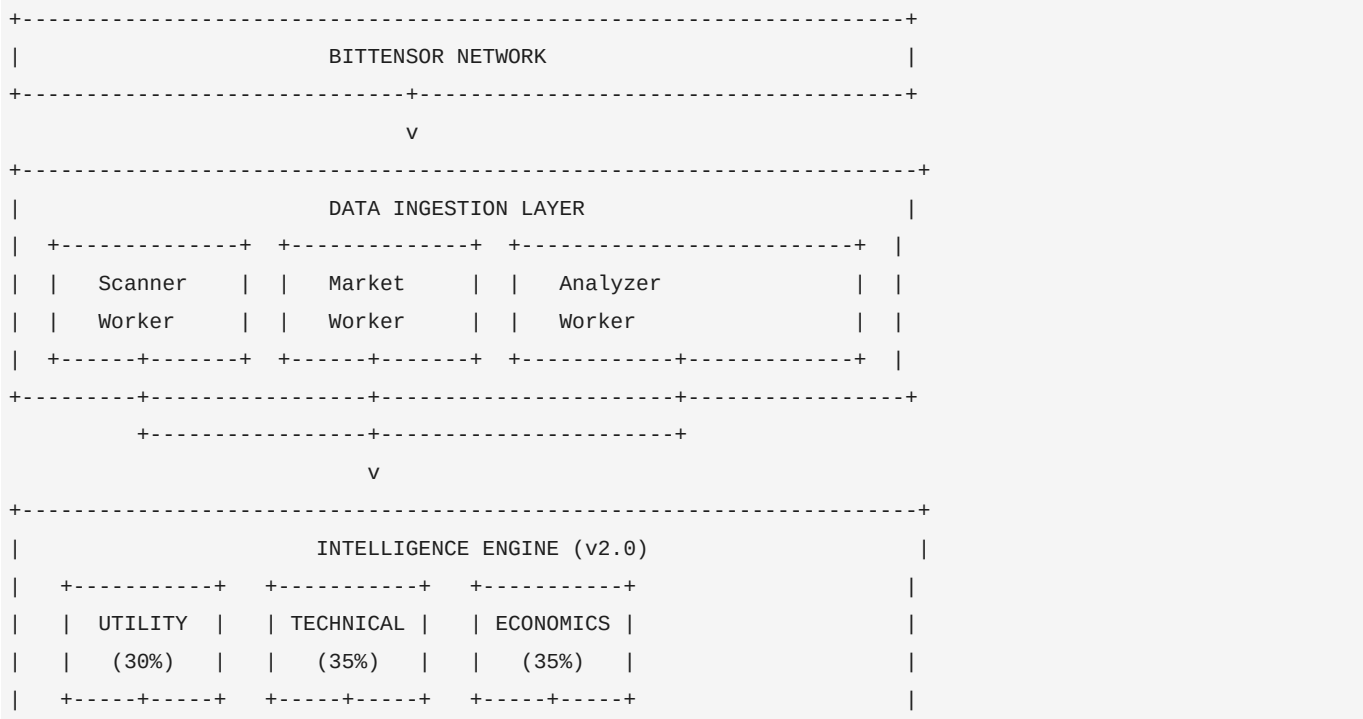
Infranex BT is a production-ready platform for discovering, analyzing, and deploying Bittensor mining operations with GPU infrastructure automation.

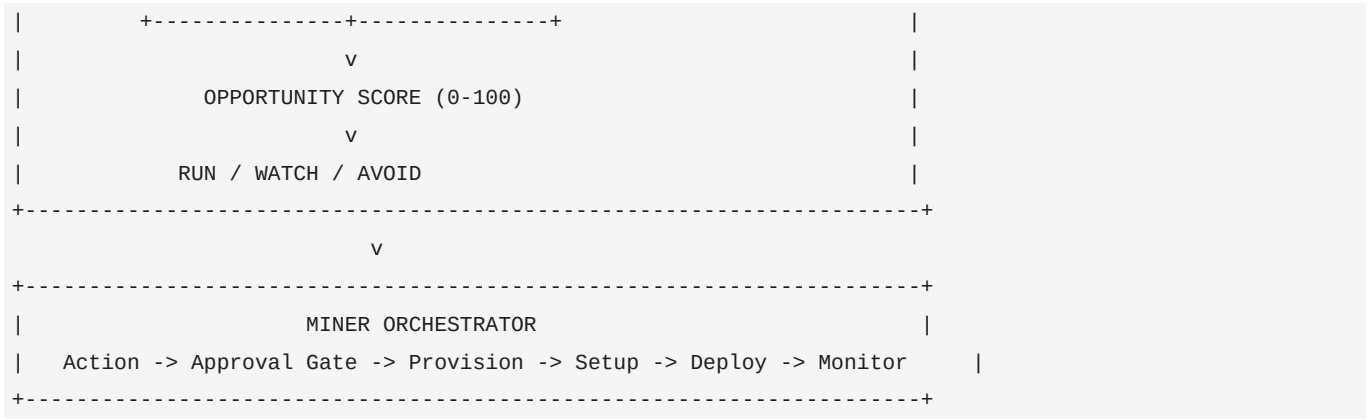
Data Flow

The system follows a layered architecture:

- Data Ingestion Layer - Scanner, Market, and Analyzer workers collect data from Bittensor, CoinGecko, and GitHub
- Intelligence Engine - 3-pillar scoring model computes opportunity scores
- Strategy Engine - Converts scores into RUN/WATCH/AVOID decisions
- Miner Orchestrator - Manages the full miner lifecycle
- DevOps Engine - Provider APIs, deployment, health checks
- Monitoring & Learning - Real-time alerts and feedback loop

Architecture Diagram




OPPORTUNITY SCORE (0-100)
RUN / WATCH / AVOID
MINER ORCHESTRATOR
Action -> Approval Gate -> Provision -> Setup -> Deploy -> Monitor

2. v2.0 Scoring Engine - 3-Pillar Model

Overview

The v2.0 scoring model reorganizes subnet opportunity assessment into three intuitive pillars. Each pillar produces a sub-score (0-100). The weighted combination produces the Opportunity Score.

Pillar 1: Utility (30%)

Question: What does the subnet actually do? Is it a useful service?

Factor	Weight	Source
Problem clarity	30%	README analysis
Uniqueness	20%	Subnet metadata
Description quality	15%	Subnet description
Development activity	15%	GitHub commits
Tokenomics	10%	Subnet registration
Governance	10%	Subnet metadata

Pillar 2: Technical (35%)

Question: Can we actually run this subnet? What hardware do we need?

Factor	Weight	Source
GPU match rate	40%	SubnetRequirement + GPUModel
Dependency simplicity	20%	SubnetRequirement.dependencies
CUDA compatibility	15%	SubnetRequirement.cuda_version
Resource feasibility	15%	RAM, storage requirements
Data confidence	10%	extraction_confidence

Pillar 3: Economics (35%)

Question: Will this subnet be profitable?

Factor	Weight	Source
Revenue potential	30%	Emission * incentive * TAO price
Competition health	20%	top_5_concentration
Reward stability	15%	top vs median incentive spread
Cost efficiency	20%	GPU cost vs expected revenue
Market momentum	15%	Price change, liquidity

Opportunity Score Computation

```
total = (utility * 0.30) + (technical * 0.35) + (economics * 0.35)
decision = RUN if total >= 75 else WATCH if total >= 40 else AVOID
```

3. Decision Engine - RUN / WATCH / AVOID

Decision Bands

Score	Decision	Action
≥ 75	RUN	Deploy miner on this subnet
40-74	WATCH	Monitor for improvement
< 40	AVOID	Do not deploy; exit if running

Portfolio Constraints

Before emitting a RUN decision, the engine checks portfolio constraints:

- Max concurrent miners (default: 10)
- Max monthly spend (default: ₹500,000)
- Max spend per miner (default: ₹50,000)
- Min days between entries (default: 7)
- Allowed subnet categories

Constraint Violation Handling

Violation	Effect
Max miners reached	Downgrade RUN -> WATCH
Monthly spend exceeded	Downgrade RUN -> WATCH
Per-miner spend too high	Downgrade RUN -> WATCH
Too soon since last entry	Downgrade RUN -> WATCH
Category not allowed	Block deployment

Decision Flow

```
1. Compute Opportunity Score
   +-- Utility Score (30%)
   +-- Technical Score (35%)
   +-- Economics Score (35%)
```

```
2. Apply Decision Thresholds
   +-- score >= 75 -> candidate: RUN
   +-- score >= 40 -> candidate: WATCH
   +-- score < 40 -> candidate: AVOID

3. Check Portfolio Constraints
   +-- If RUN and constraints violated -> downgrade to WATCH
   +-- If RUN and constraints OK -> emit RUN
   +-- WATCH/AVOID -> emit as-is

4. Apply Approval Gates
   +-- RUN -> L2_confirm (needs human approval)
   +-- WATCH -> L1_auto (logged, no approval)
   +-- AVOID -> L1_auto (logged, no approval)

5. Emit Action -> consumed by Orchestrator
```

4. Scoring API Reference

Compute Opportunity Score (v2.0)

```
POST /api/v2/opportunities/score

Request:
{
  "netuid": 1,
  "include_components": true
}

Response:
{
  "netuid": 1,
  "subnet_name": "text-prompting",
  "total_score": 72.5,
  "model_version": "v2.0",
  "decision": "WATCH",
  "pillar_scores": {
    "utility": 68.0,
    "technical": 82.0,
    "economics": 65.0
  },
  "components": [...]
}
```

Get Decision

```
GET /api/opportunities/{netuid}/decision

Response:
{
  "netuid": 1,
  "decision": "WATCH",
  "score": 72.5,
  "pillar_scores": { ... },
  "constraints_violated": [],
  "approval_level": "L1_auto"
}
```

List All Decisions

```
GET /api/opportunities/decisions
GET /api/opportunities/decisions?decision=RUN
GET /api/opportunities/decisions?min_score=75
```

Trigger Scoring Run

```
POST /api/opportunities/score-all

Response:
{
  "status": "scoring_started",
  "netuids": [1, 3, 64],
  "estimated_seconds": 30
}
```

5. Migration Guide

v1.0 -> v2.0 Mapping

v1.0	v2.0	Notes
8 components	3 pillars	Simpler, more intuitive
ENTER	RUN	Same threshold (≥75)
HOLD	WATCH	Same band (40-74)
EXIT	AVOID	Same threshold (<40)

Database Migration

```
ALTER TABLE opportunity_scores
  ADD COLUMN utility_score FLOAT,
  ADD COLUMN technical_score FLOAT,
  ADD COLUMN economics_score FLOAT,
  ADD COLUMN pillar_version VARCHAR(10) DEFAULT 'v2.0',
  ADD COLUMN decision VARCHAR(10);

CREATE TABLE utility_assessments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  netuid INTEGER,
  problem_clarity FLOAT,
  uniqueness FLOAT,
  description_quality FLOAT,
  development_activity FLOAT,
  tokenomics_score FLOAT,
  governance_score FLOAT,
  overall_utility FLOAT,
  assessed_at TIMESTAMPTZ DEFAULT NOW()
);
```

Implementation Order

- 1. Database migration - Add new columns and table
- 2. Scoring engine - Implement 3-pillar scoring
- 3. API routes - Add v2 endpoints
- 4. Strategy engine - Update decision labels

- 5. Opportunity service - Persist pillar scores
- 6. Learning engine - Track pillar accuracy
- 7. Frontend types - Update TypeScript interfaces
- 8. Frontend components - Add pillar visualization
- 9. Tests - Write tests for new scoring
- 10. Documentation - Update API docs

6. Module Inventory

Module	Location	Purpose
Intelligence Engine	backend/app/intelligence/	Scoring, GPU matching, profitability, monitoring
Strategy Engine	backend/app/strategy/	Score -> Action decision layer
Miner Orchestrator	backend/app/orchestrator/	Lifecycle state machine
Optimizer	backend/app/optimizer/	Cost savings, subnet alternatives
Recovery	backend/app/recovery/	Health checks, recovery strategies
Learning Engine	backend/app/learning/	Feedback loop, weight adaptation
Compatibility Lab	backend/app/compatibility/	GPU/server requirement validation
Monitoring	backend/app/monitoring/	Real-time alerts, SSE
Providers	backend/app/providers/	GPU provider adapters
Deployment	backend/app/deployment/	Deploy lifecycle, health checks
Approval	backend/app/approval/	L1/L2/L3 approval gates

7. Technology Stack

Layer	Technology
Frontend	Next.js 15, React 19, TypeScript, Tailwind CSS, shadcn/ui, Recharts
Backend	FastAPI, Python 3.11+, Pydantic v2, SQLAlchemy, Alembic
Database	Supabase PostgreSQL (RLS, migrations, realtime)
Auth	Supabase Auth (JWT, OAuth)
Cache	Redis/Valkey
Workers	Python async workers, triggered by Vercel Cron
Containerization	Docker multi-stage builds
CI/CD	GitHub Actions -> Vercel
Bittensor	Official Python SDK (bittensor>=9.0)

8. Security Principles

Principle	Implementation
No Coldkey Storage	Coldkeys never stored in DB, logs, env, or frontend
Hotkey Handling	Hotkeys provided at deployment time only, encrypted in transit
Approval Gates	Every deployment requires explicit user confirmation
Audit Logging	All actions logged with user, timestamp, decision
RLS	Row-level security on all user/data tables
No Auto-Spend	Zero automatic financial transactions

Approval Levels

Level	Meaning	Example Actions
L1_auto	Safe, reversible, no money. Executes immediately.	Refresh metrics, fetch status, score opportunities
L2_confirm	Small financial impact OR reversible operational change.	Register GPU under cap, restart miner, recover container
L3_mandatory	Capital-intensive, irreversible, or strategy-shifting.	Deploy new miner, migrate subnet, increase spend cap

INFRANEX BT · OPERATOR HANDBOOK

CPU Miner Setup Guide

From an empty dashboard to your first live Bittensor CPU miner — connect a Hetzner Cloud or DigitalOcean key, rent a ~$5–9/mo VPS, let cloud-init install the base stack, and finish the gated launch in DevOps Engine. Worked example: subnet 61 (RedTeam), no GPU anywhere, roughly 95% cheaper to host than the GPU path.

SN61 · REDTEAM | NO GPU NEEDED | HETZNER CLOUD · DIGITALOCEAN | HOST FROM $4.63/MO | AUTO-INSTALL FROM GIT

START HERE

The CPU path in one page

This guide is written for a first-time miner. It walks you through the exact clicks and commands that take you from a fresh sign-in to a running CPU miner on a Bittensor subnet — using subnet 61 (RedTeam) as the worked example. Every screen shot was taken from the live platform; the two figures marked *example data* use a representative offer so you can see what the catalog looks like once your own API key is connected.

The CPU path exists because a large family of Bittensor subnets do their work with ordinary processors, not graphics cards. The platform's work-type classifier reads each subnet's public git repo — its README, `requirements.txt` and Dockerfile — and files it as CPU-classified, GPU-classified, or CPU VPS-class. For those CPU-classified subnets, renting a GPU is wasted money: the typical CPU-classified subnet needs about 2 vCPU / 4–8 GB RAM / 40–50 GB disk, which costs $5–9 per month, while the cheapest useful GPU box (an RTX 3090) runs around $161 per month. Same chain, same rewards table, a fraction of the overhead.

The journey at a glance

STEP 1 · FREE

Connect a provider

paste a Hetzner or DigitalOcean API key — offers refresh live every 60s

STEP 2 · FREE

Pick a box

catalog suggests the cheapest box that actually fits the subnet

STEP 3 · COSTS

Rent & auto-install

provider creates the box; cloud-init installs docker + bittensor

STEP 4 · COSTS

Finish in DevOps

validate host, copy wallet files yourself, register, approve launch

ALWAYS ON · FREE

Monitor

My Miners + Monitoring track health, cost and P&L

The worked example used throughout

Worked example — SN61 RedTeam on a Hetzner CX32. RedTeam (α61) is a CPU-classified cybersecurity subnet by Innerworks. Its repo documents 2 cores / 8 GB RAM / 50 GB disk. The cheapest Hetzner box that comfortably clears that bar is the **CX32** (4 vCPU / 8 GB / 80 GB) at about **$8.32/mo**. You could squeeze onto a floor-spec box (CX22, 2 vCPU / 4 GB, ~$4.63/mo), but RedTeam-class boxes are the honest recommendation and they are what the catalog's "RedTeam-class box" card targets. Every screenshot in Parts A–B follows this one box, end to end.

Where the money moves

Move	Amount	What it is
You → Hetzner / DigitalOcean	$4.63–8.32/mo	The CPU box. Metered hourly (~$0.006–0.011/hr), billed monthly. Runs until you destroy it — teardown costs nothing extra.
Coldkey → chain burn	one-off recycle fee	Registering your hotkey on SN61. The app shows the **live burn figure** on the CPU Guide (05) and Subnets (03) views — it floats with seat demand, so check it before you register.
Coldkey mnemonic	—	Never leaves your machine. You copy the wallet files to the box yourself, in an approval-gated step. The platform never ships coldkeys.
Platform API keys	$0	Connecting a key is free — you only pay the provider when a box is actually created.

Before you start — have these ready

- [ ] Your **platform sign-in** (operator ID + access code) and 10 minutes of focus time
- [ ] An account at **Hetzner Cloud** or **DigitalOcean** with billing enabled — a few dollars of credit is plenty to start
- [ ] **Python 3.9+** on your laptop for btcli (wallet + registration commands run locally, not on the box)
- [ ] **TAO in your coldkey** for the one-off registration burn, plus a small buffer
- [ ] Paper and a pen — if you create a new wallet, the mnemonic is shown **once**

CPU path or GPU path?

Question	CPU path (this guide)	GPU path (companion guide)
Subnet type	CPU-classified — e.g. SN61 RedTeam, SN67 Harnyx script mining	GPU-classified — e.g. SN64 Chutes, SN36 Epago (vision)
Host cost	~$5–20/mo (2–8 vCPU VPS)	~$161+/mo (RTX 3090 or better)
How you get the box	CPU Catalog → Rent & auto-install, or manual VPS in Part C	GPU Catalog → deploy wizard
Base stack install	Automatic (cloud-init) on the platform path	Automatic after pod start
Wallet & launch	Identical on both paths: approval-gated in DevOps Engine — the platform never touches your coldkey.	

About the screenshots. Figures 1–5 and 10–12 are untouched captures of the live platform. Figures 6–9 were captured against a representative offer (a Hetzner CX32 at $8.32/mo) so the catalog, rent dialog and success screens could be shown without spending your money — with your own key connected, you will see your real prices in the identical UI.

PART A · ~15 MINUTES · $0

Platform Prep — Sign In & Connect a CPU Provider

Three goals: get into the platform, create an API key at a CPU provider, and hand that key to the CPU Catalog. Nothing here costs money — the provider only bills you when a box is actually created, which happens in Part B when you choose to click Rent.

A1 · Sign in and land on the dashboard

A1 Open the app and sign in

Open the preview link in your browser, then enter your operator User ID and access code and press Sign in. If the page was left open from a previous session, the app restores you automatically.

After sign-in you land on the **Dashboard (01)** — the opportunities table scans the chain and lists subnets with their classification, cost target and hardware hints. You do not need to configure anything here, but it is worth glancing at: every CPU-classified subnet you see is a candidate for this guide.

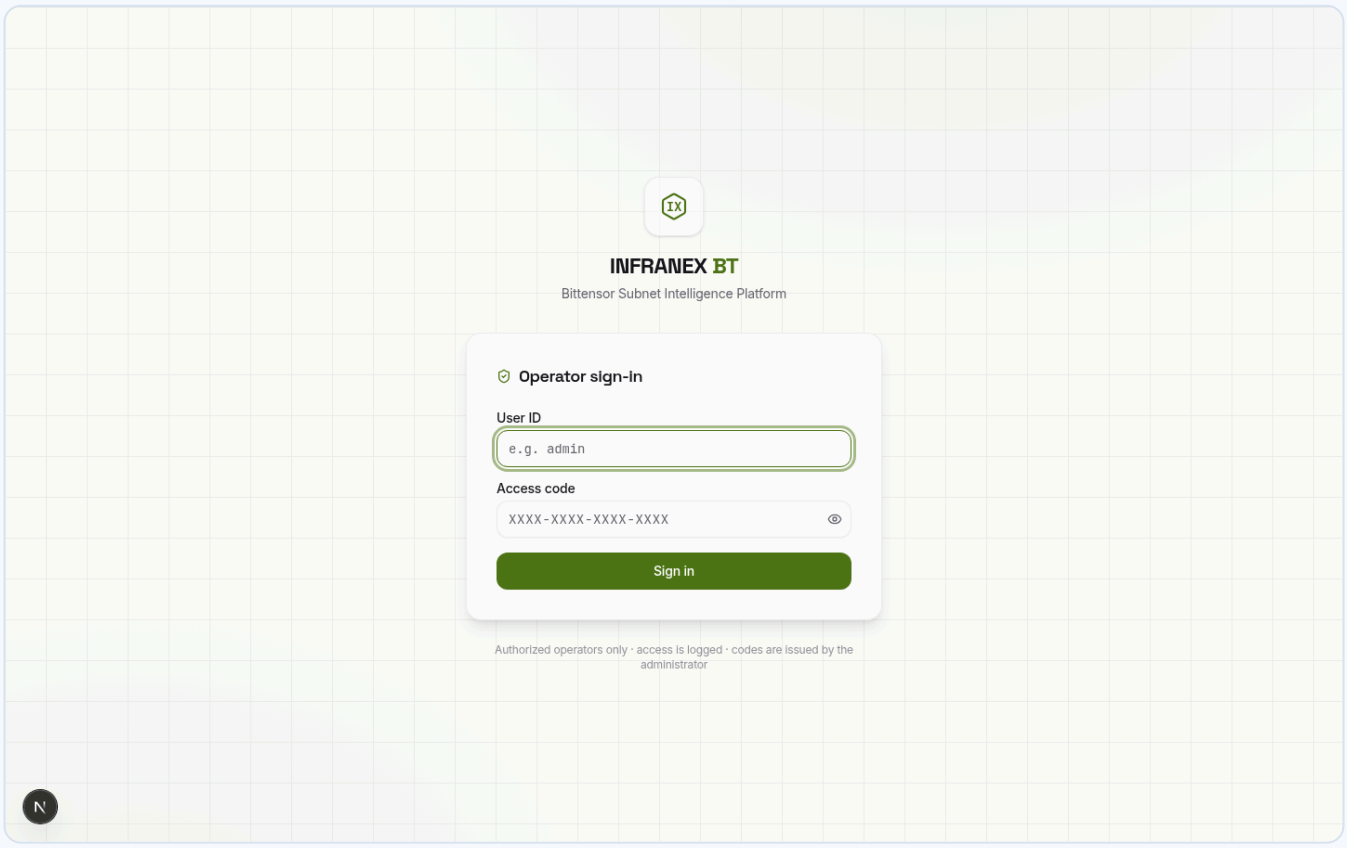


Figure 1. The operator sign-in screen. Your access code is per-user; the app keeps sessions and audit-logs every action.

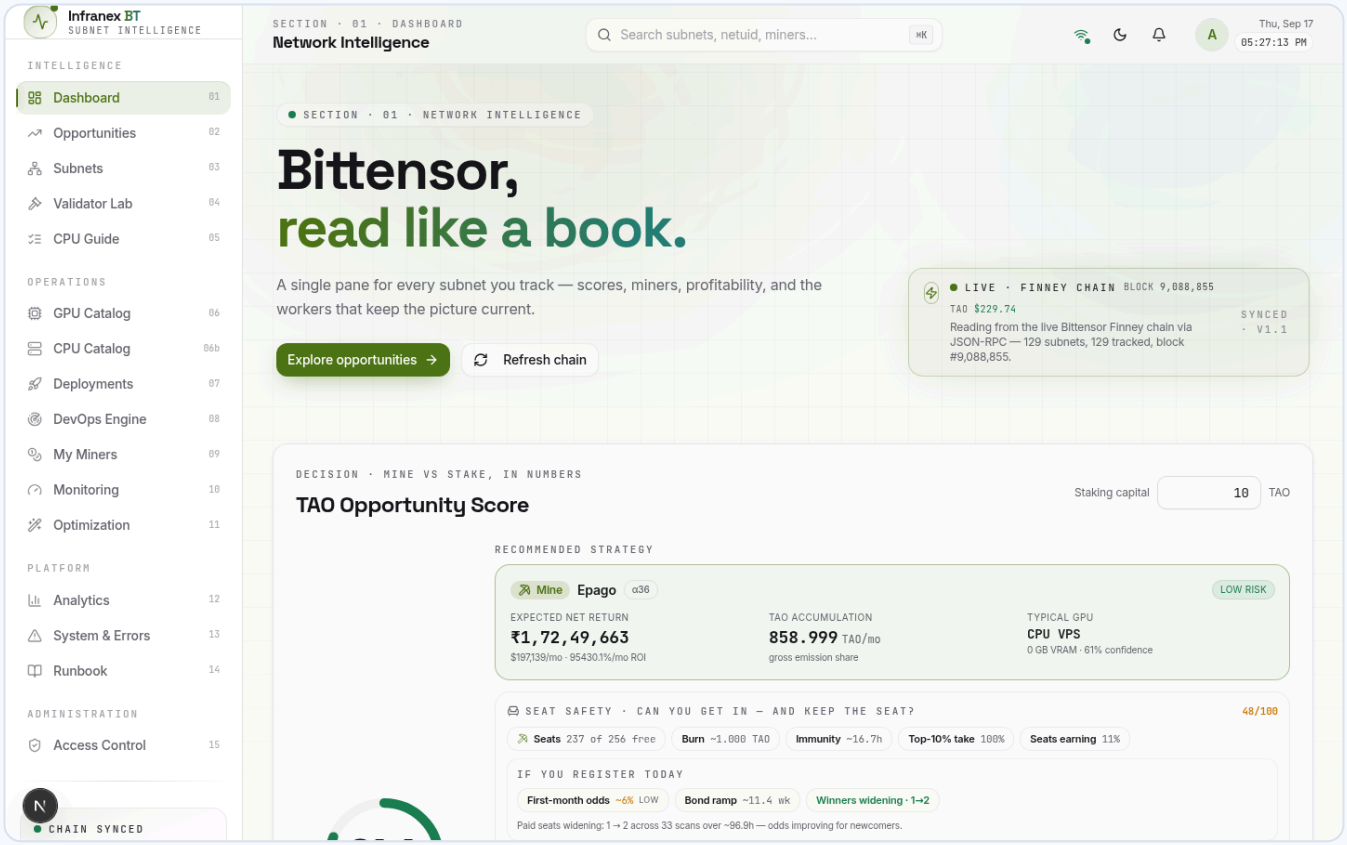


Figure 2. The Dashboard (01) after sign-in — the opportunities table on the right is where CPU-classified subnets surface first.

A2 · Create an API key at your provider

The CPU Catalog speaks to two providers: **Hetzner Cloud** (best prices, EU/US datacenters) and **DigitalOcean** (worldwide regions, slightly pricier). Either works — this guide follows Hetzner for the worked example, and the DigitalOcean steps are identical in shape.

A2a Create a Hetzner Cloud API token (worked example)

console.hetzner.cloud → select or create a project → **Security** → **API tokens** → **Generate API token**

- Choose the **Read *and* Write** permission — a read-only token can show prices but cannot create the server when you click Rent.
- Copy the token immediately — Hetzner shows it exactly once. If you lose it, delete the token and generate a new one.
- The token belongs to one project; make sure the project is in a billing-enabled state (a payment method or the free trial credit both count).

A2b Or create a DigitalOcean API token

cloud.digitalocean.com → **API** → **Generate New Token**

- Give it **Full Access** (write scope) for one-click renting, or read-only if you only want live pricing.
- Pick an expiration you will remember — when the token expires, offers silently disappear until you paste a fresh one.

Handle the key like a password. The token can create and destroy servers in your account. Paste it only into the platform's Provider API keys dialog (next step) — never into email, chat, or a shared document. You can revoke and rotate it at any time from the same provider page.

A3 · Connect the key in the platform

A3 Paste the key into Provider API keys

In the sidebar under **Operations**, open **CPU Catalog · 06b** and press **Provider API keys** (top-right). The dialog lists both CPU providers with a **Real rental** badge — that means the platform can both show live pricing *and* create the server for you.

Paste your Hetzner token into its field and press **Save & verify**. The key is validated the moment you save it — a green status pill appears when the provider accepts it. Repeat for DigitalOcean if you created a token there too. Within seconds the catalog pulls live VPS pricing; offers keep refreshing every 60 seconds from then on.

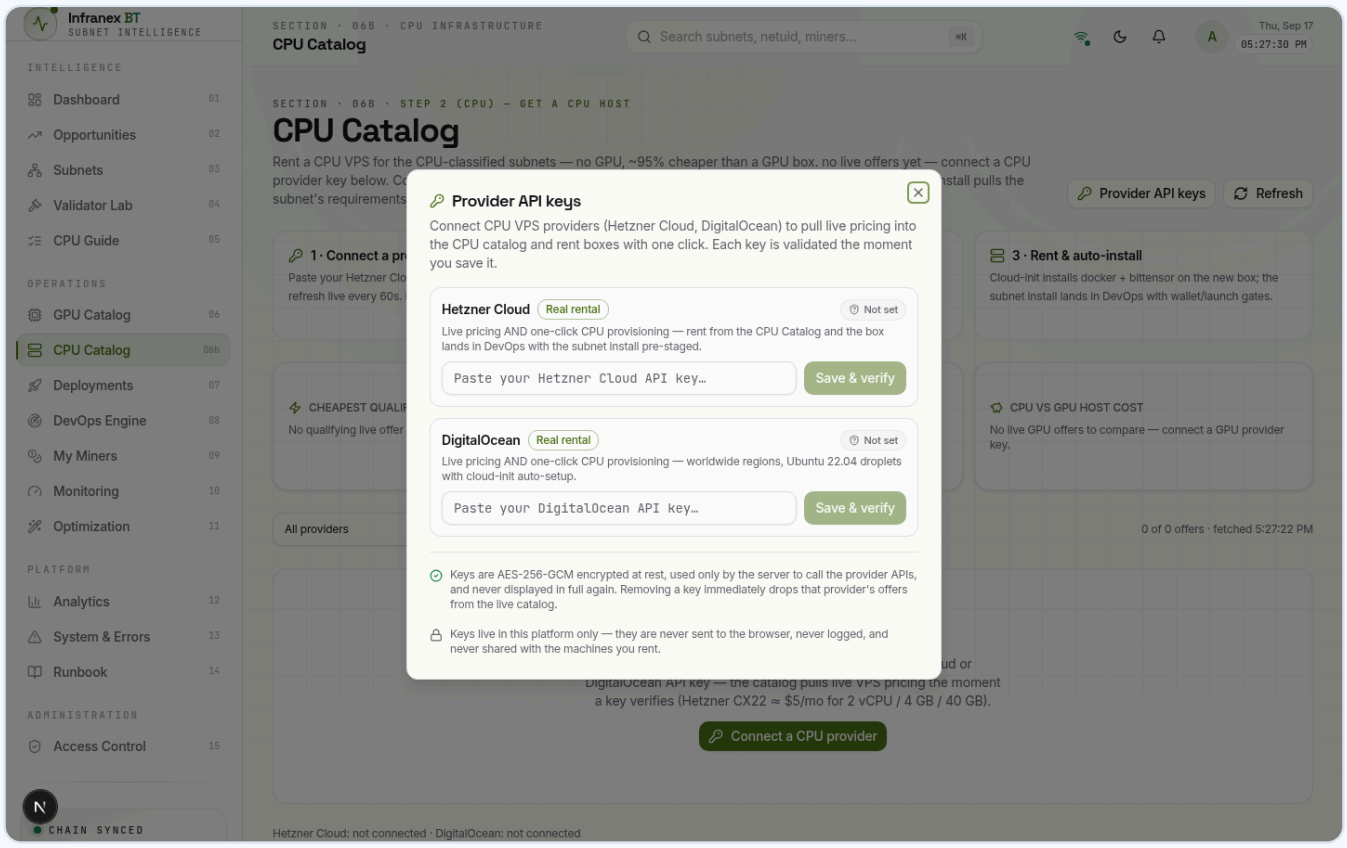


Figure 3. The Provider API keys dialog in its CPU mode: Hetzner Cloud and DigitalOcean, each with a paste field and a Save & verify button. Status pills show whether a stored key is valid.

How the key is stored. Keys are AES-256-GCM encrypted at rest on the server, used only by the server to call provider APIs, and never displayed in full again. Removing a key immediately drops that provider's offers from the live catalog. Keys are never sent to the browser, never logged, and never shared with the rented machines — the box gets a separate ephemeral SSH key at rent time.

PART A · CONTINUED · $0

Reading the CPU Catalog

The catalog is honest by design: before a key is connected it shows you exactly what is missing, and after connecting it only recommends boxes that can actually run the subnet's code. Learn the two states once and you can always tell whether you are looking at real supply.

B1 · The two states of the catalog

Without a key (Figure 4) the catalog says so plainly: "No CPU provider connected yet", the three recommendation cards read "No qualifying live offer", and the provider status line at the bottom shows each provider as `not connected`. The single green button — **Connect a CPU provider** — opens the same dialog you used in A3.

With a key (Figure 5) the same page fills with supply: the header counts the live offers and connected providers, the cards show the cheapest qualifying box and the current RedTeam-class pick, and the offers table lists every rentable VPS with vCPU, RAM, disk, region and price.

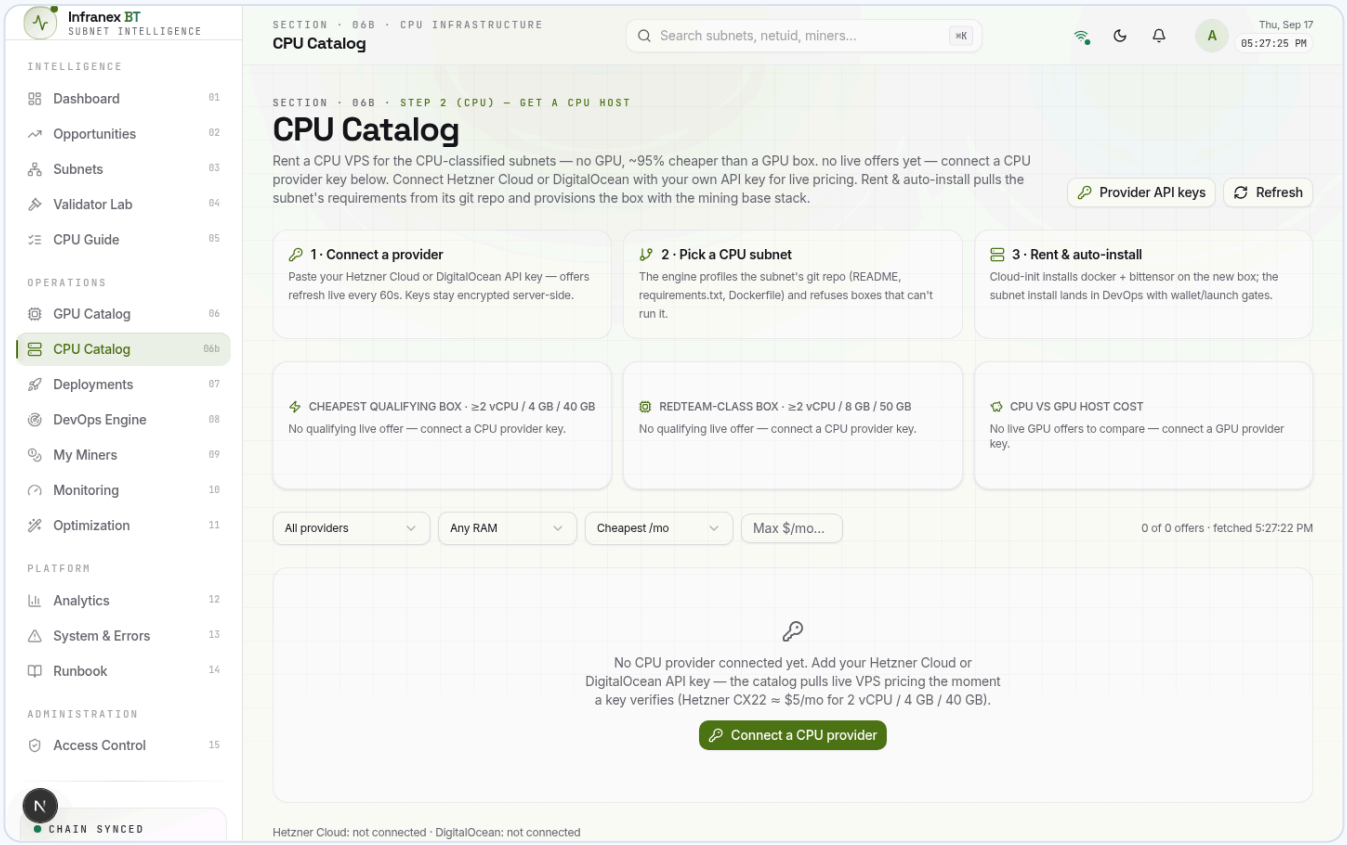


Figure 4. The catalog before any key is connected — the empty state names the missing piece and the expected price (Hetzner CX22 ≈ $5/mo) so you can tell honesty from a bug.

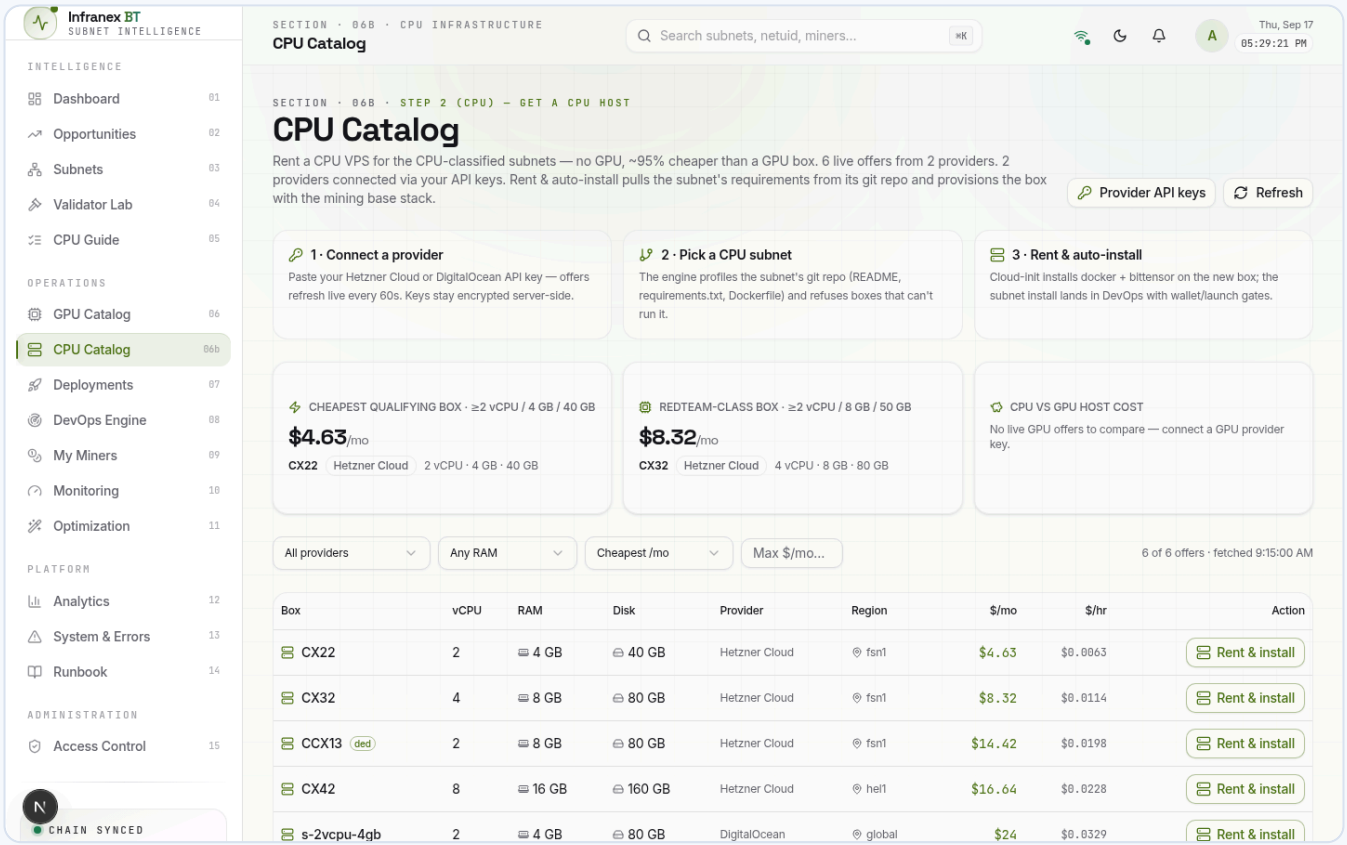


Figure 5. The catalog with a connected key — *example data* (a representative Hetzner + DigitalOcean snapshot). With your own key you will see your real, live prices in this exact layout.

B2 · The three recommendation cards

B2 How to read the cards above the table

- **Cheapest qualifying box · ≥2 vCPU / 4 GB / 40 GB** — the absolute floor for CPU-classified subnets. Real money-saver, but only choose it if your subnet is this light; a heavy subnet on a floor box will run slow or get refused by the profiler.
- **RedTeam-class box · ≥2 vCPU / 8 GB / 50 GB** — the spec SN61 RedTeam documents (2 cores / 8 GB / 50 GB disk). This is the safe default for the worked example, and for many security/agent subnets.
- **CPU vs GPU host cost** — a live comparison: the CPU box's monthly price next to the cheapest live GPU offer, and the percentage you are saving. Connect a GPU provider key too if you want this card populated.

B3 · Filters and choosing a box

Above the table sit four filters: **All providers**, **Any RAM** (raise to ≥ 8 GB for RedTeam-class), **Cheapest /mo** (or sort by vCPU/RAM/disk), and a `Max $/mo` box for a hard budget cap. The counter on the right ("6 of 6 offers · fetched 9:15:00 AM") confirms the data is live.

Box	Spec	~$/mo	Verdict for SN61 RedTeam
CX22	2 vCPU · 4 GB · 40 GB	$4.63	At the floor — fits only the lightest CPU subnets; below RedTeam's 8 GB RAM guidance.
CX32	4 vCPU · 8 GB · 80 GB	$8.32	**The worked example.** Clears RedTeam-class with headroom at a hobbyist price.
CCX13 (ded)	2 vCPU · 8 GB · 80 GB	$14.42	Dedicated cores — pick when a shared-tenancy box shows CPU-steal in Monitoring.
CX42	8 vCPU · 16 GB · 160 GB	$16.64	Overkill for SN61; sensible for running several CPU miners on one box.
DO s-2vcpu-4gb	2 vCPU · 4 GB · 80 GB	$24.00	DigitalOcean's entry tier — pick it only when you need a specific DO region (e.g. blr1, sgp1).

The profiler is your safety net. When you later pick a subnet in the rent dialog, the engine profiles the subnet's git repo and **refuses** two things outright: subnets that document GPU requirements ("pick a CPU-classified subnet or rent this one from the GPU Catalog"), and boxes below the subnet's needs. Refusals are the platform protecting you from paying for a box that can never earn.

PART B · ~10 MINUTES · PROVIDER BILLING STARTS

Rent & Auto-Install

This is the one click that creates a real server in your provider account. Everything about the flow is designed so you see exactly what will be installed before any money is committed — the subnet's requirements are pulled from its git repo and displayed first, and the wallet/launch decisions stay gated until Part D.

C1 · Open the rent dialog

C1 Pick the row, press Rent & install

Find your chosen box in the offers table and press **Rent & install** on its row. The **Rent CPU box & auto-install** dialog opens with the offer pinned at the top — model, provider, vCPU/RAM/disk and the monthly price next to its hourly equivalent (monthly ÷ 730).

For the worked example: the CX32 row — 4 vCPU · 8 GB · 80 GB, $8.32/mo ≈ $0.0114/hr. Billing starts the moment the provider accepts the create call, and stops the moment you destroy the server.

C2 · Walk through the dialog, top to bottom

C2a Subnet to install (CPU-classified)

The dialog offers one-click chips for every CPU-classified subnet the scanner currently knows — for the worked example, **SN61 · RedTeam**. You can also type any netuid into the input (e.g. 61). Choose the subnet you intend to mine; the chip you pick decides what gets profiled and staged.

C2b Requirements pulled from the subnet's git

The moment a subnet is selected, the engine fetches its public repo artifacts — chain identity, README, `requirements.txt`, Dockerfile — and prints the verdict right in the dialog: subnet name and category, a **confidence** badge (high/medium/low), repo URL, entrypoint, axon port, python version, pip dependency count and the apt packages it will install. For SN61 the panel shows **RedTeam · Security · high confidence**, entrypoint `neurons/miner.py`, axon `:8091`.

Read this panel before you click anything else — it is the contract of what the box will run. If the subnet turns out to document GPU requirements, the dialog refuses to continue and tells you to use the GPU Catalog instead.

C2c Region, wallet name, hotkey name

- **Region** — pick a datacenter close to the chain's validators. Hetzner defaults to Falkenstein DE (fsn1); DigitalOcean offers nine regions from New York to Bangalore. Any choice works; lower validator latency never hurts scores.
- **Wallet name** — your existing coldkey's name (usually `default`), created on your laptop with btcli. This is only a label used later in the install plan.
- **Hotkey name** — a fresh name for this miner, e.g. `miner-cpu`. The dialog accepts letters, digits, dashes and underscores.
- The **shield note** at the bottom of the form is worth reading twice: "Wallet files are copied by YOU in the gated DevOps step — the platform never ships coldkeys to rented machines."

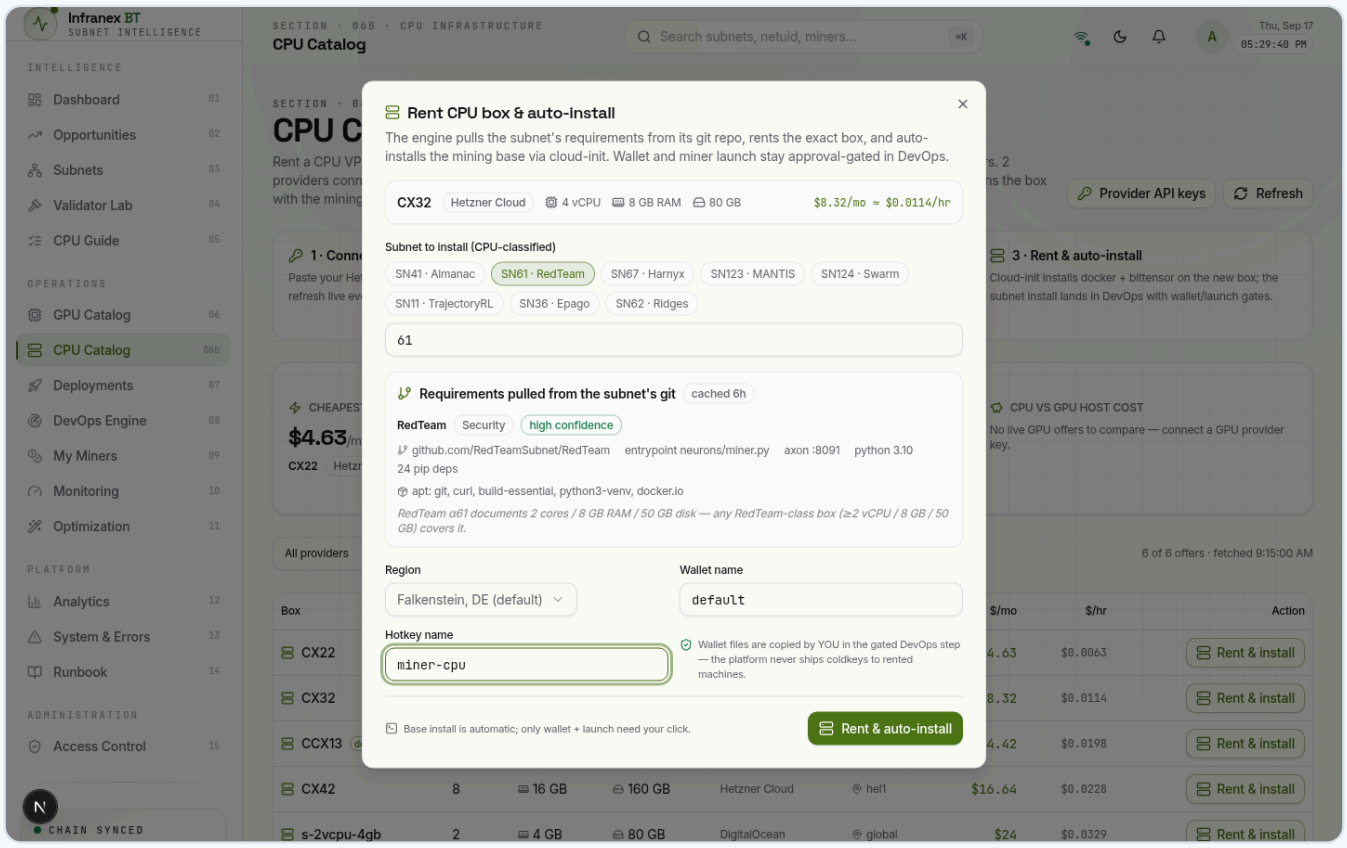


Figure 6. The rent dialog, fully filled for the worked example — *example data*. CX32 pinned at top, SN61 selected from the chips, the git-pulled requirements panel showing high confidence, region Falkenstein, wallet `default`, hotkey `miner-cpu`.

C3 · Click Rent & auto-install

C3 The button does four things in sequence

Pressing **Rent & auto-install** triggers the pipeline: **(1)** the platform calls your provider with the exact offer — CX32, fsn1 — and injects a fresh ephemeral SSH key; **(2)** cloud-init runs on first boot and installs the mining base stack (docker, python venv tooling, bittensor); **(3)** the box is registered as a DevOps host over SSH transport; **(4)** the subnet's install plan is staged on it, waiting for you in DevOps Engine.

The success panel confirms the rental — server id, status, IP address (or "IP pending" for the first seconds), monthly price and the DevOps host name — and lists the three things that happen next. None of them need you to be fast; give the base install 3–5 minutes after the IP appears.

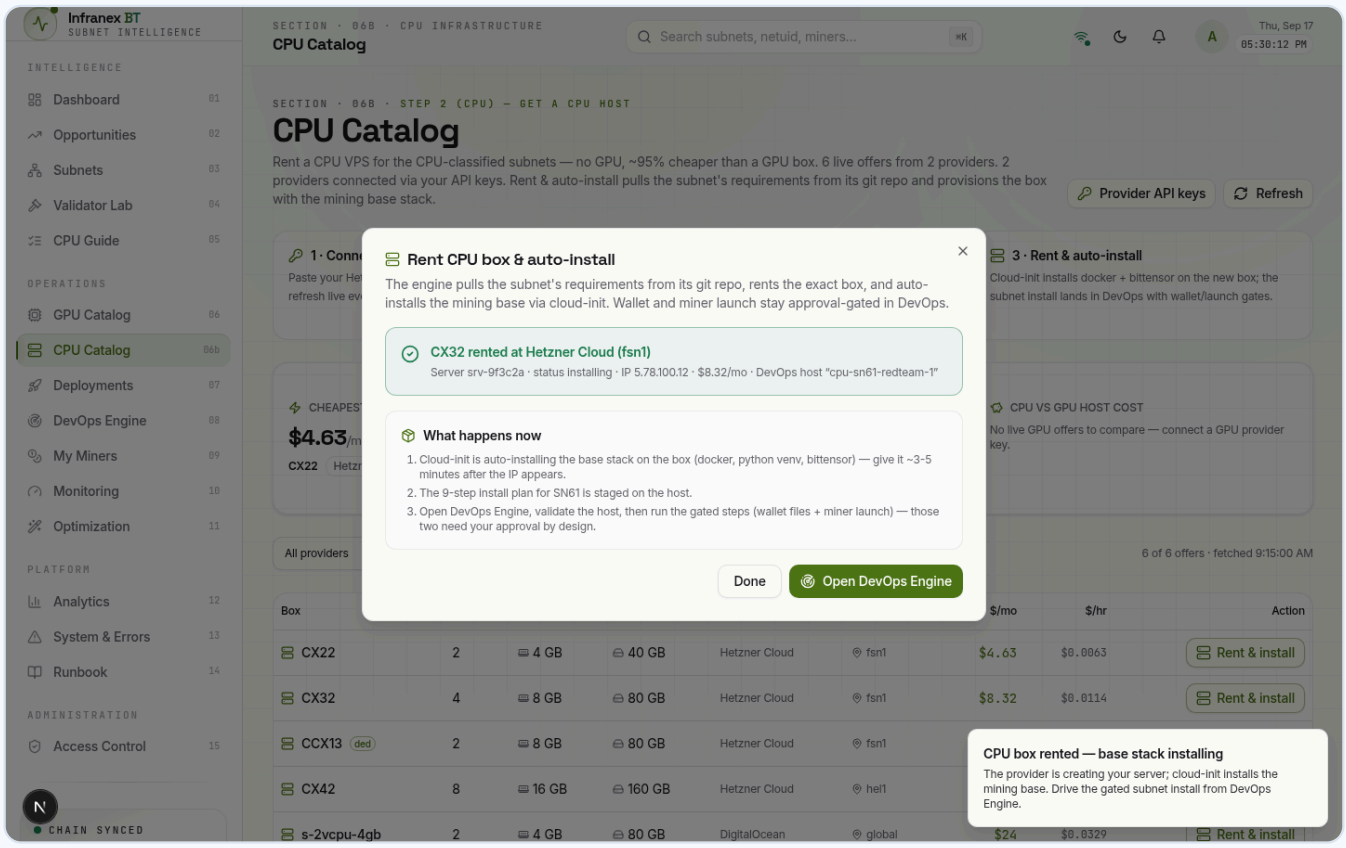


Figure 7. The success panel — *example data*. "CX32 rented at Hetzner Cloud (fsn1)" with server id, IP, price and the DevOps host name, plus the "What happens now" list and a shortcut into DevOps Engine.

What cloud-init installs, exactly. The base stack is deliberately minimal: docker (for containerized miners), python3-venv (for pip-based miners), and bittensor (btcli + SDK). Your subnet's own dependencies are *not* pre-installed globally — they are handled by the subnet's staged install plan, which you review and approve in Part D. This keeps the box clean and the audit trail honest.

PART B · CONTINUED · ONE-TIME BURN IN D2

Finish in DevOps Engine

DevOps Engine (08) is the operations cockpit where the rented box becomes a miner. The install plan arrives pre-staged, but three steps stay deliberately manual: validating the host, copying your wallet files, and approving the miner launch. This is the platform's hotkey-only policy doing its job.

D1 · Validate the new host

D1 Open DevOps Engine and find the host

From the success panel press **Open DevOps Engine**, or pick **DevOps Engine · 08** in the sidebar. Your new host appears with its provider, region, IP and status. Press **Validate** to have the platform SSH in and confirm the base stack landed: docker version, python venv, btcli. Validation turns the host green and unlocks the install plan steps beneath it.

D2 · The gated steps — wallet files, registration, launch

D2a Create (or confirm) your wallet on your laptop — free, do this before burning

If you already have a Bittensor coldkey, skip to D2b. Otherwise install btcli locally and create the wallet pair now. This runs on your machine, touches no money, and the mnemonic never leaves your laptop:

YOUR LAPTOP · TERMINAL — BTCLI · WALLET PREP

```
$ pip install bittensor
$ btcli wallet new_coldkey --wallet.name default
  mnemonic: olympic liquid piano ...   ← shown ONCE — write it on paper
$ btcli wallet new_hotkey --wallet.name default --wallet.hotkey miner-cpu
$ btcli wallet overview --wallet.name default
  coldkey: default    balance: 0.000 TAO    hotkeys: 1
```

D2b Copy the wallet files to the box yourself

DevOps Engine shows a **Copy wallet files** gate with the exact destination. The platform never moves coldkeys — you do, with your own scp command (the host record carries the SSH details). Copy only the hotkey file plus the coldkeypub.txt public file if you can; keep the sealed coldkey file on your laptop unless the subnet's plan genuinely needs it.

YOUR LAPTOP · TERMINAL — WALLET TRANSFER · GATED STEP

```
$ scp -r ~/.bittensor/wallets/default root@<box-ip>:/root/.bittensor/wallets/
# keep coldkeyseal (the sealed private file) local unless the plan requires it
```

D2c Register the hotkey on SN61 — the ONE burn

Registering binds your hotkey to subnet 61 and consumes a dynamic recycle fee — irreversible, sized by seat demand. Check the live figure first: the CPU Guide (05) view and the Subnets (03) view both show SN61's current burn cost. Then run:

YOUR LAPTOP · TERMINAL — ONE-TIME CHAIN BURN

```
$ btcli subnets register --netuid 61 --wallet.name default --wallet.hotkey miner-
cpu
  ✅ Registered hotkey miner-cpu on netuid 61 (recycle burned)
```

No green verdict, no burn. If you plan to mine SN61 the "proper" way — building a competitive agent — use the in-app CPU Guide (05) first: it profiles the champion, lets you spar against Validator Lab for free, and only then sends you to the register step. Registration burns TAO regardless of whether your miner ever earns, so treat it as the point of no return that it is.

D2d Approve the miner launch

The final gate — **Launch miner** — runs the subnet's staged install plan: clone the repo, install its pinned dependencies, write the environment (wallet/hotkey names, netuid, network), and start the miner process under supervision. Read the step list, then approve. DevOps shows the live step counter until the last step completes.

D3 · Why these steps are gated

Everything destructive or secret-bearing — moving wallet files, burning TAO, starting a process — requires an explicit human click, and every action is written to the audit log. That is not caution theater: on the GPU path the same gates exist, and they have prevented more than one mis-clicked deployment from burning a seat on the wrong subnet. The platform automates the parts that are safe to automate (provisioning, base install, dependency wiring) and keeps you in command of the parts that are not.

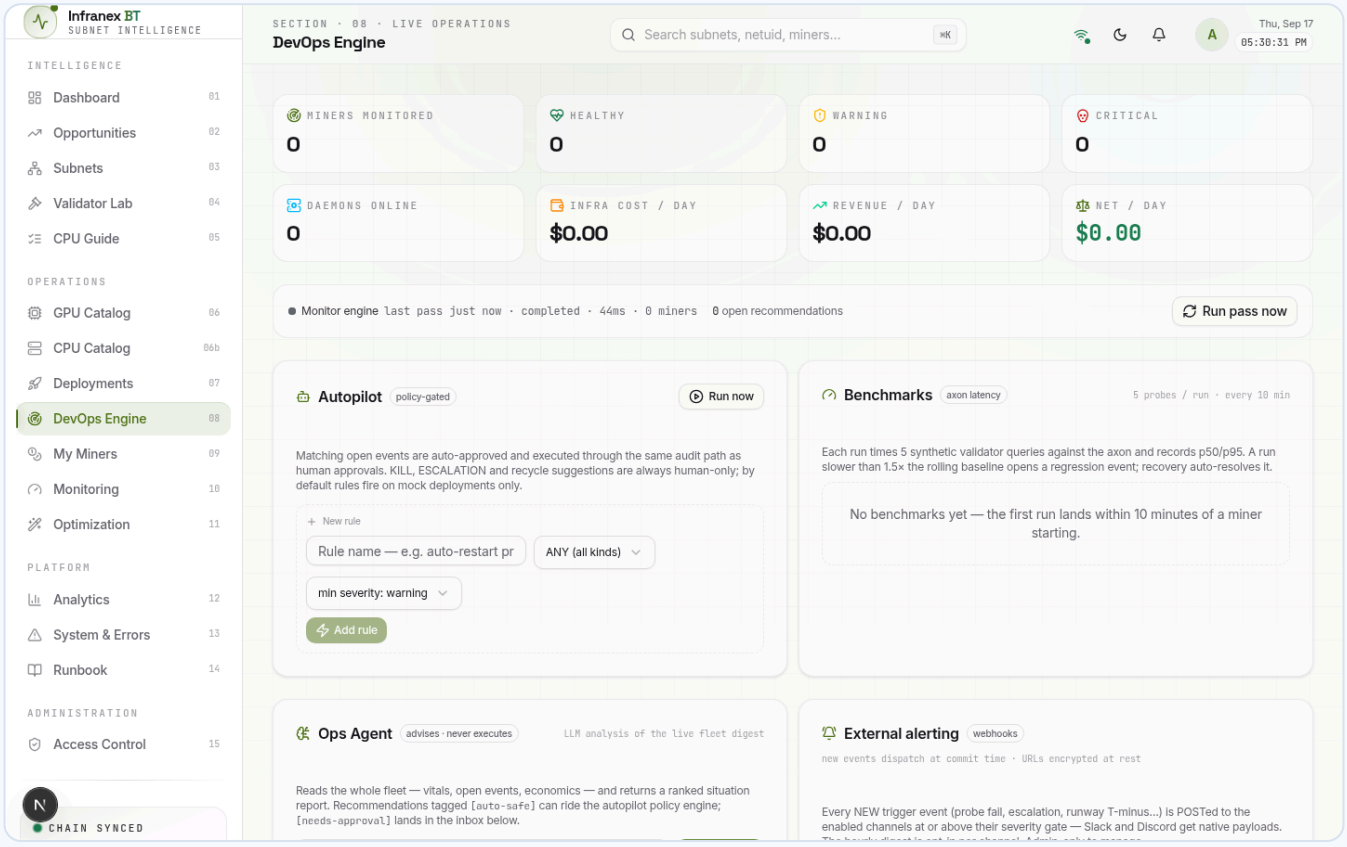


Figure 8. DevOps Engine (08) — the ops cockpit: fleet vitals up top, Autopilot policy rules and Benchmarks on the right. Your rented host and its staged install plan appear here after the rent flow completes.

Verify You Are Mining

Two layers of verification: the platform's (My Miners and Monitoring) and the box's own (processes and ports). Check both once after launch, then let the monitoring layer do its job.

E1 · Check the platform views

E1 My Miners (09) and Monitoring (10)

My Miners lists your miner with its subnet, host and status — after a successful launch the miner should show as running on SN61. **Monitoring** adds the health timeline: axon latency probes every 10 minutes, CPU and memory gauges, and cost tracking that reads the box's real hourly price. If a probe fails or the box drifts, Monitoring opens a recommendation instead of failing silently.

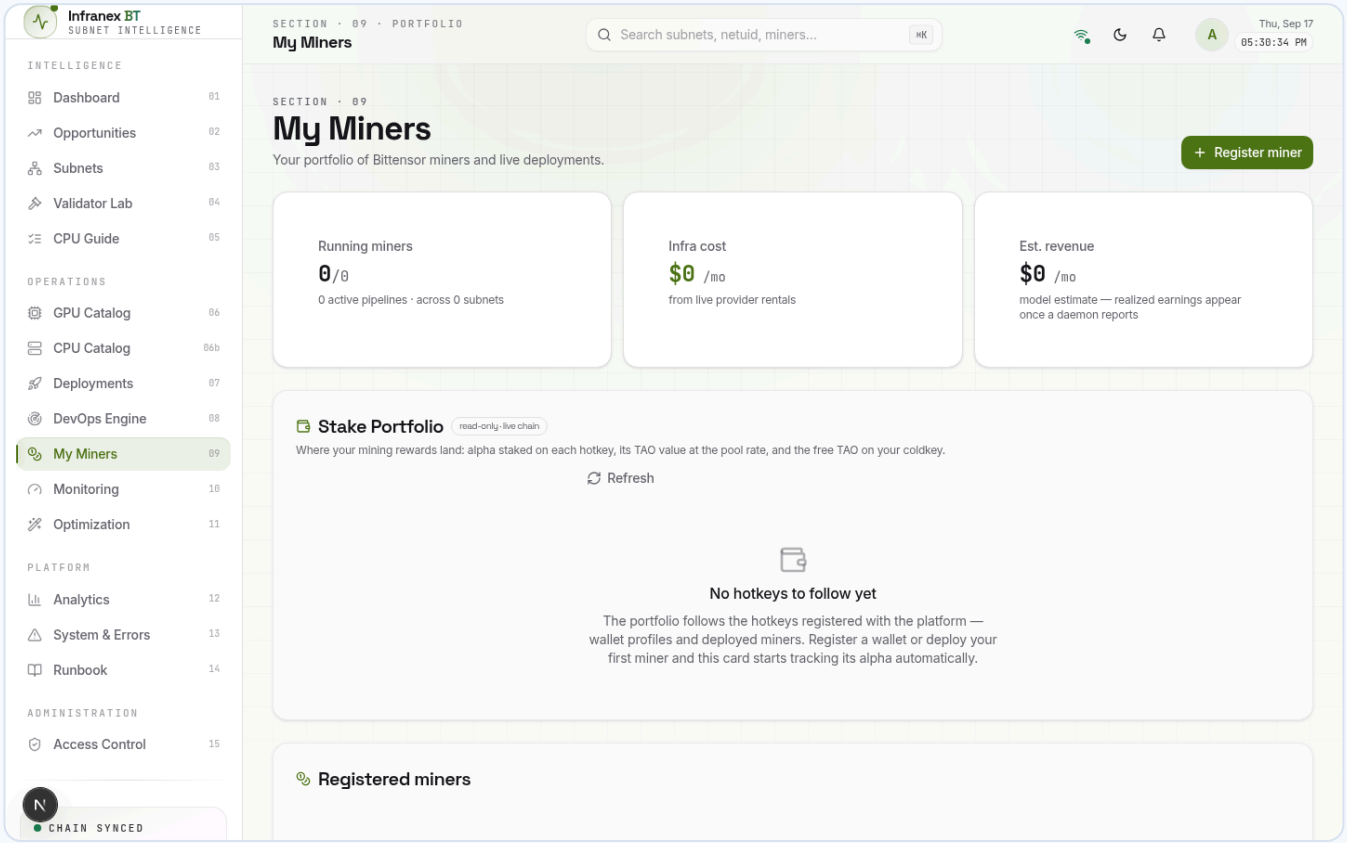


Figure 9. My Miners (09) — one row per running miner, with subnet, hotkey, host and status.

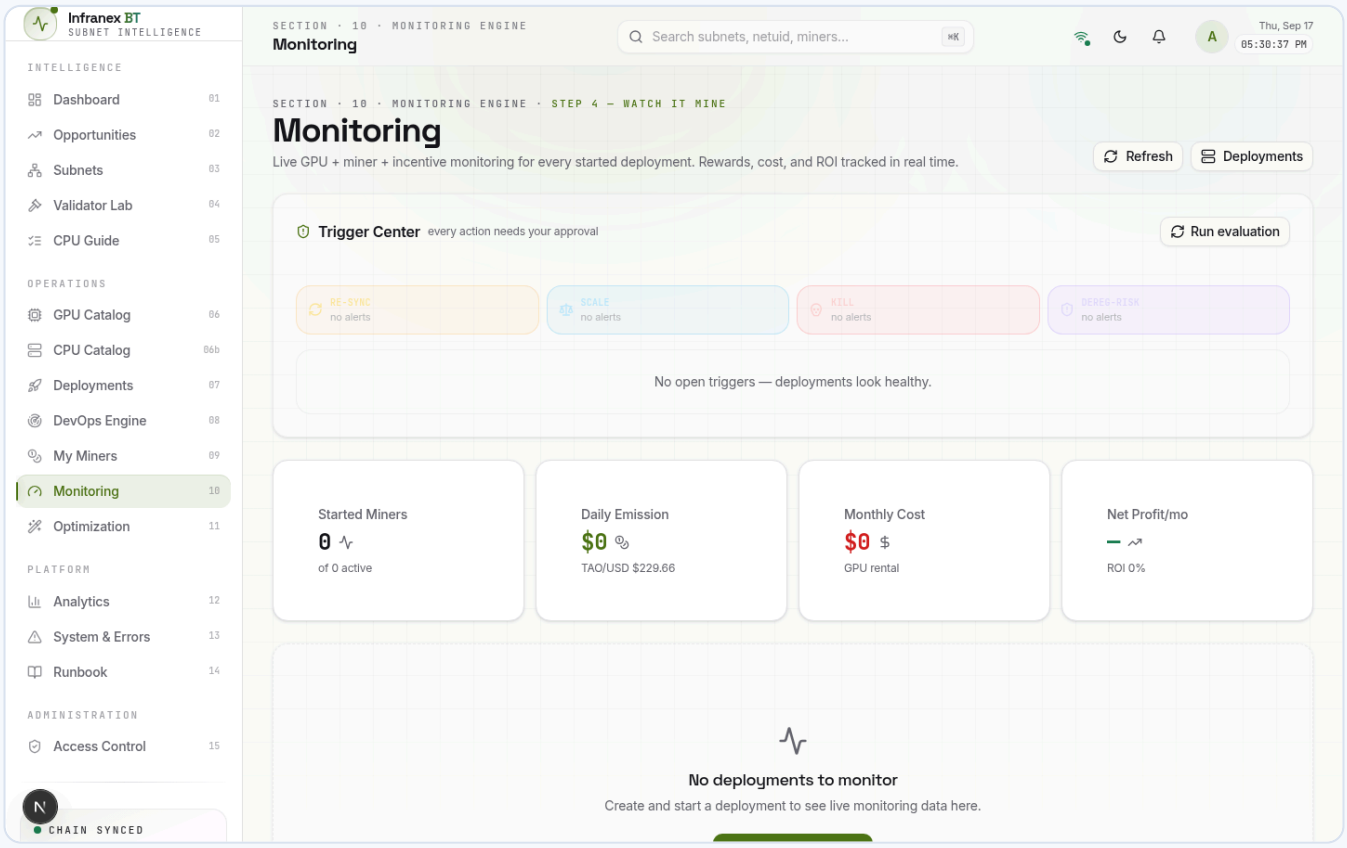


Figure 10. Monitoring (10) — axon latency benchmarks, resource gauges and cost/day for every live box.

E2 · Check the box itself

YOUR LAPTOP · TERMINAL — SSH · SPOT CHECKS

```
$ ssh root@<box-ip>
# is the miner process alive?
$ screen -ls
  There is a screen on:  1 Socket in /run/screen/S-root.
# is the axon port listening?
$ ss -tlnp | grep 8091
  LISTEN 0 4096  *:8091  *:*  users:(("python",pid=1844,fd=18))
# does the chain see your hotkey on SN61?
$ btcli wallet overview --wallet.name default --network finney
# what did the miner log recently?
$ tail -n 40 /var/log/miner/sn61.log
```

E3 · What "earning" honestly looks like

CPU subnet rewards follow emissions, competition and — for work subnets like RedTeam — the quality of what your miner serves. A freshly registered hotkey typically earns little until it proves itself in the validator's scoring window; treat the first days as calibration, watch the daily net line in Monitoring, and let the numbers decide whether to scale up, tune, or tear down. The Opportunities (02) view re-ranks subnets continuously, so if SN61 economics shift, the platform will tell you before your invoice does.

The Manual DIY Path — No Platform

Everything in Parts A–B can also be done by hand, directly in your provider's console and an SSH terminal. Some operators prefer this for learning; some need a box the platform cannot reach. You trade the auto-install and monitoring for full manual control — the same box, the same base stack, and the same "wallet never leaves your laptop" rule.

F1 · Create the box in the provider console

F1 Hetzner (or DigitalOcean) by hand

console.hetzner.cloud → your project → **Add Server**

- **Location** — Falkenstein or Helsinki for EU latency, Ashburn/Hillsboro for US.
- **Image** — **Ubuntu 22.04** (the same OS the platform's cloud-init targets).
- **Type** — CX32 for the RedTeam-class worked example (4 vCPU / 8 GB / 80 GB); CCX-series only if you want dedicated cores.
- **SSH key** — upload your laptop's public key (`~/.ssh/id_ed25519.pub`) rather than using a password.
- DigitalOcean equivalent: **Create → Droplets**, Ubuntu 22.04 x64, Basic plan s-2vcpu-4gb or larger, your SSH key, a region near you.

F2 · Install the base stack by hand

This is precisely what cloud-init automates on the platform path — five commands on a fresh Ubuntu 22.04 box:

SSH ROOT@<BOX-IP> · UBUNTU 22.04 — BASE STACK

```
$ apt update && apt -y upgrade
$ apt -y install git curl build-essential python3-venv docker.io
$ python3 -m venv /opt/btvenv && source /opt/btvenv/bin/activate
$ pip install bittensor
$ btcli --version   # sanity check
```

F3 · Wallet, registration, and the miner itself

Create the wallet pair on your *laptop* (D2a), copy the files up (D2b), and register the hotkey (D2c) — identical commands and the same one-time burn. Then clone the subnet's repo and run its miner the way its README prescribes:

SSH ROOT@<BOX-IP> · UBUNTU 22.04 SUBNET MINER · SN61 EXAMPLE

```
$ git clone https://github.com/RedTeamSubnet/RedTeam.git /opt/sn61
$ cd /opt/sn61 && pip install -r requirements.txt
$ screen -S sn61
$ python neurons/miner.py --netuid 61 \
    --wallet.name default --wallet.hotkey miner-cpu --network finney
# detach with Ctrl-A D; reattach later with: screen -r sn61
```

Read the subnet's own README last word. Repos evolve; flags and entrypoints change. The in-app CPU Guide (05) goes deeper for SN61 specifically — mining the validator profile, building a competitive agent, and local eval against Validator Lab before you ever burn. For any other subnet, the Subnets (03) view links straight to its repo.

PART D · REFERENCE

Troubleshooting & Economics

The nine failures seen most often, what each one means, and the fix. Most issues are configuration, not catastrophe — and only one of them burns money permanently.

H1 · When something goes wrong

Symptom	What it means and the fix
"Key check failed" after Save & verify	The token was copied short, expired, or lacks scope. Regenerate at the provider (Read+Write for Hetzner, Full Access for DO) and paste again. Check for a trailing space when copying.
Catalog still empty after connecting	Press **Refresh**; check the status line — it names each provider's state. "error — 401/403" means the key; "0 offers" with a valid key means a filter is hiding rows.
"No qualifying live offer" on a card	True, not a bug: no connected provider currently sells a box at that spec. Raise your provider coverage (connect both), or relax the RAM/price filters.
"SN documents GPU requirements" refusal	The profiler found GPU deps in the subnet's repo — a CPU box cannot serve it. Pick a CPU-classified subnet, or rent that netuid from the GPU Catalog instead.
"Profiler failed" in the dialog	The repo fetch timed out (rate limit or network). Retry once; if it persists you can still rent and stage the install manually from DevOps later.
Success panel says "IP pending"	Normal for the first ~30 seconds. Wait, then re-check the host in DevOps Engine before assuming a problem.
Host validation fails in DevOps	Cloud-init is still running (give it 3–5 minutes after IP) or was interrupted — re-run Validate; if it keeps failing, destroy the box and rent again.
Miner runs but never earns	Check the three usuals: hotkey actually registered on the netuid (`btcli subnets list`), axon port reachable (firewall/security-group must allow 8091), and validator scores in Monitoring. On SN61 also read the CPU Guide (05) — a weak agent scores weakly.
Invoice higher than expected	The box bills hourly while it *exists*, miner or not. Destroy idle boxes (H1 teardown below) — the platform path keeps the DevOps host record so you can re-rent in one click later.

H2 · Costs, honestly

Item	CPU path	GPU path (for comparison)
Host (rented)	$4.63–16.64/mo (worked example $8.32)	~$161/mo (RTX 3090 class)
Registration burn	One-off, dynamic recycle fee, same on both paths — live figure shown in CPU Guide (05) / Subnets (03) before you commit.	
Platform fees	$0 — the platform bills nothing; your costs are the provider and the burn.	
Idle cost	~$0.006–0.011/hr while the box exists	~$0.22/hr while the pod exists
Typical host share of a $100/mo net expectation	~8% (CX32)	~160% (GPU needs the big subnets to pay)

H3 · Tearing down cleanly

H3 Destroy the box when you stop mining

Platform path: DevOps Engine → your host → **Destroy** — removes the server at the provider and marks the host record closed. **Manual path:** provider console → the server → Delete (Hetzner) / Destroy (DigitalOcean). Billing stops at deletion; the registration burn is chain state and is not refunded, but your hotkey stays valid if you come back with a new box under the same wallet names.

H4 · Command cheat sheet

Command	What it does
`btcli wallet new_coldkey --wallet.name default`	Create a coldkey on your laptop (mnemonic shown once).
`btcli wallet new_hotkey --wallet.hotkey miner-cpu`	Create the miner's hotkey under that coldkey.
`btcli subnets register --netuid 61`	Register the hotkey on SN61 — the one-time burn.
`btcli wallet overview --network finney`	Balances, hotkeys and registrations as the chain sees them.
`screen -S sn61 / screen -r sn61`	Run the miner inside a persistent session; reattach anytime.
`ss -tlnp | grep 8091`	Confirm the axon port is listening.
`tail -f /var/log/miner/sn61.log`	Follow the miner's log live.

H5 · Three habits that keep the CPU path cheap

01 **Check the burn before you spend it.** The recycle fee moves with seat demand. CPU Guide (05) and Subnets (03) show the live figure — look, then register.

02 **Destroy boxes you are not using.** The bill is hourly while the server exists. DevOps Engine keeps the host record, so re-renting later is one click, not a rebuild.

03 **Let Monitoring argue with your optimism.** A weak agent scores weakly, and the daily net line knows it before you do. Scale what earns; tear down what does not.

Mine with what you have.

The CPU path is the cheapest honest way onto Bittensor: a few dollars a month of VPS, a base stack that installs itself, and two human decisions — where your wallet files go, and when the miner starts. Everything else is clicks.

- ✓ Provider key connected — live offers every 60 seconds, stored AES-256-GCM encrypted
- ✓ Box rented at the spec the subnet actually needs — refused if it can't run the code
- ✓ Base stack auto-installed by cloud-init — docker, venv, bittensor, ~3–5 minutes
- ✓ Wallet files moved by you, burn spent knowingly, launch approved by design
- ✓ Monitoring watching axon latency, resources and cost from the first minute

Where to go next: the in-app **CPU Guide (05)** for the SN61 agent-building deep dive, the **GPU Miner Setup Guide** for the GPU path, and docs.bittensor.com for everything btcli.

INFRANEX BT · OPERATOR HANDBOOK

GPU Miner Setup Guide

From an empty dashboard to your first live Bittensor miner on subnet 64 (Chutes) with an RTX 4090 — every click, every command, every screenshot, in the exact order that keeps your costs at zero until the moment you choose to spend. Plus the no-GPU alternative: script mining on SN67 (Harnyx) from any CPU.

SN64 · CHUTES | RTX 4090 · 24 GB · $0.34/HR | CPU PATH · SN67 HARNYX | BINANCE FUNDING | REGISTER LAST | HOTKEY-ONLY SECURITY

Edition **v1.4** · September 14, 2026

Network **Bittensor Finney (mainnet)**

READ THIS FIRST

The Golden Order

One principle runs the whole guide: **free things first, metered things last.** Your wallet costs nothing, your provider key costs nothing, the GPU meter starts the second a pod is created, and the registration burn starts your earning clock. Do them in this order and you never pay while you are still setting up.

LAPTOP · FREE

Wallet

coldkey + hotkey via btcli — keys never leave your machine

PLATFORM · FREE

Provider key

paste RunPod API key — connecting is free

WIZARD STEP 4 · COSTS

Rent GPU

pod created — $/hr billing starts now

WIZARD STEP 4

Install

automatic 10-step installer runs on the pod

WIZARD STEP 5 · COSTS

Register

burn fee → UID — earning clock + immunity start

ALWAYS ON

Monitor

daemon + alerts + P&L track everything

Where the money moves

Move	Amount	What it is
Binance → coldkey	burn fee + ~2 TAO	Plain deposit of funds. Nothing is burned; fully reversible by your own keys.
coldkey → chain burn	dynamic recycle fee	The actual burn — automatic inside the `btcli subnets register` command. SN64 burn-entry currently ≈ **0.0005 TAO**.
card → RunPod	$0.34/hr (4090)	GPU rent. Runs from pod creation until you **Terminate**.
Coldkey mnemonic	—	Never leaves your laptop. Never typed into any website, never stored by the platform.

Before you start — have these ready

- [ ] A laptop with **Python 3.9+** (for btcli — all wallet steps run here, not on the GPU)
- [ ] A **RunPod account** with billing set up, and its API key ready to paste
- [ ] **TAO on Binance** (or anywhere) — enough for the burn fee plus a ~1 TAO buffer
- [ ] **45–60 minutes** of uninterrupted time from the moment you click *Rent*
- [ ] Paper and a pen — the coldkey mnemonic is shown **once** and must be written down

PART A · ~5 MINUTES · $0

Platform Prep — Log In & Connect RunPod

Two goals here: get into the platform, and hand it your RunPod API key so it can rent on your behalf later. Neither step costs anything — the key is only used when you actually create a pod.

A1 Open the app and sign in

Open the preview link, then sign in with your operator ID and access code.

Login screen → enter **User ID** + **Access code** → **Sign in**

You land on the **Dashboard** — the chain badge (top right, green dot) should read *LIVE · FINNEY CHAIN* with a recent block number. That is your confirmation the platform is reading live chain data.

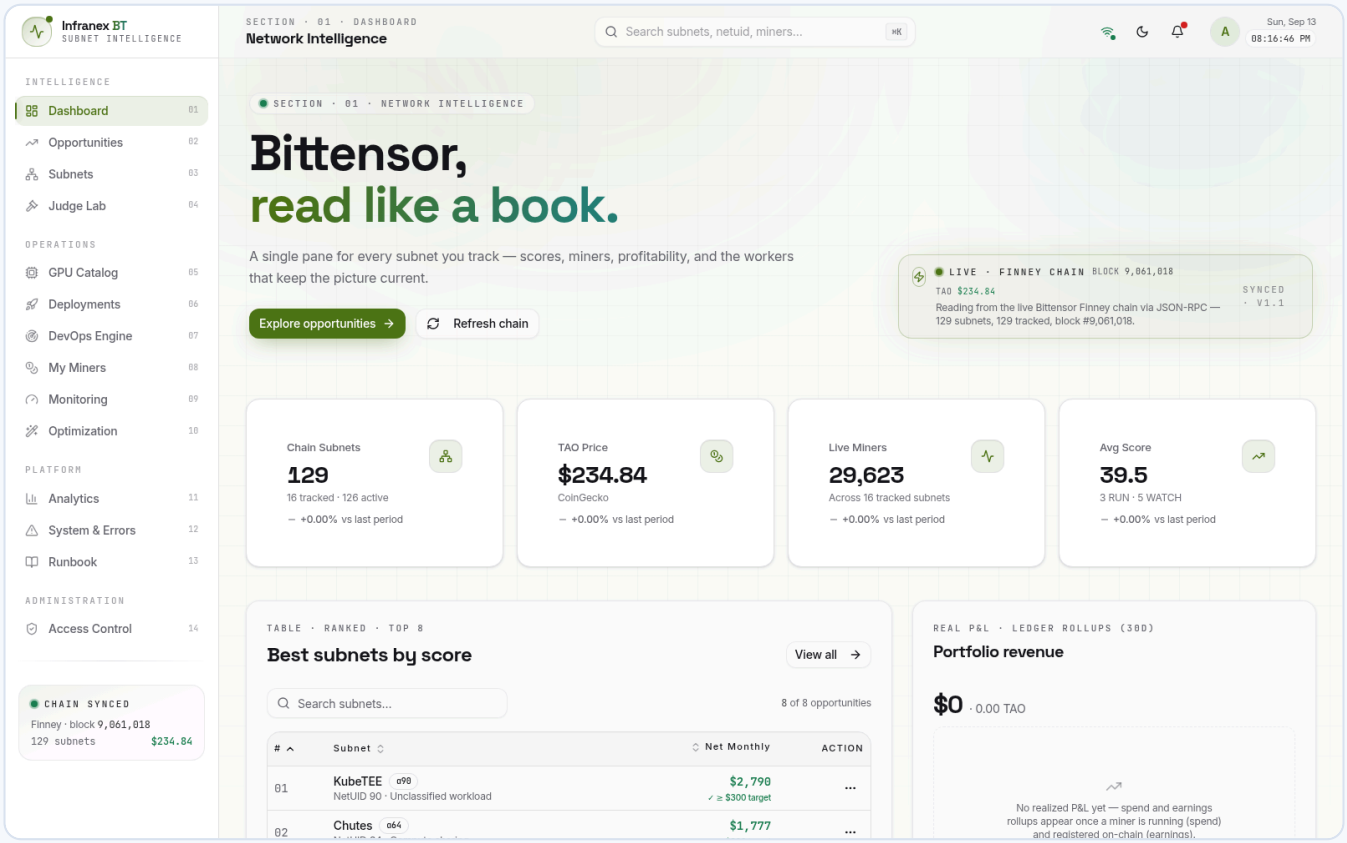


Fig A-1. The sign-in gate. Sessions are per-device; every sign-in is recorded in the audit trail (System & Errors → Audit trail).

A2 Connect your RunPod API key

GPU Catalog 05 → **Provider API keys** → paste key into the **RunPod** field → **Save & verify**

Get the key from `runpod.io → Settings → API Keys → Create API Key`, then paste it here. The platform validates the key on the spot and encrypts it at rest before storing anything. When validation passes you will see the green *saved* state on the RunPod row, and the GPU offer list (later steps) shows live prices.

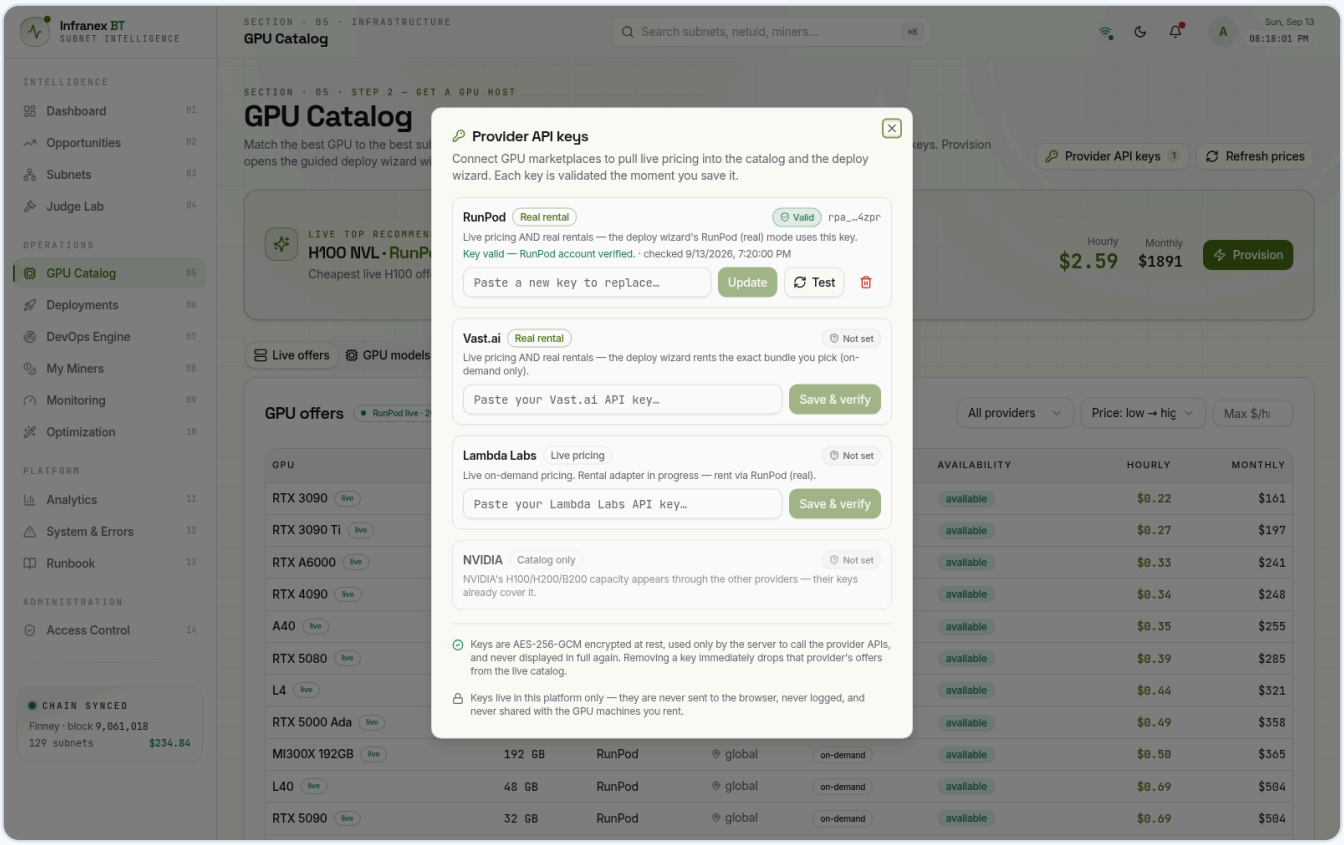


Fig A-2. The Provider API keys dialog — one encrypted slot each for RunPod, Vast.ai and Lambda Labs. Only the run-time platform can decrypt these; the UI never shows a saved key again.

Why RunPod first? The wizard's GPU step lists offers from every connected provider. With RunPod connected you will see the live RTX 4090 at ≈ **$0.34/hr** (prices move — check at rent time). Vast and Lambda can be added later; more providers just means more offers to choose from.

A3 Optional now, recommended: save your wallet profile

System & Errors 12 → scroll to **Wallet profiles** → **Add profile**

Save a **label** plus the btcli **wallet name** and **hotkey name** you plan to use (example used throughout this guide: wallet `infranex`, hotkey `miner`). This is public metadata only — it makes the deploy wizard prefill your wallet name instead of you typing it every time. The platform **cannot store mnemonics or private keys**; the API rejects any attempt.

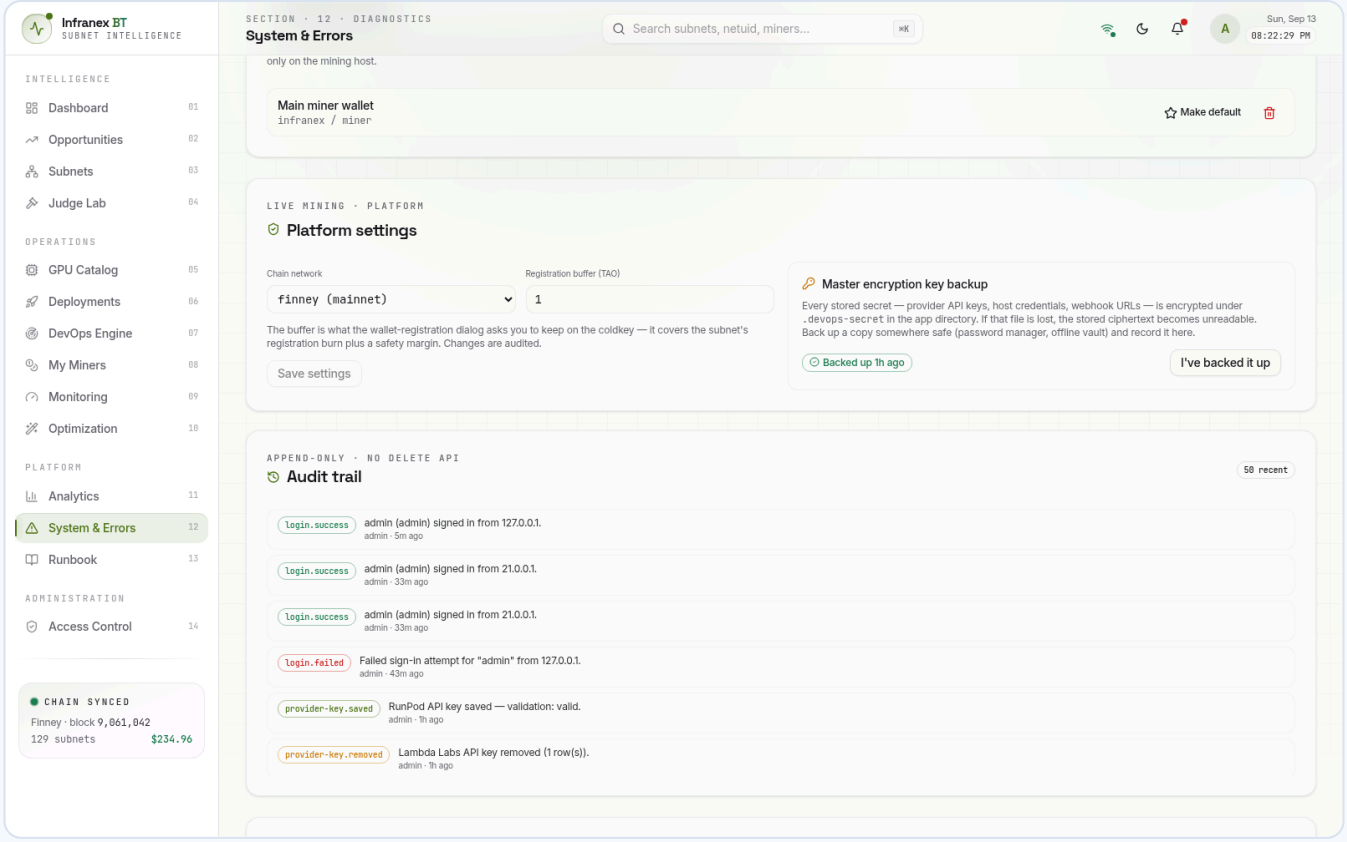


Fig A-3. System & Errors — bottom section. Wallet profiles (public metadata only), Platform settings (chain network + registration buffer), Master-key backup acknowledgment, and the append-only Audit trail.

PART B · ~15 MINUTES · $0 (ON YOUR LAPTOP)

Create the Wallet & Fund It

The platform opens this same wizard for you later (Deployments → *Start wallet setup*) with every command generated and a **copy** button on each. You can follow either — the commands are identical. Do these four steps **before** renting the GPU, so the billing clock never runs while you set up keys.

B1 Open the Wallet & registration wizard (optional shortcut)

Deployments 06 → Mining Journey card → **Start wallet setup**

The wizard has 8 steps. Steps 1–4 happen on your laptop and cost nothing; steps 5–8 come back to the platform after the GPU is rented. Everything below walks the same eight steps with the exact commands.

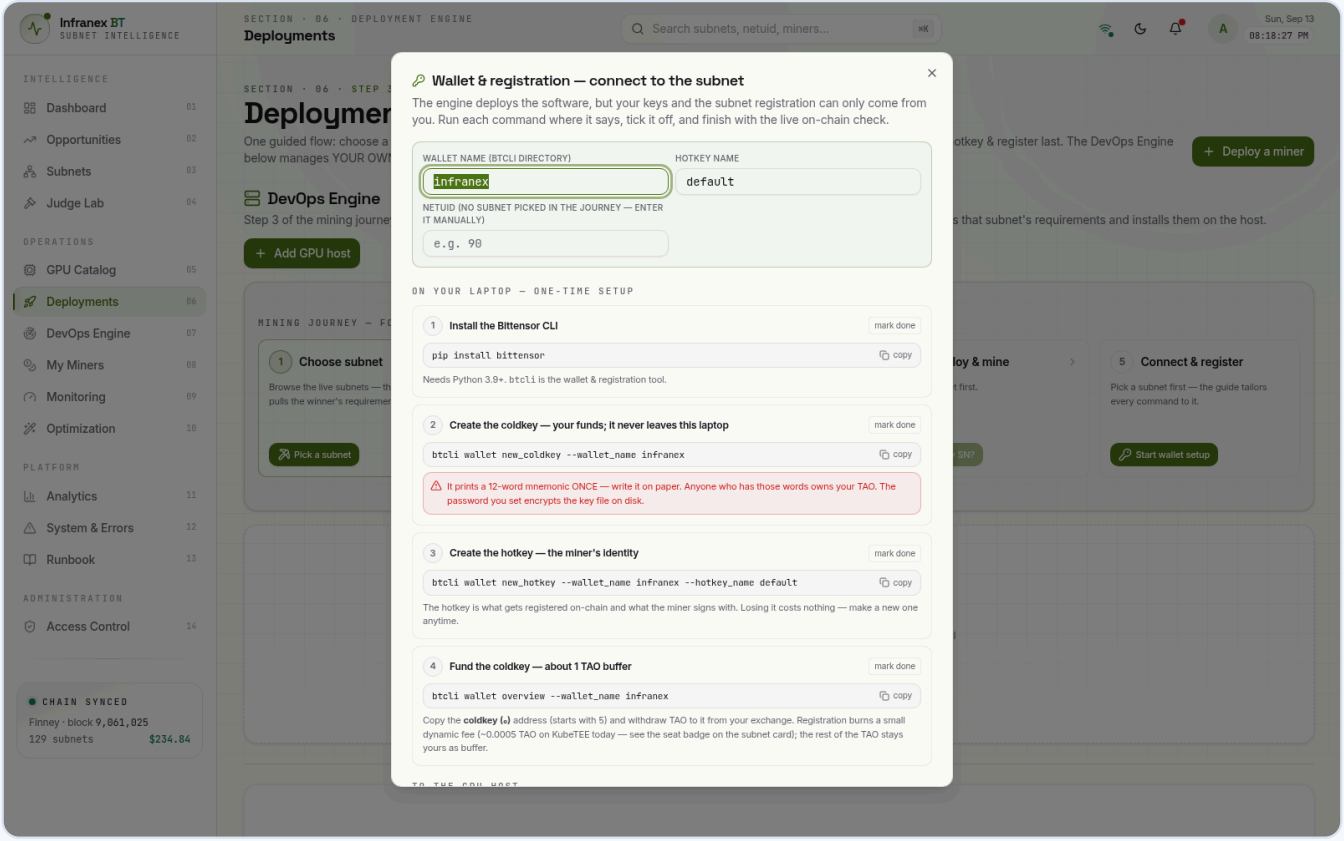


Fig B-1. The wizard's laptop section — wallet name, hotkey name, and steps 1–4 with copy-paste commands. The red warning is the one that protects your life savings: the mnemonic prints once.

B2 Install the Bittensor CLI

On your laptop terminal (Python 3.9+ required):

YOUR LAPTOP · TERMINAL — STEP 1 OF 8

```
$ pip install bittensor
Successfully installed bittensor-9.x.x substrate-interface-1.x.x ...
$ btcli --version
btcli v9.x.x
```

B3 Create the coldkey — your funds

YOUR LAPTOP · TERMINAL — STEP 2 OF 8

```
$ btcli wallet new_coldkey --wallet_name infranex
Wallet dir: /home/you/.bittensor/wallets/infranex
IMPORTANT: Store this 12-word mnemonic safely.
It is the ONLY way to recover your funds if you lose access:
1 word    2 word    3 word    4 word
5 word    6 word    7 word    8 word
9 word    10 word   11 word   12 word
Coldkey SS58 address: 5CA1abcdef...your-coldkey-address
```

The 12 words are your money. Write them on paper (never screenshot, never cloud notes, never paste anywhere — including into the platform). Whoever holds those words holds your TAO. The platform has no field for them by design.

B4 Create the hotkey — the miner's identity

YOUR LAPTOP · TERMINAL — STEP 3 OF 8

```
$ btcli wallet new_hotkey --wallet_name infranex --hotkey_name miner
Hotkey SS58 address: 5Grwabcdef...your-hotkey-address
```

The hotkey is what gets registered on-chain and what the miner signs with. Losing it costs nothing — you can create a new one anytime; only the coldkey is precious.

B5 Fund the coldkey from Binance

First, print your addresses:

YOUR LAPTOP · TERMINAL — STEP 4 OF 8

```
$ btcli wallet overview --wallet_name infranex
Wallet: infranex   ✓- coldkey: 5CA1abcdef...your-coldkey-address
                   ℏ- hotkey:  5Grwabcdef...your-hotkey-address
```

Then in Binance: **Spot → Withdraw → TAO**. Full click-by-click in **Chapter 7**. Send to the **coldkey** address (the ₵ one, starts with 5), amount = burn fee + ~2 TAO headroom. Skip ahead if you want the funding details now — the rest of this guide assumes TAO is on the coldkey.

PART C · ~20–30 MIN · COSTS START AT C4

Pick the Subnet & Rent the GPU

The platform gives you two doors into the same wizard: the **Mining Journey** card (recommended — it tracks your progress through all five steps) or **Deploy a miner** (the raw wizard). This guide uses the Journey.

What about the DevOps Engine strip and its "Add GPU host" button? That is the third thing on the Deployments page, and it belongs to a *different* path: machines **you already own or rented yourself** (a home GPU box, or a pod created on the provider's own website). **Add GPU host** just asks for the machine's SSH address / user / password, the engine connects over SSH and runs the same 10-step install pipeline on it. **You do not need it for this guide.** When you rent through the wizard (C4 → C5), the platform registers the rented pod itself, connects with provider-issued credentials, and streams the install log inside wizard Step 4 — the pod shows up as a managed deployment without you ever clicking *Add GPU host*. Rule of thumb: **rented here → touch nothing in the DevOps strip; own the box → Add GPU host.**

Reading the subnet verdicts — RUN / WATCH / AVOID. Every subnet in **Opportunities** carries a verdict computed from the Miner's Ledger score: mid-pack monthly USD *after* GPU rent (weight 30%), seat safety — slot pressure, reward concentration, burn, immunity (20%), alpha economics — price trend, liquidity, slippage (20%), earning reality — rewarded ratio, newcomer ramp (15%) and fit & feasibility (15%). The bands: **RUN** = score ≥ 60 (healthy mid-pack economics — green), **WATCH** = 40–59 (viable *with skill or cheap entry* — amber), **AVOID** = below 40, or expected net profit under your $300/mo target (forced AVOID — red). **WATCH is advice, not a lock**: the wizard will happily deploy a WATCH subnet today. But mid-pack on those subnets hovers near breakeven once rent is paid, and the registration burn is the same on every subnet — so a "try WATCH now, move to RUN later" plan pays the burn twice. **First miner → RUN subnet** (that is why this guide mines Chutes SN64); consider WATCH only when you have a specific edge — a cheaper GPU, existing code for that subnet's niche, or you deliberately want a low-cost experiment.

C1 Choose the subnet — Chutes (SN64)

Deployments 06 → Mining Journey step 1 → **Pick a subnet** → search `chutes` → **SN64 Chutes · RTX 4090 24GB**

The picker already knows which GPU matches: it literally offers *SN64 Chutes — RTX 4090 24GB*. Confirm with **Mine this subnet**. The journey card now shows *SN64 · Chutes · needs ≥ 24 GB VRAM* on step 1.

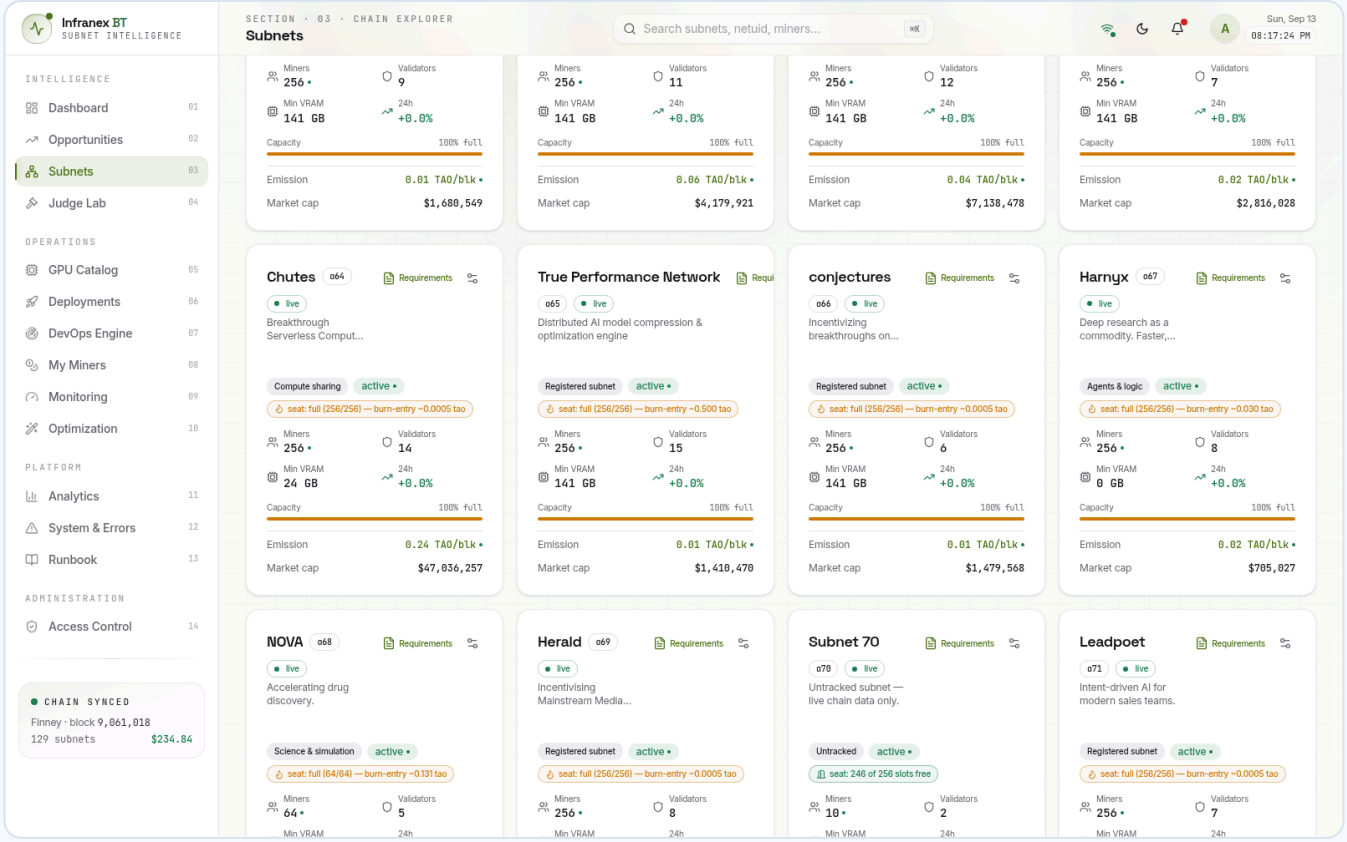


Fig C-1. Subnets view — the Chutes card carries the numbers that matter: **Min VRAM 24 GB** (a 4090 fits exactly), **burn-entry ≈ 0.0005 TAO** (your registration cost), 256 miners, emission 0.24 TAO/block. The Requirements button opens the full hardware profile.

C2 Run the wizard — subnet → requirements → GPU

Deployments 06 → **Deploy a miner** → Step 1 pick **SN64 Chutes** → Step 2 review requirements → **Find a GPU**

Step 2 shows the subnet's real requirements profile pulled live (what the miners on SN64 actually run). Step 3 lists **offers meeting the 24 GB requirement, cheapest first** — RTX 3090 at $0.22/hr, A6000 at $0.33/hr, **RTX 4090 24 GB at ≈ $0.34/hr**, and more.

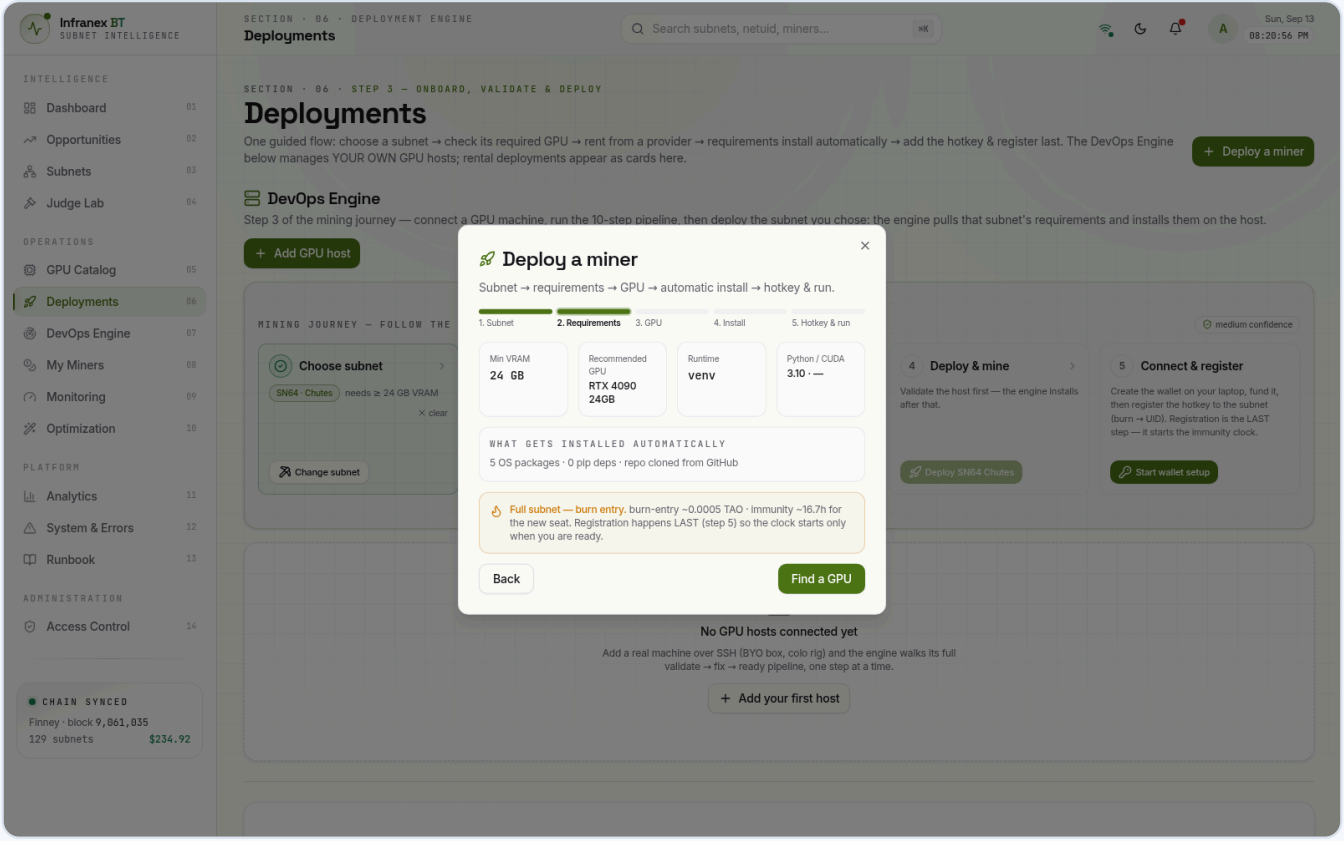


Fig C-2. Wizard step 2 — the requirements profile for SN64. With the journey subnet set, the wizard opens pre-selected here.

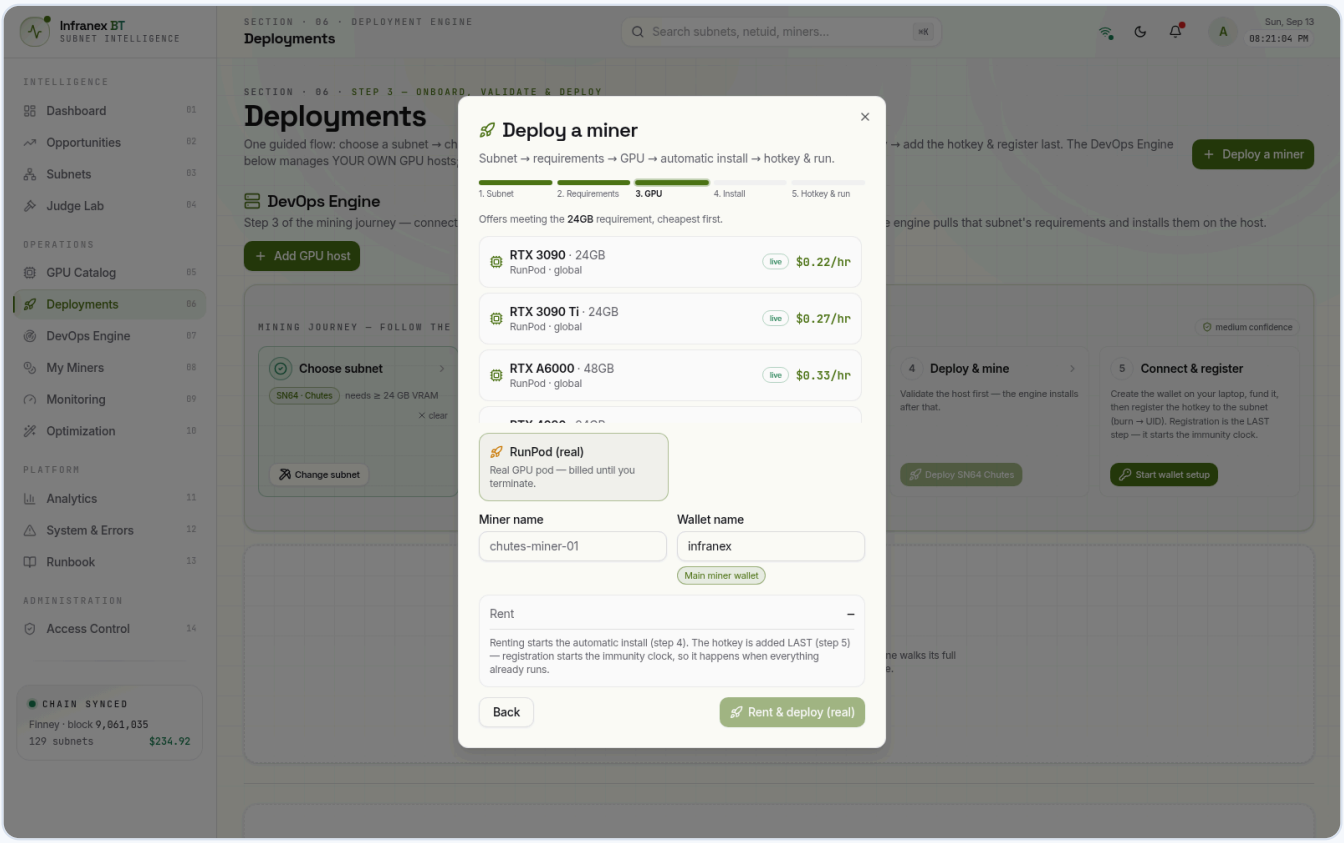


Fig C-3. Wizard step 3 — live offers filtered to ≥24 GB. Note the form below: miner name (auto: `chutes-miner-01`), wallet name prefilled from your saved profile (`infranex`), and the reminder that the hotkey is added LAST.

C3 Confirm the GPU and the names

Select the **RTX 4090 · 24GB · RunPod** offer. Check the two name fields: **Miner name** is the label in your dashboard; **Wallet name** must match the btcli wallet you created in Part B (`infranex` in this guide — it prefills automatically from the wallet profile). Do not click the rent button until you are truly ready to start the meter.

C4 Rent & deploy — the meter starts

Wizard step 3 → select **RunPod (real)** → **Rent & deploy (real)**

From this click: the platform creates the pod on RunPod (billing starts at the offer's $/hr), then runs the **automatic installer** — about ten steps shown live in the wizard's Install step: pull the image, write the subnet config, generate the pod's own SSH keypair, install the Node Daemon, start the miner process. You do nothing here except watch the log tick. When it finishes, the deployment card shows **started** and the wizard sits on step 5 waiting for your hotkey.

> **What "started" means now:** the miner process is running but serving *nothing yet* — it has no registered hotkey, so validators have nobody to score. This is by design: the expensive part (registration) waits until the cheap part (install) is proven working.

PART D · ~2 MINUTES · $0

Put the Hotkey on the Pod

The miner on the pod needs the hotkey **keyfile** to sign messages. This file travels directly from your laptop to the pod over SSH — it never passes through the web app, and the coldkey file stays on your laptop.

D1 Run the generated SCP command (wizard step 5)

Wizard step 5 → **"Copy the key files to the host"** → copy the `scp` command → run it on your laptop

The wizard generates the exact command with your pod's real IP, SSH port and path — it looks like:

YOUR LAPTOP · TERMINAL — STEP 5 OF 8

```
$ scp -P 12345 ~/.bittensor/wallets/infranex/hotkeys/miner
root@149.x.x.x:/root/.bittensor/wallets/infranex/hotkeys/
miner                              100%  496    12.3KB/s   00:00
```

Mark the step done (the *mark done* button) and move to step 6. Only the hotkey file crossed over; your coldkey and mnemonic are still exclusively on your laptop.

PART E · ~5 MINUTES · BURN HAPPENS HERE

Register LAST — Burn, UID, Verify

Your miner is installed and healthy — **now** you pay the registration fee. The burn is not a transfer you make by hand: it happens automatically inside the register command, taken from your coldkey the moment you confirm. In exchange your hotkey gets a **UID** on SN64, and both the immunity clock and your earning window start.

The gate before you burn: the registration fee is the same on every subnet, so fail *cheap* first — run the free **Validator Lab** pre-flight (Part H, 10 minutes, $0) and confirm your spec scores **competitive or better** against SN64's judge. If it does, come straight back here and register. If it doesn't, Part H also shows you the highest-leverage fixes — much cheaper to apply them now than to discover a weak score after the burn.

E1 Register the hotkey (wizard step 6, on your laptop)

Wizard step 6 → **"Register the hotkey"** → copy the command → run it on your laptop → confirm the fee

YOUR LAPTOP · TERMINAL — STEP 6 OF 8

```
$ btcli subnets register --netuid 64 --wallet_name infranex --hotkey_name
miner
Your balance:  2.0493   TAO
Recycle burn: 0.0005   TAO (subnet 64 entry cost)
Do you want to register? [y/n]: y
✓ Registered.
UID: 173   Netuid: 64   Block: 9,061,842
```

btcli always prints the exact current fee and asks for confirmation — nothing is spent until you type y. The fee is **per registration**: if the hotkey ever deregisters, re-registering costs the fee again. That is why the platform watches deregistration risk for you after this point.

E2 Restart the miner so it serves its axon (wizard step 7 — one click)

Wizard step 7 → **Restart miner** button on the deployment card → the engine runs it over SSH for you

The miner announces its axon (IP:port) on-chain at startup — that announcement is how validators find and score you. The platform shows *restart pending* until this happens; one click on the card and the engine does the rest. Prefer the terminal? The wizard also prints the exact SSH command.

E3 Verify on-chain (wizard step 8 — one paste, one click)

Wizard step 8 → paste your **hotkey SS58** (the ℏ address from `btcli wallet overview`) → **Verify**

The platform scans the live metagraph itself and reports:

```
PLATFORM · STEP 8 RESULT                                  CHAIN-VERIFIED

✓ REGISTERED — UID 173 on SN64 (Chutes)
active: yes      incentive: 0.0% (ramps up as validators score you)
emission: 0 rAO  block: #9,061,842
```

The only thing you ever paste into the platform is this **public** address — never a private key, never a mnemonic. Deployment status flips to **started · UID · registered**, the burn amount is recorded on the deployment, and the Mining Journey goes all-green.

PART F · FROM NOW ON · AUTOMATIC

You Are Live — What Watches What

From the verified-UID moment, the platform's worker passes run every 90 seconds and you stop being a babysitter. Here is the map of what watches what, so you know where to look when something moves.

Card / panel	What it tells you
Dashboard — Portfolio revenue	Real P&L: spend accrued every 90 s vs earnings rolled up daily from chain emissions, with a 14-day bar strip. Costs appear within minutes of the pod starting; earnings appear once validators start scoring your UID.
Deployments — deployment card	Status (started), UID, registration state, install log, the **Restart miner** and **Terminate** controls.
My Miners — UID Defense	Your UID's live chain vectors: incentive, emission, deregistration-risk sparkline. If your axon stops serving, this flags it with a runbook action before validators forget you.
Monitoring / Runbook	Alert rules (pod down, axon unreachable, registration lost, spend anomaly) — wire a Slack/Discord webhook in the channels dialog so they reach you.
System & Errors — Audit trail	Every credential change, approval, and sign-in, append-only. If something moved, it is written here.

Stopping vs terminating. If you ever pause a miner, remember RunPod bills the pod's volume storage **even while stopped**. If you are done with a pod for good, use **Terminate** — otherwise an idle 100 GB volume quietly costs about $7/month forever.

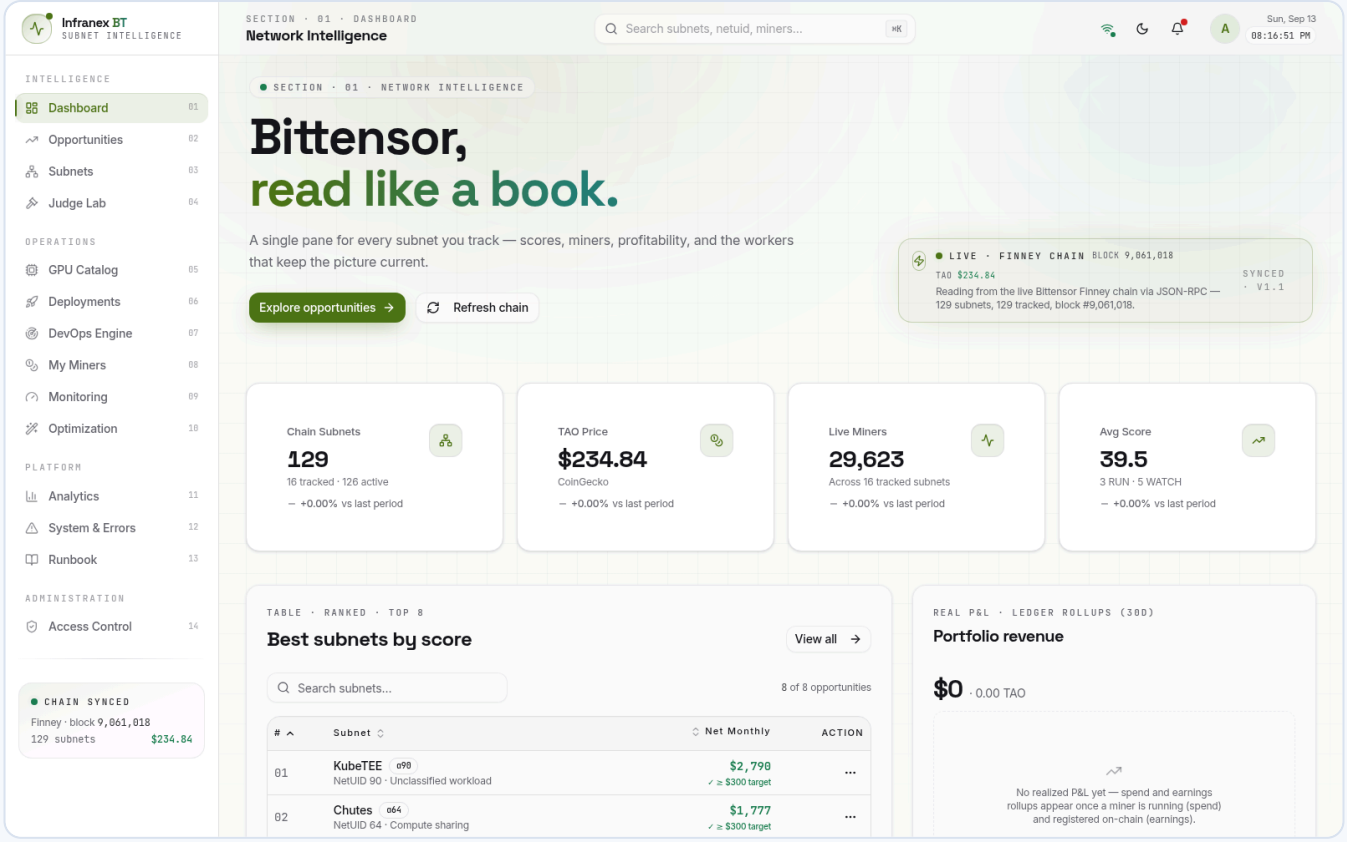


Fig F-1. The dashboard once you are live: chain badge, portfolio revenue (currently the honest empty state — it fills from the first accrual), and your tracked subnets with Chutes ranked 2nd by score.

PART G · YOUR JOB NOW · ~5 MIN/DAY

Living as a Miner — The Operator Routine

Once deployed, the machine lifecycle runs on rails: **provisioning → installing (10 steps) → started → registered → earning**. "Started" alone earns nothing — only "registered" puts your hotkey on the metagraph so validators can score you. From that moment your job is not babysitting a terminal; it is reading five surfaces every day and acting on alerts. The platform watches every 90 seconds — **you decide**.

Pick a posture first. DevOps Engine 07 tags every miner with a strategy mindset, shown on the Strategy Board: **DEFEND**, **STEADY**, **EARN MORE**, or **OPTIMIZE**. For your **first 1–2 weeks run DEFEND** — your only goal is keeping the UID alive, because the burn is expensive to repeat. Once the UID survives a full week with nonzero incentive, move to STEADY, and only then consider EARN MORE (push throughput) or OPTIMIZE (cut $/hr for the same output).

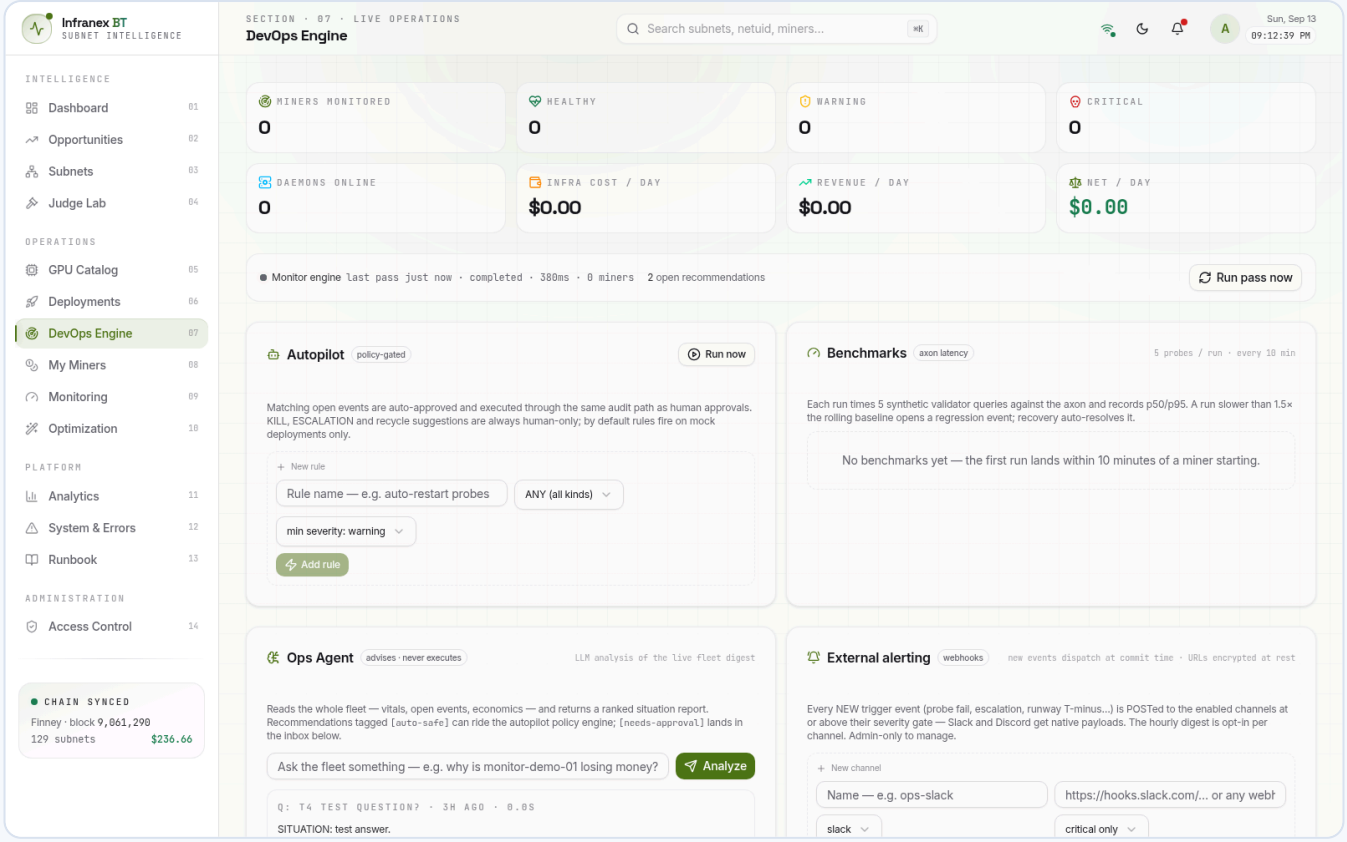


Fig G-1. DevOps Engine 07 — the automation deck. Counters for healthy/warning/critical miners, the 90-second monitor pass line, policy-gated Autopilot, axon latency benchmarks, the Ops Agent situation reports, and the External alerting panel where a real Slack/Discord webhook plugs in.

G-1 Every day — read the deployment card

Deployments 06 → your deployment card

The status line must read **started · UID n · registered** with all ten install steps ticked green. *started* without a UID means registration never landed — reopen the wallet wizard and run the register step again. A *stopped* card usually means the pod died or the RunPod balance hit zero — fund the account and press **Restart miner**; the engine does the SSH work in one click.

G-2 Every day — check UID Defense is *healthy*

My Miners 08 → UID Defense panel → risk level + sparkline

The engine scores your UID against the live metagraph on every pass and raises exactly six risk codes. Learn them once — they are the complete map of how a miner gets hurt:

Risk code	Meaning → your action
NOT_REGISTERED *critical*	Hotkey missing from the metagraph → re-run registration.
ZERO_INCOME	Registered but earning 0 → normal in the first hours; if it persists past a day, restart and check the miner version.
EVICTABLE	Subnet at capacity and your immunity expired — you are a cut candidate → act fast: restart, update, improve; do not ignore this one.
UNDERPERFORMING	Scoring below your peers → update the miner revision; check subnet news.
INCENTIVE_COLLAPSE	Your incentive cratered → first check whether the whole subnet dipped (Subnets 03) or just you.
PERSISTENT_DECLINE	Days of falling incentive → consider a fresh deploy, a different GPU, or a different subnet.

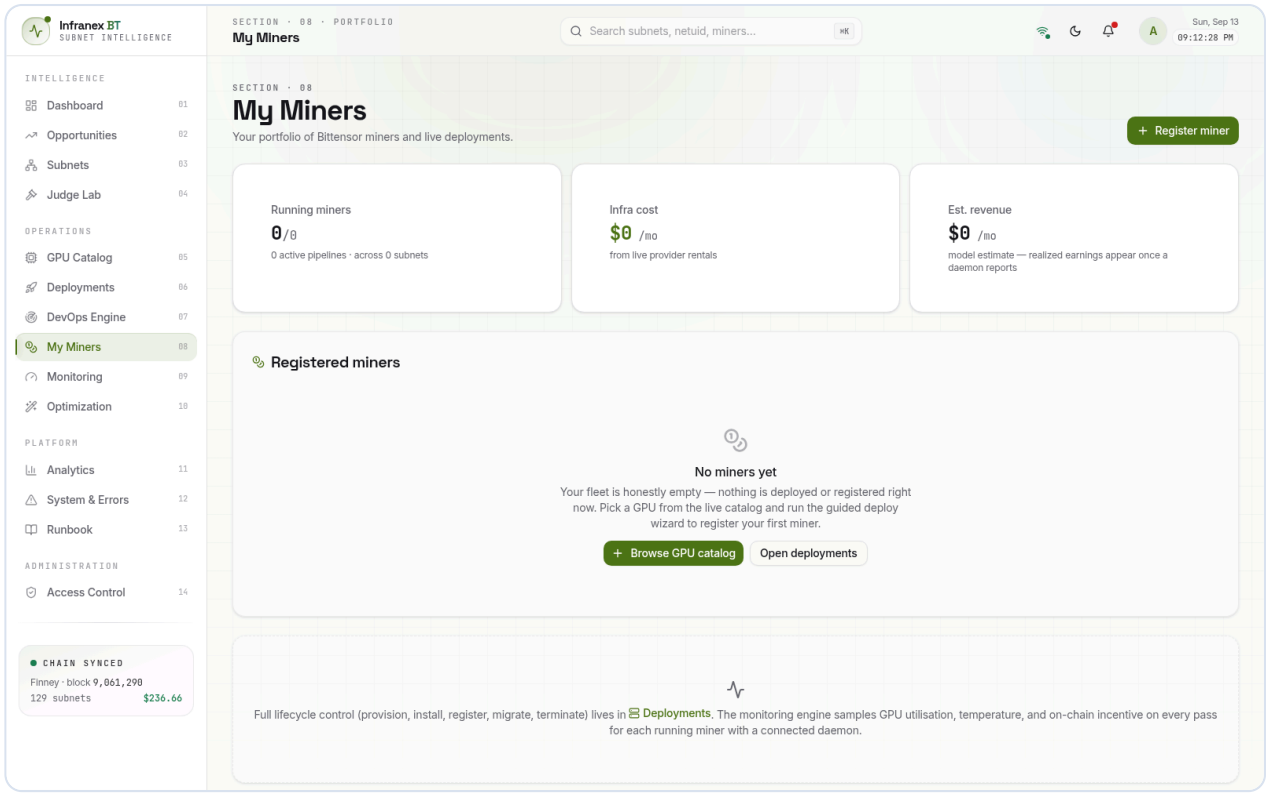


Fig G-2. My Miners 08 — the portfolio: running miners, monthly infra cost, estimated vs realized revenue, and the Registered miners list where UID Defense lives. Honest empty state here — yours fills the moment the first deployment registers.

G-3 Every day — read the Monitoring rows

Monitoring 09 → miner row → GPU util · temp · incentive

The engine samples each running miner every 90 seconds. **GPU utilisation** should spike regularly while the miner serves queries (60–95% bursts are normal) — a sustained flat 0% for hours means you are not being queried: check registration and version. **Temperature** is fine up to roughly 83–85 °C; sustained readings above that risk throttling — restart and re-check. **Incentive** (e.g. `I: 0.0042`) is your validator score: small daily growth is the goal. The four header cards give the money view: **Daily Emission** (at the live TAO/USD price), **Monthly Cost** (GPU rental), and **Net Profit/mo with ROI%** — for example `Net +$4.10/mo · ROI 61%`.

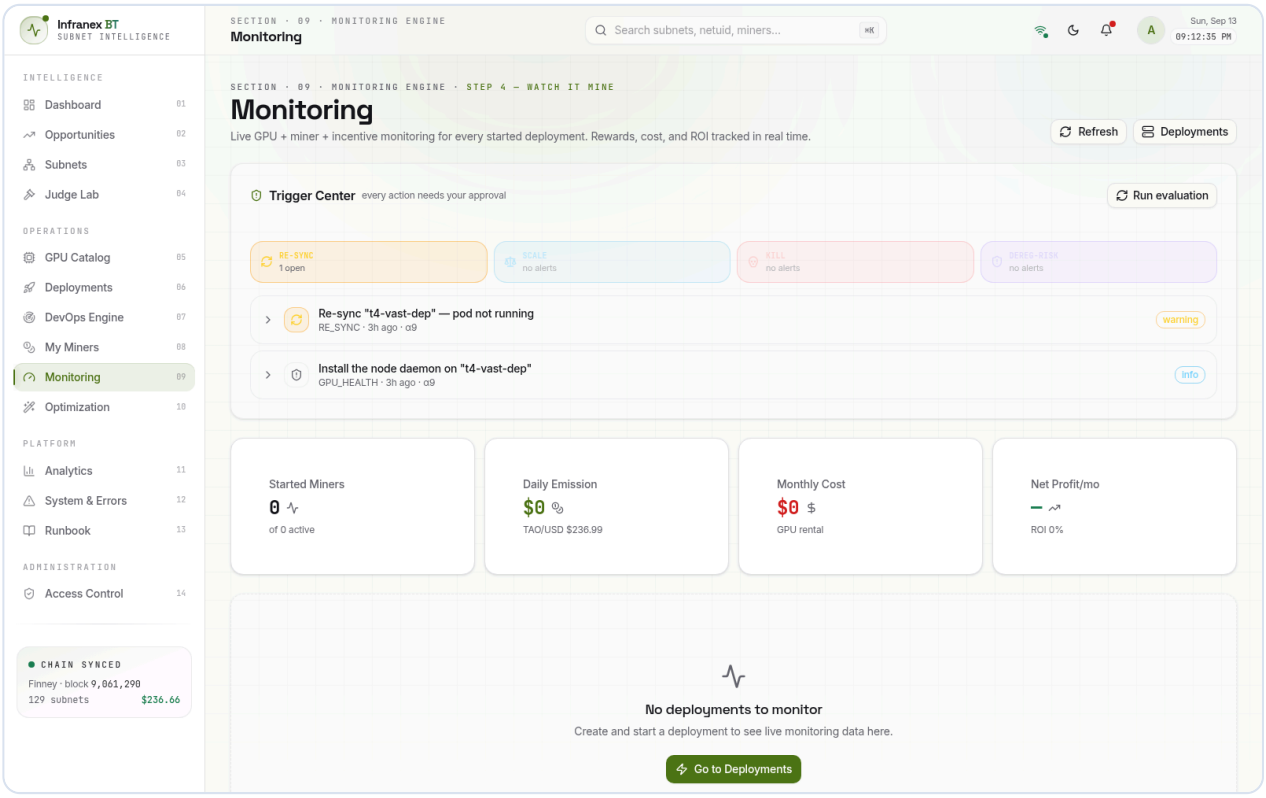


Fig G-3. Monitoring 09 — Trigger Center on top (every action needs your approval; here one RE-SYNC warning from a pod that is not running), then the money cards: Started Miners, Daily Emission at TAO/USD $236.99, Monthly Cost, Net Profit/mo with ROI.

G-4 Every day — glance at the P&L card

Dashboard 01 → Portfolio revenue card

Spend accrues within minutes of the pod starting; earnings roll up daily from chain emissions once validators score you. A healthy first week looks like: spend climbing steadily (about $8/day at $0.34/hr) while earnings start near zero and inflect upward as the UID earns trust. A card that only ever climbs on the cost side after a week of registration is your signal to revisit G-1 through G-3.

G-5 Every day — keep 2–3 days of RunPod runway

An idle balance check takes ten seconds on the RunPod billing page. At $0.34/hr the burn is about $8.16/day, so keeping $17–25 of credit means never waking up to a stopped pod. If the balance reaches $0, RunPod stops the pod, the miner goes offline, and income stops — after topping up, one click on **Restart miner** brings it back. Put it on your calendar as a top-up reminder rather than trusting memory.

PART G.2 · WEEKLY & MONTHLY · ~15 MIN/WEEK

The Deeper Cadence

When	Where	What & why
Weekly	My Miners 08	Compare incentive Mon→Mon instead of staring daily — weekly deltas are the real trend line.
Weekly	Subnets 03 → SN64	Is the subnet itself healthy (emission, price, registration count)? A dying subnet drags every UID down — this is also where a better home would show up.
Weekly	Laptop terminal	Hotkey balance check with `btcli wallet overview` (example below). Move TAO hotkey→coldkey only once meaningful amounts accumulate — never rush this in week 1.
Weekly	RunPod console	Any *stopped* pods you are done with? **Terminate** them — an idle 100 GB volume costs ≈ $7/month forever.
Weekly	System & Errors 12 → Audit	Scan logins and credential events; anything you do not recognize = rotate the provider key now.
Monthly	GPU Catalog 05	Re-check RTX 4090 pricing — your $0.34/hr moves, and a cheaper identical GPU elsewhere is a valid OPTIMIZE move (Vast offers appear alongside).
Monthly	Deployments 06 → revisions	Apply miner revision updates when flagged — Chutes refreshes its model list and stale miners lose incentive fast.
Monthly	Analytics 11	Month-over-month earned vs spent vs ROI — the grow / hold / migrate decision, made on data.

YOUR LAPTOP · TERMINAL — WEEKLY BALANCE CHECK

```
$ btcli wallet overview --wallet_name infranex
Coldkey     Hotkey   Network   Stake α     USD
infranex    miner    finney    0.0831      $19.44   ← growing weekly ✓
```

What you never have to do manually: spend accrual, GPU utilisation and temperature sampling, incentive sampling, UID Defense scoring, daily earnings rollups, and the append-only audit log all run on the platform's 90-second worker passes — and alerts dispatch to your webhook the moment a trigger fires. **The whole daily routine in one line:** deployment card green → UID Defense healthy → GPU% and incentive nonzero → P&L not bleeding → RunPod balance funded → done.

PART H · PRE-FLIGHT & ONGOING · $0 · ~10 MIN

Validator Lab — Test Before You Burn

Every subnet has a **judge** — the validator that scores miners and decides your incentive. Judges are not alike: one pays for raw speed, another for output quality, another for the cheapest price. Validator Lab (nav **04**) reads the subnet's *actual validator code*, extracts what it rewards and how brutally it pays, then **simulates your miner spec against it** — all before you spend a rupee. It is the only place in this guide where failing is free.

H-1 Mine the judge profile — SN64

Validator Lab 04 → pick **α64 · Chutes** → **Re-mine profile**

The lab resolves the subnet's repo from on-chain identity, scans the validator source for scoring evidence (*24 evidence hits* for Chutes), and classifies the judge into an archetype. Chutes comes back as **Market Clearing** at **66% profile confidence** — the lab tells you how much code evidence it found, so you know how much to trust the model.

H-2 Read the profile — what the judge pays for, and how brutally

Two things matter. **Composite weights** — for SN64: Price Competitiveness 58%, Throughput 17%, Response Speed 12%, Uptime 7%, Quality 5%, Resource Fit 2%. Translation: on Chutes, a cheap, fast miner beats a brilliant-but-pricey one. **Cohort brutality** — *Brutal 62/100*: 256 miners registered, only 17 earning (6.6% earn ratio), and the top 10% of earners take 30% of the incentive. This is the same "reward concentration" risk the Opportunities verdict prices in — here you see it raw.

H-3 Set your real spec and run the simulation

In the **Mock-validator simulator**, dial the sliders to what your 4090 will actually run: **2.00s** response latency, **99.50%** uptime, **80%** quality, **25 units/s** throughput, **$0.50**/1M tokens. Then **Run simulation**. The scoring curves are deterministic — same spec, same result — so you can iterate freely.

H-4 Read the verdict and the levers

The example run scores **62.1** — verdict **competitive**, estimated cohort percentile **~p82**, **1.36×** the cohort median. Below the numbers: a per-dimension breakdown (speed 0.98, uptime 0.90, quality 0.77 — price only 0.50, the weak axis on a Market Clearing judge) and the **highest-leverage fixes** ranked by composite points gained: *+14.5 cut asking price*, *+3.6 increase throughput*, *+0.7 resource fit*. Fix what is cheap to fix, re-run, and watch the composite move.

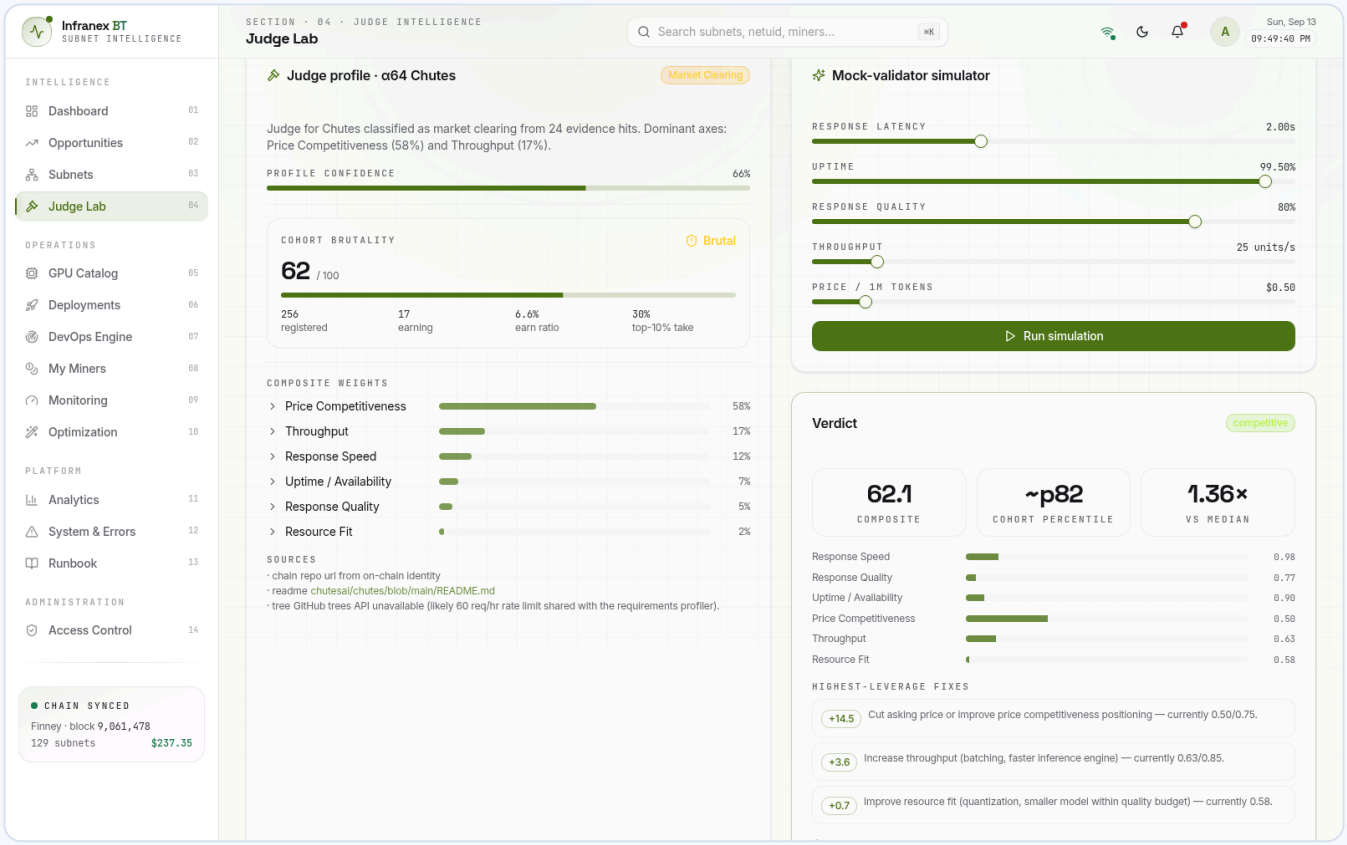


Fig H-1. Validator Lab on SN64 — left: the mined judge profile (Market Clearing, 66% confidence, brutality 62/100, composite weights with Price at 58%); right: the mock-validator simulator at the 4090 default spec and its verdict — composite 62.1, *competitive*, ~p82 percentile, 1.36× vs median — with the ranked highest-leverage fixes.

Verdict	Composite	What it means for your burn
strong	≥ 75	Green light — top of the cohort; register with confidence.
competitive	60–74	**The pass bar** — viable mid-pack seat, above-median earnings. Register.
marginal	45–59	Apply the listed fixes first — burning now locks in a weak seat.
weak	< 45	Do not burn on this subnet with this spec — re-spec or change subnet.

The pre-burn rule: register only when the simulation says **competitive (≥ 60) *and* above 1.00× vs median** — that is the cheapest insurance this guide can buy, because the alternative way to learn your score costs the burn plus weeks of rent. **Honest caveat** (the lab prints it too): this is a deterministic model of the judge built from its code — not the validator itself. Confidence % tells you how strong the evidence was; calibrate with small stakes. **When to re-run:** before your first registration (the gate note in Part E), after any spec/config change, whenever Monitoring shows incentive drifting down, and as part of the Part G.2 weekly cadence — the judge's weights shift when the subnet updates its validator.

PART I · THE SECOND PATH · $0 HARDWARE · ONE SMALL BURN

The CPU Path — Harnyx SN67, Script Mining

Everything up to here assumed a GPU. SN67 (Harnyx) is the second way in: **script mining**. Your miner is a Python agent script submitted to the subnet — validators execute it inside their own sandboxes and score the answers. There is no pod to rent, no CUDA to configure, no 24/7 box to babysit; the machine that matters is theirs, and a CPU is enough to build and submit. The app now ships a dedicated **CPU Guide** (nav **05**) that walks this path with live chain numbers and a per-user progress checklist — this part maps 1:1 to it. Entry is the cheapest seen on Finney: the registration burn floats around **0.05–0.08 τ (≈ $12–18)** against SN64's ~1 τ.

I-1 Read the economics — why SN67 is the low-risk entry

CPU Guide 05 → live strip: **SN67 economics**

The strip is the Opportunities scanner's own data, re-rendered as six live numbers. What they said at press time: **256/256 seats full** (burn is the entry path — the situation Part E describes), **217 miners earned** last epoch — an **~85% earn ratio**, the friendliest cohort in this guide by far (SN64 Chutes: 6.6%) — **~9.8 τ/day** flowing to miners, alpha at **0.0195 τ**, and a **top-10% incentive share of ~24%**: a wide spread where mid-pack miners keep their seats instead of feeding a whale. The base-case net from the scoring engine: **~$100/mo** (bear −$37, bull +$179). Re-read the strip weekly — if the alpha price bleeds or the spread tightens, the deal changed.

I-2 The nine phases — the golden order, CPU edition

The CPU Guide view tracks your progress phase by phase (marks persist per signed-in user). The order is the same discipline as always — **register LAST** — but phases 0–5 cost nothing and need no hardware beyond the machine you already own:

P0 · FREE

Size the deal

Opportunities filter "CPU-only" → SN67 scored 65.5, net ~$100/mo base case.

P1 · FREE

Wallets

Coldkey + hotkey on your box; record the hotkey address in the Wallets view.

P2 · FREE

Get the code

Clone harnyx/harnyx; install the harnyx-miner-sdk in a small venv.

P3 · FREE

Study the champion

Sync the judge profile (netuid 67) in Validator Lab; probe the top miner via MCP.

P4 · FREE

Build the agent

Targets: <30 s, ≤$0.50/1M, ≥25 tps, quality ~80% with citations.

P5 · FREE

Local eval

Spar with Validator Lab — re-simulate after every design change.

P6 · COSTS

Register

The ONE burn (~$12–18) — only on a green gate. btcli subnet register.

P7 · FREE

Submit script

Harnyx CLI's job — the app deliberately has no upload path.

P8 · FREE

Monitor & earn

Weekly strip check; MCP + Harnyx dashboard for scores; taostats for payouts.

I-3 The gate — Part H's rule, SN67's numbers

Sync the profile (Validator Lab 04 →netuid **67**→**Re-mine profile**): SN67's judge is **Market Clearing** with **Price 56%** + Quality 19% composite weight, a **~30 s** response deadline, a **~$0.50/1M** price reference and **~25 tps** throughput reference. The Part H verdict table applies verbatim. The pre-flight that proved the rule on this very subnet: a slow-but-deep agent (240 s, $0.60/1M, 72% quality) scored **41.9 — weak**; a tuned fast+cheap agent (25 s, $0.45/1M, 80%, 25 tps) scored **62.1 — competitive ≈ p90, 1.87× the median**. Same spec, same judge model, two different decisions: don't burn, and clear to burn.

I-4 The commands — four steps, the burn in the middle

YOUR LAPTOP · TERMINAL — CPU PATH · STEPS 1–4

```
$ git clone https://github.com/harnyx/harnyx.git && cd harnyx
$ python -m venv .venv && source .venv/bin/activate
(python 3.10+ is plenty — the SDK wraps the network protocol)
$ pip install -e harnyx-miner-sdk
build your agent inside the SDK harness · verify in Validator Lab (I-3)
$ btcli subnet register --netuid 67 --wallet.name harnyx --wallet.hotkey
sn67miner
burns ~0.05–0.08 τ — run ONLY when the gate says competitive or better
```

After registering, submit the agent through the repo's Harnyx miner tooling (the SDK's submission flow is the source of truth — it signs with your keys and hands the script to the validators). Your hotkey then earns on SN67 every epoch it clears the judge.

I-5 Honest boundaries — what the app does NOT do on this path

The platform's execution layer stays GPU-first by design, and on this path that is a feature, not a gap. Three things the app deliberately does not do: **no CPU pod provisioning** — the Deploy Wizard rents GPU pods only (CUDA is hardcoded end-to-end), and script mining needs no rented box; **no script submission** — the app has no upload path and doesn't need one, that's the Harnyx CLI's job (I-4); **no earnings ledger for self-managed miners** — My Miners tracks deployments the app provisions itself, so a CLI-run miner is invisible there. The ground truth for this miner lives in three places: the app's strip for economics, Harnyx's own dashboard (dashboard.harnyx.ai/benchmark) + MCP for your agent's scores, and taostats for the coldkey's actual payouts.

The one-line picture: the app decides *where* to mine and *whether to burn*; your CPU builds the agent; the Harnyx platform runs it; the app keeps watching whether the deal is still good.

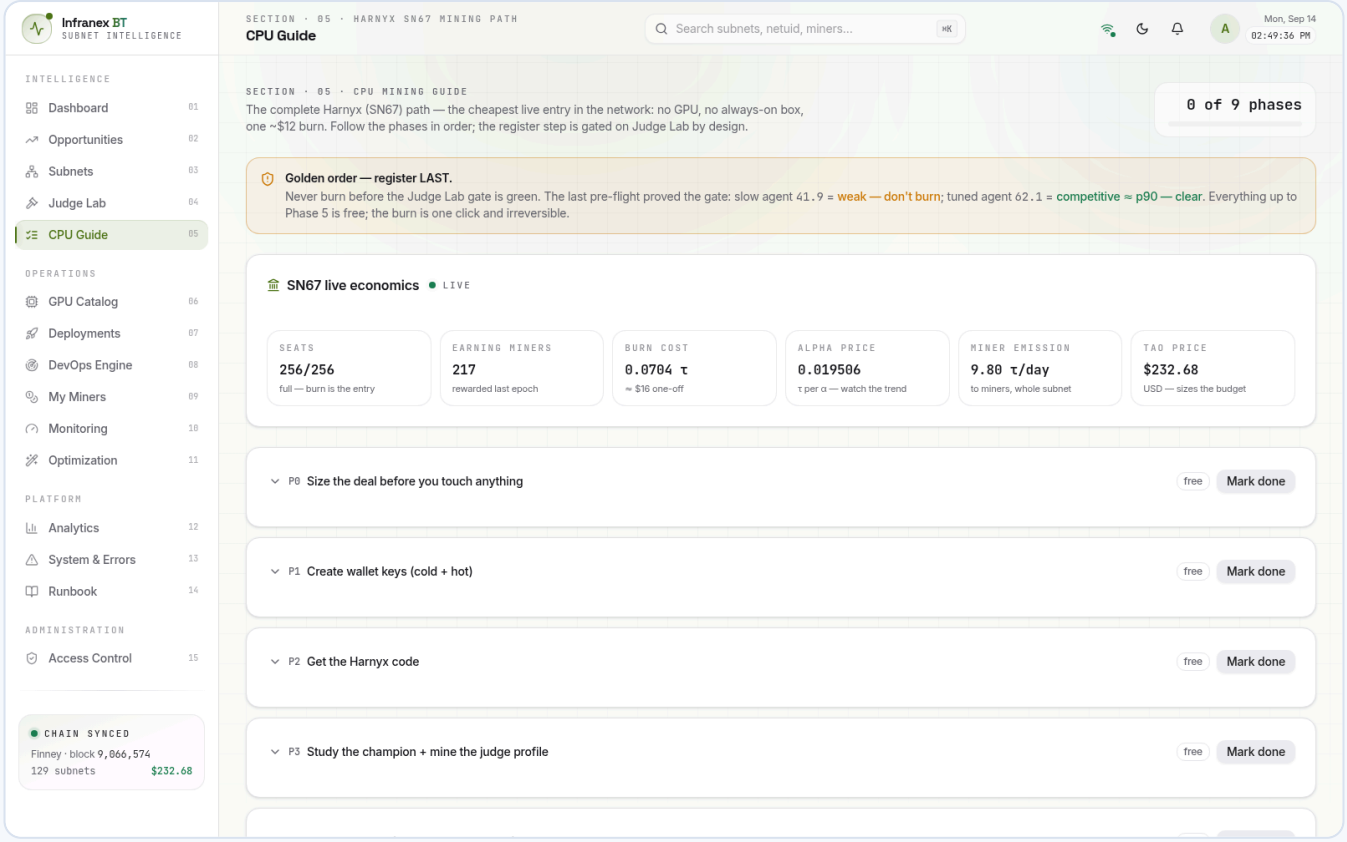


Fig I-1. The CPU Guide (nav 05) — top: the live SN67 economics strip (256/256 seats full, 217 earning miners, burn 0.0784 τ ≈ $18 at press time, alpha 0.0195 τ, 9.80 τ/day miner emission) and the golden-order banner with the gate numbers; below: the nine phase cards with copyable commands, cost badges and per-user progress counter, the Validator Lab gate card (41.9 weak vs 62.1 competitive), budget at a glance, and the honest-boundaries panel.

Budget at a glance: burn ~$12–18 one-off (irreversible, floats with seat demand — read the strip before executing) · hardware $0 — an existing CPU box is fine · expected net ~$100/mo base case. **Cadence:** the Part G.2 weekly routine applies here too, minus the pod checks — strip trend, judge re-mine after any subnet validator update, cohort spread. **Decision rule:** if the net expectation sits under your target for weeks straight, deregister and move the (small) burn elsewhere — that flexibility is the whole point of the cheap seat.

CHAPTER 7 · ~10 MINUTES · EXCHANGE-SIDE

Funding from Binance — Click by Click

This is a plain deposit into your own wallet — no burn happens here. The only two ways to get this wrong are the **network** and the **address**, so both are double-checked below.

B-1 Copy your coldkey address from btcli (never from memory)

Run `btcli wallet overview --wallet_name infranex` and copy the coldkey (¢) address — the one **not** labeled ℏ. Both start with 5; the ¢/ℏ marker is what tells them apart. If the marker is unclear, the address under your **coldkey** name is the funds wallet.

B-2 In Binance: Spot → Withdraw → TAO

- **Step 1:** Binance app or web → **Assets → Spot → Withdraw** → search **TAO (Bittensor)**.
- **Step 2:** choose **Withdraw → Crypto**. Paste your coldkey address. Use **Binance mobile app** or web — both work identically.
- **Step 3: Network: choose "Bittensor (native)"**. This is the critical choice — TAO is its own chain. If a network dropdown shows anything else (a wrapped or bridged variant), stay on native Bittensor.
- **Step 4: No memo/tag is needed**. If Binance asks for one, you are on the wrong network — stop and re-check.
- **Step 5:** Amount: burn fee + ~2 TAO. Binance shows its withdrawal fee on the confirmation screen before you confirm.
- **Step 6:** Verify the **first 6 and last 6 characters** of the pasted address against the btcli output, then confirm (2FA). Arrival is typically minutes; Binance occasionally pauses TAO withdrawals during chain upgrades — the button greys out and resumes later.

B-3 First time? Send a test amount first

Send 0.5 TAO, wait for it to land (check `btcli wallet overview` — the balance appears next to the coldkey), then send the rest. Costs one extra network fee; buys certainty on a chain you have never used.

YOUR LAPTOP · TERMINAL — AFTER FUNDING

```
$ btcli wallet overview --wallet_name infranex
Wallet: infranex   ✓- coldkey: 5CA1abcdef...  →  balance: 2.0493 TAO
```

Three rules of exchange funding. Native Bittensor network only. Coldkey address only (hotkeys are not for holding funds). Address check by first/last 6 characters every single time — there is no undo on a blockchain transfer.

CHAPTER 8 · WHEN SOMETHING LOOKS OFF

Troubleshooting & Health Checks

The System & Errors view runs live checks on every dependency. It is also the first place to look when anything in the pipeline misbehaves — most incidents are one of the five below.

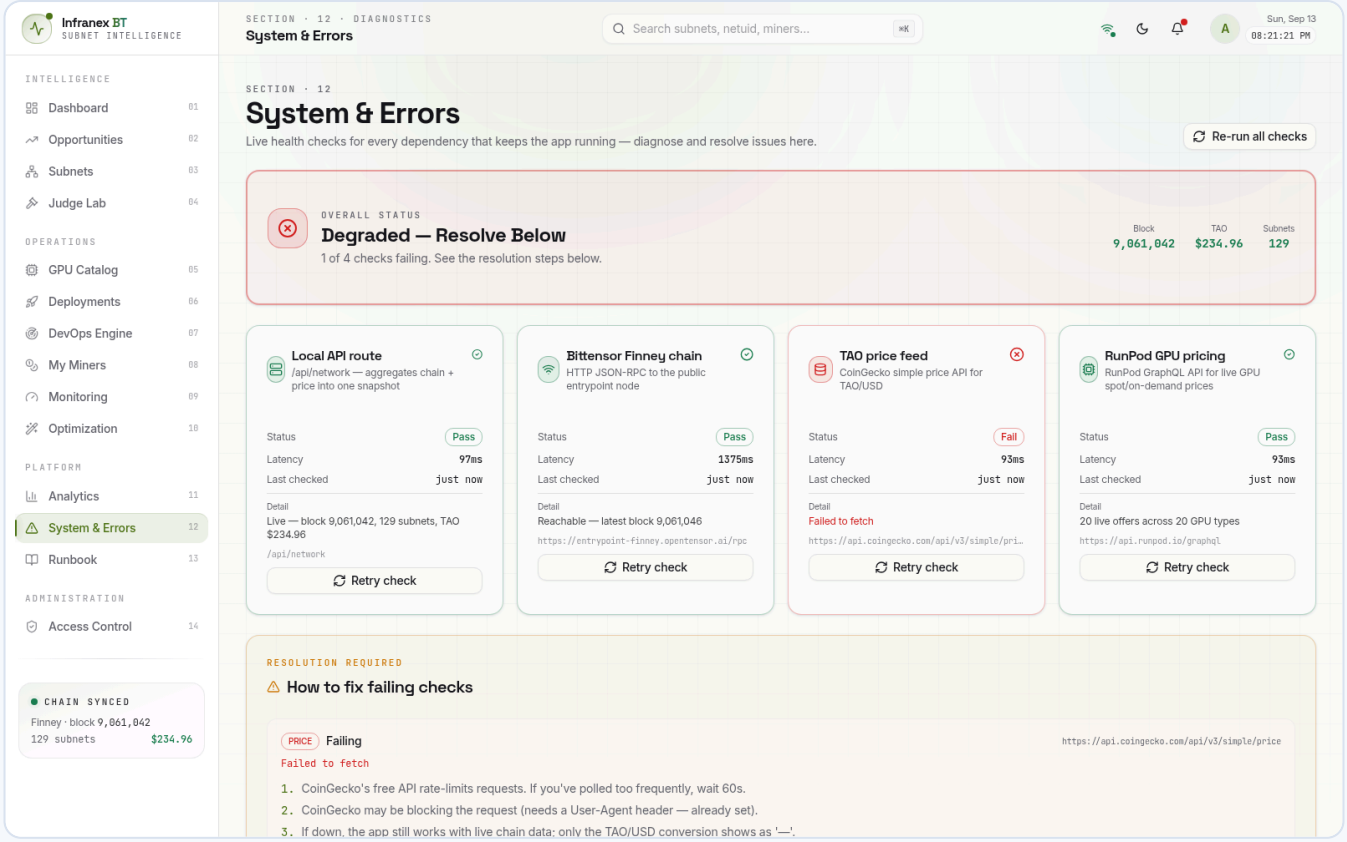


Fig 8-1. System & Errors health board. In this capture the TAO price feed is rate-limited by CoinGecko (transient — the platform keeps working on live chain data and self-heals on retry); chain, API and RunPod pricing are green.

Symptom	Fix
Verify says "Not registered yet"	Registration takes one block to land (~12 s). Wait and click Verify again. Also confirm you registered with the same wallet/hotkey names you pasted.
GPU step shows no offers	No provider key saved, or the saved key lost validity. GPU Catalog → Provider API keys → re-paste and **Save & verify**.
Install step stuck	Open the deployment card's install log — each of the ~10 steps shows its output. Pod-boot delays up to ~2 min are normal on RunPod; a red step tells you exactly which script failed.
Restart pending won't clear	The restart runs over SSH to the pod. Check the pod is running (RunPod console) and the deployment's ssh port is reachable; then press **Restart miner** again.
Emissions stay 0	Normal for the first minutes — validators must probe your axon before scoring. Watch the incentive % on the UID Defense panel; if it stays flat for hours, check axon reachability from the Runbook panel.

You're mining.

Wallet created offline. Keys where they belong. GPU rented at the right second. Registered last. The platform watches the rest — spend, earnings, axon health, deregistration risk — every 90 seconds, with alerts when anything drifts.

- ✓ Coldkey mnemonic — on paper, in your safe, never online
- ✓ Hotkey file — on the pod only, via your own SCP
- ✓ RunPod key — encrypted at rest, audited on every change
- ✓ Registration burn — recorded on the deployment card
- ✓ P&L — real accruals from the first minute, not estimates
- ✓ CPU path — SN67 script mining available from any CPU, gated the same way (Part I)

Next moves when ready: scale to a second miner (same wallet, new hotkey — Part B again), or wire a real Slack/Discord webhook so alerts follow you. And when you terminate any pod — Terminate, never Stop.

OPERATOR GUIDE · LAPTOP TO LIVE

Start a CPU Miner on Your Own Machine

The complete workflow, in order: connect your laptop, create and fund the wallet, run the setup from the Deploy stepper, pass the free Validator Lab gate — and only then burn to register. Seven steps, one burn, $0 in rent.

Target subnet — SN67 · Harnyx (CPU-only, agents & logic)
Platform — Infranex control plane · Deployments · Validator Lab · DevOps
Prepared — September 20, 2026

01 · THE MAP

The workflow at a glance

Mining SN67 Harnyx with your laptop is a **sequence**, not a pile of screens. Three things must exist before any money moves: your machine must be reachable by the app, your wallet keys must exist **on that machine**, and the subnet's software must be installed there. Only after a free pre-flight passes do you spend anything — one burn of roughly **0.0433 TAO (≈ $11.62 at $268.33/TAO, block 9,101,516)** to register. Everything else in the workflow is free.

The order below is deliberate. The laptop comes first because every later command — wallet creation, subnet registration, the miner itself — executes **on your machine** through the agent you enroll in step 1. The wallet comes before registration because the register command consumes TAO from the coldkey that lives on the laptop. The Validator Lab gate comes before the burn because registration is irreversible: once a hotkey takes a seat on SN67, that TAO is gone whether or not your agent ever earns. Follow the order and you never pay before you are ready.

1 **Connect your local machine** FREE

CPU Catalog → "Or skip the cloud" → name it, run two commands in Ubuntu (WSL2 on Windows). The agent dials out to the app — no open ports, no inbound firewall rules.

2 **Create & fund the wallet** FREE NEEDS TAO

Generate coldkey + hotkey on the laptop with `btcli`, back up the mnemonic on paper, then transfer ~0.1 TAO to the coldkey so the burn in step 5 has funds.

3 **Pick SN67 & run setup** FREE

Deployments stepper: step 1 pick SN67, step 2 "Local machine", step 3 "Start setup on laptop" — the agent clones the Harnyx repo and installs the Python 3.11 stack.

4 **Pass the Validator Lab gate** FREE

Sync netuid 67, read the validator's scoring shape, simulate your agent, iterate until the verdict is competitive. **No green verdict — no burn.**

5 **Register — the ONE burn** BURN ≈ $11.62

Step 4 of the stepper: "Send register command to laptop". This takes a seat on SN67 and is irreversible — the gate from step 4 must be green first.

6 **Submit the agent & go live** FREE

Package your agent through the harnyx-miner-sdk per the repo's own flow. Validators execute it in sandboxes and score it every epoch.

7 **Monitor & earn** FREE

Weekly check: Opportunities strip, Harnyx dashboard for your agent's rank, taostats for real payouts. If the deal dies, deregister and move the burn elsewhere.

01 · CONTINUED

When to do what — the master schedule

This table answers the three questions every first-time operator asks: **when do I connect the wallet, when do I connect the local machine, and when do I register?** Print it or keep it open — each row links back to the step where the full instructions live.

Milestone	Where in the app	When	Cost
Connect local machine	CPU Catalog → "Or skip the cloud"	First — once, before anything else	`free`
Create cold + hot keys	Terminal on the laptop (btcli)	After the machine is online	`free`
Fund the coldkey	Exchange or your main TAO wallet	Right after key creation, before step 5	`~0.1 TAO` moved
Wallet / hotkey fields	Deploy stepper (prefilled)	Step 3 review, before launching	`free`
Validator Lab gate	Validator Lab → netuid 67	Before ANY burn — no exceptions	`free`
Register on-chain	Deploy stepper → step 4 burn gate	Only after the verdict is green	`burn 0.0433 τ`
Submit the agent	harnyx-miner-sdk CLI on laptop	Immediately after registration	`free`
Monitor & review	Opportunities, Monitoring, DevOps	Weekly, forever	`free`

> **Why the wallet must be created on the laptop:** the register command runs through your machine's agent, so `btcli` looks for the wallet files in `~/.bittensor/wallets/` **there**. Keys created on another computer are invisible to it. Never paste a mnemonic into the app — it stores metadata only.

02 · PREPARATION

Before you start — the checklist

The whole workflow takes 30–60 minutes the first time, spread across your browser and one terminal window. Nothing in it needs a GPU, Docker, or CUDA — SN67 is an agents-and-logic subnet where **validators run your script in their own sandboxes**, so your laptop only has to be good enough to hold the repo and talk to the network. Confirm every item below before step 1; each one exists because skipping it is the usual cause of a failed or stalled deployment later.

- ☐ **A laptop or desktop with 2+ CPU cores, 4–8 GB RAM and ~10 GB free disk.** The Harnyx stack is a Python workspace, not a model — typical CPU-subnet requirements are tiny compared to a GPU box (the catalog's floor is 2 vCPU / 4 GB / 40 GB, and SN67 needs well under that).

- [ ] **Ubuntu available.** On Windows: open PowerShell and run `wsl --install -d Ubuntu`, then reopen and continue in Ubuntu. On Linux or macOS the native terminal works as-is.
- [ ] **A stable internet connection.** The agent pulls commands over HTTPS and the miner talks to the network continuously — a machine that sleeps or disconnects stops earning.
- [ ] **Access to the Infranex app** — sign in with your user and access code. Every screen named in this guide (CPU Catalog, Deployments, Validator Lab, DevOps) lives in the sidebar.
- [ ] **~0.1 TAO reachable** on an exchange or an existing wallet you control. This is **transferred**, not spent: ~0.0433 TAO covers the registration burn and the rest stays in your coldkey as buffer and fees.
- [ ] **Honest expectations.** SN67 recently showed ~117–222 rewarded seats, a burn of 0.0433 τ and per-earning-miner estimates that ranged widely across reads (~$100/mo net base-case to ~$1,282/mo in fresher snapshots). Emissions move — re-read the live Opportunities row before committing, and treat every number in this guide as a snapshot of September 2026.

> **Why SN67 is the recommended target:** it is the best CPU-classified subnet in the fresh scan — no GPU needed, no Docker, no CUDA, validators execute your agent, ~117 miners currently rewarded, and the cheapest meaningful entry burn in the network (≈ $11.62 vs ~1 τ on heavier subnets). Runner-ups: SN45 AlphaRidge and SN62 Ridges.

03 · STEP 1 — DAY ZERO

Connect your local machine (your laptop)

Do this **first**. Every command in the rest of this guide — setup, wallet creation, the register burn — is queued through the Infranex agent that you install in this step. The agent is a small Python process that makes **outbound-only** HTTPS calls to the app, signs every request with HMAC, and pulls its own command queue. You never open a port on your router and never expose SSH; if you close the terminal, the machine simply shows offline until you start it again.

IN APP **CPU Catalog** → scroll to "Or skip the cloud — connect your local machine" (the same flow also lives in **DevOps → Local machines**)

Enter a machine name (something you will recognize later, like `hp-ultra5-laptop`) and press **Connect my laptop**. The app mints a one-time

enrollment token and shows two commands. Run them in your Ubuntu terminal — paste the first exactly as shown (the token is valid for 30 minutes and works once), then start the agent with the second:

```
# 1 — enroll (the app shows this filled with your real server + token)
curl -fsSL <app-url>/api/agent/agent.py -o /tmp/infranex-agent.py && \
  python3 /tmp/infranex-agent.py enroll --server <app-url> --token <ONE-TIME-TOKE
N>

# 2 — start the agent (keep it alive)
python3 /tmp/infranex-agent.py run

# keep-alive option: run inside tmux, or:
nohup python3 /tmp/infranex-agent.py run &
```

Within a few seconds the machine flips to an **online** chip in the catalog and in DevOps → Local machines, together with its specs (CPU model, cores, RAM, OS, Python version). The agent polls the app roughly every ten seconds, so a queued command lands on your laptop almost immediately and its output streams back into the app. Revoke the machine at any time from DevOps to cut its access instantly.

> **Keep it running:** the agent must stay alive for the machine to stay online. Use `tmux` or the `nohup … &` form above, and disable automatic sleep on the laptop — a suspended machine cannot mine and cannot register. Windows users: run everything inside the WSL2 Ubuntu window and leave it open.

04 · STEP 2 — KEYS & FUNDS

Create and fund the wallet (before any register)

A Bittensor wallet is two keys: a **coldkey** that holds your funds and signs, and a **hotkey** that does the work and serves. Both are generated locally with `btcli` — on the laptop, not in the app and not on another computer — because the register command in step 5 runs there and looks for the key files in `~/.bittensor/wallets/`. The names you pick here are exactly the ones you will see prefilled in the Deploy stepper later; this guide uses `harnyx` and `sn67miner`. If `btcli` is missing, install it first with `pip install bittensor` (the Harnyx venv from step 3 also ships it).

```
# run these on the laptop, inside Ubuntu
btcli wallet new-coldkey --wallet.name harnyx --wallet.path ~/.bittensor/wallets
btcli wallet new-hotkey  --wallet.name harnyx --wallet.hotkey sn67miner

# check the addresses when done
btcli wallet overview --wallet.name harnyx
```

When the coldkey is generated, `btcli` shows a mnemonic — a sequence of words that **is** the wallet. Write it on paper and store it offline; if it leaks, your TAO leaks, and no support desk can restore it. Copy the two ss58 addresses into the app's **Wallets** view for reference — it stores records and metadata only, never mnemonics.

Fund the coldkey now, while you are here: withdraw or transfer about **0.1 TAO** from your exchange or main wallet to the coldkey's ss58 address. The transfer itself is not a cost — it becomes the burn in step 5 (~0.0433 τ) plus a safety margin for fees and price movement. A registration attempt with an empty coldkey fails on-chain, so this is the single most common "register failed" cause. Give the transfer a few minutes to confirm before moving on.

> **The mnemonic never goes into the app, a screenshot, or a chat message.** Paper only. The app never asks for it, and any flow that does is not part of this workflow.

05 · STEP 3 — SETUP

Pick SN67 and run the setup from the Deploy stepper

With the machine online and the wallet funded, everything else happens on one screen: **Deployments**. The four-step stepper at the top of the page is the same wizard used for GPU subnets — for a CPU subnet it simply swaps the GPU-provider step for a compute choice, and the final step knows your burn is deliberate. The fastest entry is the **Start CPU miner** button on your machine's row in the CPU Catalog: it opens the stepper with SN67 and your laptop already preselected. Manually, it looks like this:

A

Stepper step 1 — Subnet: pick SN67 Harnyx

The row is tagged CPU-only (minimum VRAM 0). Picking it reshapes the wizard: step 2 becomes "Compute — cloud, or your laptop" and the requirement strip reads "CPU-only subnet — no GPU needed".

B

Stepper step 2 — Compute: "Local machine" → your laptop

Two target cards appear: CPU provider (rented Hetzner/DO boxes — skip it) and Local machine. Pick **Local machine**, then click your laptop from the list of online hosts. Cost shows **$0 — your hardware, your electricity**. Confirm the wallet and hotkey fields: they default to your saved wallet profile or `infranex` / `sn67miner` — set them to `harnyx` / `sn67miner` to match step 2 of this guide.

C

Stepper step 3 — "Start setup on laptop"

One click queues the whole SN67 setup as a single agent command: clone the Harnyx repo into `~/infranex/sn67`, install `uv`, fetch Python 3.11 (the repo pins `>=3.11,<3.12`) and sync the workspace including `harnyx-miner-sdk`. Live output streams into the pane; when the agent reports done, step 4 opens by itself.

```
# what "Start setup on laptop" runs on your machine (one command):
mkdir -p ~/infranex/sn67 && cd ~/infranex/sn67 \
  && (git clone https://github.com/harnyx/harnyx.git . || git pull) \
  && curl -LsSf https://astral.sh/uv/install.sh | sh \
  && uv python install 3.11 && uv sync --python 3.11 \
  && pip install -e harnyx-miner-sdk && echo SETUP-OK
```

If the setup fails (network hiccup, git conflict), the pane shows the failing lines and a **re-queue** button — fix the obvious cause if there is one and queue it again; nothing has been spent. The command feed stays available afterwards under **DevOps → Local machines**, which is also where you can run quick manual commands on the machine at any time.

06 · STEP 4 — PRE-FLIGHT

Pass the Validator Lab gate — free, before any burn

SN67's validators score agents on a visible shape, and you can read that shape **before** spending anything. Open **Validator Lab** and sync netuid 67 — the profile arrives in about fifteen seconds from the subnet's repo and the chain. The last mined shape carried **Price ≈ 56%** of reward weight with **Quality ≈ 19%**, a **~30-second** response deadline, a **~$0.50 per 1M-token** price reference and a **~25 tokens-per-second** throughput reference. Write those three numbers down: fast, cheap, good-enough with citations — that is

what wins on this subnet. Shapes change; re-sync before you trust the numbers.

> IN APP **Validator Lab** → Sync netuid 67 → read the champion & scoring shape → simulate your agent's numbers

Then use the lab as a sparring partner. Drag the sliders to your agent's real numbers — planned response time, price per million tokens, expected quality, throughput — and read the verdict: weak, below-median, or **competitive**, with the percentile position. The proof from the last pre-flight: a slow-but-deep agent (240 s, $0.60, 72% quality) scored 41.9 — weak; a tuned fast-and-cheap one scored 62.1 — competitive, roughly the 90th percentile. Iterate in minutes, for free, until the composite clears the bar. Only your CPU and your time are at stake here.

> **The gate rule that protects your TAO: no green verdict — no burn.** The register button in the next step exists precisely for the moment this gate is green, not before. If your agent is not ready yet, stop here and come back — the seat price may wiggle, but it has stayed in the cents-of-a-TAO range, and patience is cheaper than a wasted burn.

07 · STEP 5 — THE BURN

Register on-chain — the one irreversible click

Back on **Deployments**, the stepper's step 4 is the go-live gate. With setup done, it shows the **burn card**: a deliberate, clearly-labeled action that queues the registration on your laptop — nothing is sent on-chain by the app itself until you press it. Before pressing, do the three-second sanity check: the Validator Lab verdict is competitive, the burn shown in the subnet strip is still around 0.0433 τ (it floats with seat demand), and the coldkey you funded in step 2 is the wallet named on the card. Then press **Send register command to laptop**.

```
# what the button queues on your laptop (coldkey signs, hotkey claims the seat):
btcli subnet register --netuid 67 --wallet.name harnyx --wallet.hotkey sn67miner
```

The command consumes **~0.0433 τ (≈ $11.62)** from the coldkey and takes a seat on SN67 for your hotkey. Verify it worked: the hotkey should appear in SN67's metagraph (`btcli subnets metagraph --netuid 67`, or the Subnets view in the app), and the seat starts with an **immunity window** before it can be de-

registered by weight competition — use that window to get your agent submitted and scoring, not to celebrate. From this point your laptop is the miner's home: keep it awake, keep the agent alive, and keep the terminal (or `tmux` session) running.

> **This is the point of no return.** The burn is not refundable and the seat is not transferable. Everything before this step was free by design; everything after it earns or does not earn based on your agent's quality. If any pre-flight check is uncertain, walk back to step 4 — the burn has waited this long, it can wait another hour.

08 · STEP 6 — GO LIVE

Submit the agent and start serving

Registration claims the seat; the **agent script** is what earns on it. SN67's miner is a Python agent built inside the `harnyx-miner-sdk` harness that step 3 installed. Package your agent per the repo's own submission flow — the SDK's job, not the app's (the app deliberately has no upload path) — and submit it with your Harnyx wallet keys: the coldkey signs, the hotkey serves. Validators pick the agent up, execute it in their sandboxes against live queries, and score it every epoch.

- **Keep responses under the ~30 s deadline** — slow answers lose on Market Clearing no matter how good they are.
- **Price at or below the ~$0.50/1M reference** and hold quality ⩾ 80% with citations — price carries the majority of the reward weight.
- **Target ⩾ 25 tps throughput** — the validator rewards speed per token, not just total time.
- **Watch the first epochs** on `api.harnyx.ai/mcp` and `dashboard.harnyx.ai/benchmark` before calling it done — your agent's rank there is the ground truth the chain rewards against.

If the first epochs score poorly, do not panic and do not re-burn anywhere else: tune the agent against the Validator Lab shape (latency down, price down, quality held) and resubmit through the SDK. Iteration after registration is free — it is only the seat that cost TAO, and the seat is yours until weights push it out, which the immunity window gives you time to prevent.

09 · STEP 7 — STEADY STATE

Monitor & earn — the weekly ritual

A miner is a small business; give it a ten-minute weekly review. Emissions on SN67 move — recently ~9.8 τ/day spread across a couple hundred rewarded seats with a wide spread where the top 10% of miners hold roughly a quarter of the incentive — so mid-pack miners keep their seats, but the deal can still die slowly. The weekly check exists to catch that before it eats your ROI, and every surface you need already lives in the app you are running.

Check	Where	Healthy signal
Deal economics	Opportunities strip and the SN67 row (alpha price, emission/day, seat churn)	Alpha price stable or rising; your seat still rewarded
Agent rank	api.harnyx.ai/mcp and dashboard.harnyx.ai/benchmark	Composite at or above the competitive band (~p90 ≈ 62 on the last read)
Real payouts	taostats — your coldkey's TAO flow	Steady or growing weekly TAO arrivals
Machine health	DevOps → Local machines (load, memory, temp, agent online)	Agent online, temp < 78 °C, no failed commands
Decision rule	Opportunities + Monitoring, weekly	If net expectation stays under target for weeks — deregister and move the burn

10 · REFERENCE

Command cheat sheet & troubleshooting

Every terminal command in this workflow, in the order you will actually meet them. The app builds the enrollment and setup lines for you (with real URLs and tokens) — the ones below show the shape they take.

Action	Command (on the laptop, Ubuntu)
Windows: get Ubuntu	`wsl --install -d Ubuntu`
Enroll the agent	`curl -fsSL <app-url>/api/agent/agent.py -o /tmp/infranex-agent.py && python3 /tmp/infranex-agent.py enroll --server <app-url> --token <TOKEN>`
Run the agent	`nohup python3 /tmp/infranex-agent.py run &` (or inside tmux)
Create coldkey	`btcli wallet new-coldkey --wallet.name harnyx --wallet.path ~/.bittensor/wallets`
Create hotkey	`btcli wallet new-hotkey --wallet.name harnyx --wallet.hotkey sn67miner`
Register — BURN	`btcli subnet register --netuid 67 --wallet.name harnyx --wallet.hotkey sn67miner`
Verify the seat	`btcli subnets metagraph --netuid 67`

Symptom	Cause → fix
Machine stuck "awaiting enroll"	The token is one-time and expires in 30 minutes → press **Connect my laptop** again and run the fresh command.
Machine shows offline	Agent process died or laptop slept → restart python3 /tmp/infranex-agent.py run inside tmux; disable automatic sleep.
Setup failed in stepper step 3	Network or git hiccup → read the output pane, press the re-queue button; nothing was spent.
Register button disabled	Setup not reported done, or the host is offline → wait for the green setup state with the agent online.
Register failed on-chain	Coldkey not on the laptop, or insufficient TAO → repeat step 2 of this guide; check balance with btcli wallet overview.
Not earning after go-live	Agent scoring below the band → back to Validator Lab: cut latency, cut price, hold quality, resubmit via the SDK.

> **The workflow in one breath:** connect the laptop → create & fund the wallet on it → pick SN67 in the stepper and run setup → pass the free Validator Lab gate → send the one burn → submit the agent → review weekly. Free until the gate is green; one burn when it is.

Free until the gate is green. One burn when it is.

That is the whole discipline. Your laptop is the mine, the Validator Lab is the safety inspector, and the burn is the only money that ever moves — deliberate, gated, and reversible by nothing. Keep the agent alive, keep the numbers honest, and review the deal every week.

Numbers in this guide were snapshots from September 2026 (block 9,101,516 · TAO $268.33 · SN67 burn 0.0433 τ). Emissions, seat prices and validator shapes move — re-read the live Opportunities row before committing budget, and treat this guide as the method, not today's price.

DEPLOYMENT & HOSTING GUIDE

InfraNexBT on DigitalOcean

Verified 2026 pricing for Droplets, App Platform, Managed PostgreSQL and GPU Droplets, benchmarked against Hetzner, RunPod and Vast.ai, with a step-by-step production blueprint.

InfraNex Platform Operations

September 2026 · Next.js 16 · Bittensor control plane

Table of Contents

1. The Best List (TL;DR) 2

1.1 Which pick matches which scenario . . . 2

2. What This App Needs to Run 3

2.1 Required environment variables . . . 3

2.2 Background work and process model . . . 4

3. DigitalOcean Deep Dive: Hosting the Web App 5

3.1 App Platform versus Droplets . . . 5

3.2 Managed PostgreSQL and DOKS . . . 5

4. DigitalOcean GPU Droplets: The Mining Angle 6

4.1 The honest mining economics . . . 6

5. Cross-Provider Comparison 7

5.1 GPU providers versus DigitalOcean . . . 7

6. Production Blueprint: Deploy on a DigitalOcean Droplet 8

6.1 Steps 1-4: create, harden, install, configure . . . 8

6.2 Steps 5-7: build, supervise, serve over TLS . . . 9

6.3 Steps 8-10: backups, monitoring, updates . . . 9

7. Cost Scenarios and FAQ 10

7.1 Frequently asked questions . . . 10

1. The Best List (TL;DR)

This guide answers the deployment and hosting questions for the InfraNexBT web platform and delivers a researched, price-verified DigitalOcean list as of September 2026. The short version: the application is a single Next.js 16 standalone server with a SQLite database, a small set of background workers, and outbound integrations to SSH hosts, the Bittensor chain, GitHub, and cloud providers. That profile fits a modest Virtual Private Server perfectly, and a DigitalOcean Droplet is the strongest overall fit because the same account also acts as a provisioning provider inside the platform itself. The table below ranks every option this guide covers; the chapters that follow justify each pick with verified pricing and trade-offs.

Pick	Choice	Verified price	Why
Best overall (web app)	DO Droplet, Basic 2 vCPU / 4 GB	$24/mo ($0.0357/hr)	Comfortable build + run headroom, full filesystem control, snapshots
Best budget	DO Droplet, 1 vCPU / 1 GB	$6/mo	Runs the prebuilt standalone server fine; build off-board or add swap
Best zero-ops	DO App Platform, Basic web + worker	from $10/mo (2 services)	GitHub-to-deploy with no server admin; forces DB migration off SQLite
Managed database	DO Managed PostgreSQL	from $15/mo (HA $30/mo)	Automatic backups and failover when the app outgrows SQLite
Cheapest DO GPU	NVIDIA RTX 4000 Ada (20 GB)	$0.76/hr	Only DigitalOcean GPU whose hourly rate can plausibly pay for itself in mining trials
Balanced DO GPU	RTX 6000 Ada or L40S (48 GB)	$1.57/hr	Serious inference or validation workloads in Toronto (TOR1)
High-end DO GPU	H100 80 GB (SXM)	from about $3.39/hr; 8x node $35.28/hr	AI training class hardware; rarely economic for subnet mining
Budget GPU alternative	RunPod RTX 4090 (24 GB)	$0.34-$0.69/hr	Cheapest integrated path for experiments; community cloud is spot-priced

Table 1: The ranked picks with verified September 2026 pricing.

$24/mo
Recommended control-plane Droplet (2 vCPU / 4 GB)

$0.76/hr
Cheapest DigitalOcean GPU (RTX 4000 Ada, TOR1)

$15/mo
Managed PostgreSQL single node (1 vCPU / 1 GiB)

1.1 Which pick matches which scenario

Scenario	Deploy this	Estimated monthly
Just run the dashboard for a few users	1 vCPU / 1 GB Droplet + swap + snapshots	$7-$8
Production control plane	2 vCPU / 4 GB Droplet + nightly SQLite backups + Caddy TLS	$29-$34
Hands-off operation, no server admin	App Platform web + worker + Managed PostgreSQL	$35-$40
Trial GPU mining on DigitalOcean	RTX 4000 Ada GPU Droplet in TOR1, hourly, stop-loss checked	~$554 if 24/7
Multi-provider miner fleet	$24 Droplet control plane + RunPod/Vast GPU workers behind DevOps Engine	$34 + metered GPU

Table 2: Scenario-to-pick mapping with realistic all-in monthly cost.

Two honest caveats belong up front. First, DigitalOcean GPU rates are competitive with mainstream clouds but are undercut by RunPod and Vast.ai for the consumer-class cards that dominate Bittensor mining, so keep the control plane and the mining fleet decisions separate. Second, the $24 Droplet recommendation assumes the platform keeps its SQLite database; if you prefer managed PostgreSQL from day one, the same Droplet plus the $15 database lands near $39 per month before snapshots. Both paths are detailed in chapters 3 and 6.

2. What This App Needs to Run

Hosting advice grounded in the actual codebase beats generic Next.js checklists, so this chapter states the verified runtime profile of the InfraNexBT platform. The app builds to a standalone Next.js 16 server (React 19, Tailwind 4, shadcn/ui) and is started with Bun via bun .next/standalone/server.js on port 3000. Data lives in a single-file Prisma 6 SQLite database at db/custom.db, which means hosting requires a persistent filesystem and file-level backups rather than a stateless container. The dependency list includes ssh2 (the platform opens real SSH sessions to provisioned hosts), the Polkadot API (live Bittensor chain reads), and z-ai-web-dev-sdk (AI analysis features), so the server needs unrestricted outbound HTTPS plus outbound SSH, and accurate clock sync because the daemon channel signs requests with time-bounded HMAC signatures.

2.1 Required environment variables

Variable	Required	Purpose
DATABASE_URL	Yes	Prisma connection; file:/opt/infranex/db/custom.db on a Droplet
APP_SESSION_SECRET	Yes	Signs session cookies; rotating it logs out all users
DEVOPS_SECRET	Yes	HMAC key for the daemon command/telemetry channel; lost if regenerating it invalidates paired miners
INFRANEX_REPO_ROOT	Production	Absolute project path used by the standalone server start command
RUNPOD_API_KEY	Optional	Fallback RunPod credential; provider keys are normally stored in the database via the UI
INFRANEX_PLATFORM_URL	Optional	Self-referential public URL used by the daemon bridge

Table 3: Environment variables referenced by the codebase (verified via source scan).

2.2 Background work and process model

In the current development setup seven background workers cycle inside the dev server, and maintenance scripts such as the chain reconcile job run under Bun. On a production Droplet the web server becomes one systemd unit, and recurring jobs become systemd timers or cron entries calling bun scripts/.ts. Nothing in the codebase requires a message broker; the daemon command queue lives in the application database and is drained by miners polling /api/daemon/commands every 60 seconds. That design keeps a single small server viable and is the main reason Kubernetes is not recommended here. RAM is the one real constraint: the production build (next build) needs roughly 2 GB, so either pick the 4 GB tier or build the artifact elsewhere and copy .next/standalone to a 1 GB box.

One warning about the repository root: the checked-in docker-compose.yml is a stale template from the original scaffold. It references a Python backend on port 8000, a Supabase Postgres image, and a Redis worker that do not exist in this application. Do not deploy from it, and do not let an automation tool discover it and assume it is the intended stack. The supported topology is the standalone server plus systemd, which chapter 6 walks through step by step.

3. DigitalOcean Deep Dive: Hosting the Web App

DigitalOcean offers four plausible homes for this application: classic Droplets, the App Platform PaaS, Managed PostgreSQL for the data layer, and Kubernetes (DOKS). Verified September 2026 pricing makes the comparison unusually one-sided for a workload like this one. Droplets bill per second with a 60-second minimum, usage is capped at 672 hours per month, and each plan includes a monthly outbound transfer allowance, so a control plane that idles at low CPU costs exactly its plan price.

Droplet plan (Basic)	vCPU / RAM / Disk	Price	Fit for this app
Entry	1 vCPU / 512 MB - 1 GB / 10-25 GB	from $4/mo	Too small to build; runs prebuilt artifact with swap
s-1vcpu-1gb	1 / 1 GB / 25 GB	$6/mo	Budget pick; add 2 GB swap for build safety
s-2vcpu-2gb	2 / 2 GB / 60 GB	$18/mo	Workable; tight during next build
s-2vcpu-4gb	2 / 4 GB / 80 GB	$24/mo ($0.0357/hr)	Recommended; build and run on one box
Premium AMD/Intel	2 / 4 GB and up	about 15-30% over Basic	Faster clocks and NVMe for the same footprint

Table 4: Basic Droplet tiers relevant to this app (DigitalOcean pricing, verified Aug-Sep 2026).

3.1 App Platform versus Droplets

App Platform is DigitalOcean's managed PaaS: connect the GitHub repo, it builds and runs the container, handles TLS, and restarts on failure. Basic web services start at $5 per month, static sites are free, dedicated instances scale up to $156 per month, and each component bills separately, so a web service plus a background worker starts at $10 per month. The honest catch for this codebase is persistence: App Platform containers have ephemeral filesystems, so the SQLite file would be wiped on every deploy. Adopting App Platform therefore implies migrating Prisma from the sqlite provider to Managed PostgreSQL, which is a schema-provider change plus a data migration, not just a connection-string swap. For teams that never want to touch a server, that migration is worth it; for a single-server operator, a $24 Droplet with snapshots delivers the same reliability for less money and zero code changes.

3.2 Managed PostgreSQL and DOKS

Managed PostgreSQL starts at $15 per month for a single node with 1 vCPU and 1 GiB of RAM, with automatic daily backups and encrypted at rest; high-availability two-node clusters start at $30 per month. It is

the right upgrade when the platform accumulates real operational history that cannot be re-scraped, or when you want point-in-time recovery instead of snapshot-based file backups. Managed Kubernetes (DOKS) starts around $12 per month for the control plane alone before worker nodes, and given that the whole application is one process plus cron-style jobs, DOKS adds operational surface without adding capability. Revisit it only if the platform becomes multi-tenant with isolated customer workloads.

Verdict: a Basic 2 vCPU / 4 GB Droplet at $24 per month running the standalone server with SQLite, Caddy for TLS, and nightly file backups is the recommended configuration. Treat Managed PostgreSQL as a paid upgrade path, App Platform as the zero-ops alternative, and DOKS as not applicable today.

4. DigitalOcean GPU Droplets: The Mining Angle

DigitalOcean matters to this platform twice: once as the host for the control plane, and once as a provisioning provider that DevOps Engine can drive directly. The provider-key dialog in the application stores DigitalOcean credentials in the platform database (alongside RunPod, Vast.ai and Hetzner), after which the engine can create GPU Droplets through the DigitalOcean API exactly as it does for the other providers. That makes DigitalOcean the only mainstream cloud in the integration set where the web app and the miner fleet can live under one bill. The verified GPU catalog and rates follow; all of them bill per second with a 60-second minimum, so short experiments are cheap to abandon.

GPU (per-GPU hourly)	VRAM	Price	Availability notes
NVIDIA RTX 4000 Ada	20 GB	$0.76/hr	Single-node SKU, Toronto (TOR1)
NVIDIA RTX 6000 Ada	48 GB	$1.57/hr	Single-node SKU, TOR1
NVIDIA L40S	48 GB	$1.57/hr	Single-node SKU, TOR1
NVIDIA H100 80 GB (SXM)	80 GB	from about $3.39/hr	Multi-GPU nodes; 8x node $35.28/hr (about $4.41/GPU)
NVIDIA H200 / AMD MI300X	141 GB / 192 GB	quote/limited	Listed in the portfolio; availability varies by region

Table 5: DigitalOcean GPU Droplet catalog with on-demand rates (docs verified August 2026).

4.1 The honest mining economics

Cloud GPU rates and Bittensor subnet emissions do not mix well at the high end. An H100 at $3.39-$4.41 per hour costs roughly $2,440-$3,175 per month if run 24/7, which exceeds the emissions most subnets distribute to a single miner, so an H100 lease is a bet on a specific subnet's exceptional economics rather than a default. At the other end, the RTX 4000 Ada at $0.76 per hour (about $554 per month at full uptime) is the only DigitalOcean GPU whose rate sits in the same class as the consumer cards that typically mine, and it is

the right choice for a controlled trial. Before any long-running lease, check the platform's economic panels for the target subnet, set a stop-loss reminder for the first 48 hours, and remember that RunPod's community-cloud RTX 4090 at $0.34-$0.69 per hour undercuts DigitalOcean for pure experiments. DigitalOcean's advantages are the integrated API, predictable datacenter hardware, and per-second billing, not absolute price.

One operational note: single-GPU SKUs are concentrated in the Toronto (TOR1) datacenter, so provisioning code should treat region as a first-class parameter and surface honest errors when a region is out of stock instead of retrying silently. DevOps Engine's gate model (AUTO/MANUAL/APPROVAL) exists precisely for this: an out-of-stock or mispriced SKU should stop at an approval gate rather than drain the account.

5. Cross-Provider Comparison

DigitalOcean should still be benchmarked against the alternatives, both because the platform already integrates several providers and because 2026 pricing shifted. Hetzner raised cloud prices in June 2026 (some lines moved multiple-fold; the CPX11 went from 3.85 to 5.49 euros per month), which narrowed but did not close its raw performance-per-dollar lead over DigitalOcean. For the web app, the meaningful comparison columns are entry price, RAM at that price, operational effort, and whether the provider is integrated into the platform's provisioning engine.

Provider	Entry price	What you get	Verdict for this app
DigitalOcean	$6-$24/mo	1-4 GB RAM, 12 regions, GPU catalog, App Platform, integrated API	Best overall; one bill for control plane + provider API
Hetzner	EUR 5.49+/mo (CPX11)	2 GB RAM class, EU + some US regions, best raw CPU per euro	Best budget CPU after June 2026 hikes; not the integrated focus
Vultr	$6/mo	1 GB RAM, 32 regions, similar catalog to DO	Fine DO substitute; no platform integration
Linode (Akamai)	$5/mo	1 GB RAM, strong enterprise network	Legacy-stable, fewer modern extras
Railway	$5/mo Hobby (usage)	PaaS, no server admin, volumes for SQLite	Good zero-ops alternative; usage pricing can surprise
Render	$7/mo per service	PaaS, per-service pricing, auto-deploy	Web + worker model doubles cost; SQLite needs volumes

Table 6: Web-app hosting candidates, September 2026.

5.1 GPU providers versus DigitalOcean

Provider	RTX 4090 class	A100 / L40S class	H100 class	Integrated
DigitalOcean	n/a	L40S $1.57/hr	$3.39-$4.41/hr	Yes
RunPod	$0.34-$0.69/hr	A100 $1.39/hr	$2.89-$2.99/hr	Yes
Vast.ai	about $0.30-$0.40/hr	marketplace	marketplace	Yes

Table 7: GPU rates for the providers integrated by the platform (on-demand, verified Aug-Sep 2026).

Reading the two tables together produces a clean division of labor. DigitalOcean hosts the control plane and is the integrated provider of record when you want cloud-grade reliability on miner nodes; RunPod and Vast.ai win the metered-GPU experiment lane on price, especially for consumer cards; Hetzner is the value pick if you later colocate dedicated CPU miners and accept EU-centric regions. Nothing prevents running all of them at once, since the platform stores per-provider keys and routes each deployment to whichever provider you select. The only hard rule is to keep the control plane off GPU billing entirely, so a runaway mining experiment can never take the dashboard down with it.

6. Production Blueprint: Deploy on a DigitalOcean Droplet

This chapter is the copy-paste path from an empty account to HTTPS on a domain. It assumes the recommended 2 vCPU / 4 GB Basic Droplet on Ubuntu 24.04 LTS, a domain you control, and about thirty minutes. Every command runs as a non-root sudo user except where noted, and the final state is: systemd supervising the standalone server, Caddy terminating TLS, nightly SQLite backups, and the DigitalOcean monitoring agent reporting metrics.

6.1 Steps 1-4: create, harden, install, configure

- **Create the Droplet:** Ubuntu 24.04 LTS, Basic 2 vCPU / 4 GB, your SSH key injected, DigitalOcean Monitoring enabled, weekly snapshot policy on.
- **Harden it:** create a sudo user, disable SSH password auth, then UFW allow 22, 80, 443.
- **Install the runtime:** Bun (which the start script uses) plus git; clone the repository to /opt/infranex.
- **Configure secrets:** write /opt/infranex/.env with the five variables below; keep DEVOPS_SECRET stable across reinstalls so already-paired miners keep verifying.

Environment file

```
# /opt/infranex/.env  (chmod 600)
DATABASE_URL=file:/opt/infranex/db/custom.db
APP_SESSION_SECRET=<64-hex>
DEVOPS_SECRET=<64-hex>
INFRANEX_REPO_ROOT=/opt/infranex
```

6.2 Steps 5-7: build, supervise, serve over TLS

Build the standalone artifact, install it as a systemd unit, and put Caddy in front for automatic Let's Encrypt certificates. Caddy is preferred over Nginx here because it obtains and renews certificates with zero configuration beyond the domain line, and its config file is five lines long. The unit below deliberately runs the server on 127.0.0.1:3000 so nothing is exposed except through Caddy.

Build

```
cd /opt/infranex
bun install
bunx prisma generate && bunx prisma db push
bun run build   # next build + standalone copy
```

systemd unit

```
# /etc/systemd/system/infranex.service
[Unit]
Description=InfraNexBT web platform
After=network-online.target
[Service]
WorkingDirectory=/opt/infranex
Environment=NODE_ENV=production
EnvironmentFile=/opt/infranex/.env
ExecStart=/home/deploy/.bun/bin/bun /opt/infranex/.next/standalone/server.js
Restart=always
RestartSec=5
[Install]
WantedBy=multi-user.target
```

Caddyfile (auto-HTTPS)

```
# /etc/caddy/Caddyfile
app.example.com {
    reverse_proxy 127.0.0.1:3000
}
```

6.3 Steps 8-10: backups, monitoring, updates

- **Backups:** nightly cron running sqlite3 db/custom.db '.backup /var/backups/custom-$(date +\%F).db', then rclone to DO Spaces or another offsite target; keep 14 days.
- **Health probe:** curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:3000/login must return 200 or 307; anything else pages you via the DO monitoring webhook.
- **Clock discipline:** enable systemd-timesyncd; the daemon HMAC channel tolerates plus-or-minus 5 minutes of skew and then rejects telemetry, so time drift shows up as miner silence.
- **Updates:** git pull, bun install, rebuild, systemctl restart infranex; test on a snapshot-restored clone first if a migration touched the schema.

Two final operational rules. Never co-locate miners on the control-plane Droplet: the whole point of the separation is that mining experiments can burn an account without ever taking the dashboard down. And treat APP_SESSION_SECRET and DEVOPS_SECRET as environment-level state, not code: rotate them deliberately (sessions die on rotation, and miners must be re-paired if DEVOPS_SECRET changes), not as a side effect of a server rebuild. This incident already happened once in September 2026 when a sandbox reset silently dropped the variable from the environment file.

7. Cost Scenarios and FAQ

The three scenarios below bundle chapter 3 and 4 pricing into realistic monthly bills, followed by direct answers to the deployment questions that come up most often for this application. Prices are DigitalOcean list prices verified in September 2026; GPU spend is metered separately because it is decided per-experiment, not per-month.

Scenario	Components	Monthly
Minimal	1 vCPU / 1 GB Droplet ($6) + snapshots (about $1.20)	about $8
Recommended	2 vCPU / 4 GB Droplet ($24) + snapshots ($5) + Spaces backup ($5)	about $34
Scale	Droplet ($24) + Load Balancer ($12) + Managed PG HA ($30) + Spaces ($5)	about $71 + GPU

Table 8: All-in monthly cost scenarios (GPU fleet metered separately).

7.1 Frequently asked questions

Can App Platform run this app as-is? The web server deploys fine, but the SQLite file does not survive ephemeral container storage, and the seven background workers want a second always-on service. Practically that means App Platform plus Managed PostgreSQL plus a worker component: zero server admin for roughly $35-$40 per month, at the cost of a Prisma provider migration. If that migration does not appeal, the $24 Droplet is the better buy.

Do I need Kubernetes or a load balancer? No. The application is a single stateful process with an in-database command queue; DOKS adds a control-plane fee and operational surface without adding capability. A Load Balancer earns its $12 per month only when you run two app Droplets behind it for zero-downtime deploys, which is a scale-era concern.

Where do the RunPod, Vast.ai, Hetzner and DigitalOcean keys live? In the platform database, entered through the provider-keys dialog; only RUNPOD_API_KEY is also read from the environment as a fallback. That means migrating hosts means moving the database file (or the managed database), not just the .env file, and it is also why the database backup policy in chapter 6 matters beyond the web data itself.

Can the same server host miners? Technically the Droplet could run CPU miners, but it should not: mining workloads distort the control plane's latency, and the daemon telemetry the platform relies on would be measuring itself. Keep the dashboard on its own small box and provision miner hosts through DevOps Engine so gates, monitoring, and verdict push stay meaningful.

How does hosting interact with the daemon verdict channel? Miners pull commands from /api/daemon/commands and post signed telemetry every 60 seconds, so the control plane needs a stable public hostname with valid TLS (Caddy provides it), an accurate clock, and the shared DEVOPS_SECRET. Silence longer than 10 minutes raises a warning in the platform's monitor, which is the signal to check DNS, certificates, or server time first.

What if I outgrow SQLite? Move to Managed PostgreSQL deliberately: switch the Prisma datasource provider, run db push against the managed cluster, migrate the snapshot-exported data, and keep SQLite as a local dev convenience. The $15 single node is enough for years of the platform's write volume, and the HA tier exists if the dashboard becomes revenue-critical.

GPU MINER OPERATIONS FIELD GUIDE

DEVOPS ENGINE

What it does · What to watch · When to act

A hands-on handbook for Bittensor GPU miners operating through the Infranex platform: how the DevOps Engine registers, validates, provisions and continuously monitors your miner — and the exact checks to run in the first ten minutes, the first hour, the first day and every week after that.

Infranex Bittensor Platform

MINER OPERATIONS · SUBNET MINING · CONTINUOUS OPS

SEPTEMBER 2026

GUIDE V1.0

CHAPTER 1

What the DevOps Engine Is

The DevOps Engine is the miner-facing control plane of the Infranex platform. It is the system that takes an empty GPU machine — a rig in your garage, a colo box, or a rented pod — and turns it into a registered, validated, provisioned and continuously supervised Bittensor miner. Everything it does is grounded in two sources of truth: the Bittensor chain (which tells it which subnet really runs where, via the on-chain identity registry) and the subnet's own GitHub repository (which tells it what the miner actually needs to run). It never invents data: a host that cannot be reached fails honestly, a subnet with no published requirements says so, and a GPU sample exists only because a real daemon reported it.

The engine is built from seven cooperating components. Each one owns a distinct part of the miner lifecycle, and each one leaves an audit trail you can inspect from the web UI or from the API. Understanding this division of labor is the fastest way to understand what the engine can do for you — and, just as importantly, what it deliberately refuses to do for you (like touching your private keys, or silently retrying a failing machine).

Component	What it actually does
Host Registry	Inventory of real SSH-reachable GPU machines. Credentials are stored AES-256-GCM encrypted and never returned to the browser. Mock/fake hosts are banned by design — only real SSH targets can be registered.
Environment Inspector	A 10-step validation pipeline run over SSH: connection, OS, GPU, driver, Docker, NVIDIA toolkit, CUDA ⩾ 12, a real GPU-in-Docker test, one-click fixes, and environment setup. Every step is persisted, resumable and shows its raw output.
Requirements Profiler	Resolves the subnet's GitHub repo from on-chain identity, scrapes its README, and derives the real install spec: minimum VRAM, CUDA/Python versions, package manager (pip / uv / Dockerfile), entrypoint, axon port, OS packages and cluster requirements. Cached 6 hours, invalidated when the override changes.

Requirement Installer	Builds a 9-step provisioning plan from that profile (packages, runtime, clone, dependencies, env file, wallet gate, launch gate, verify) and executes it one step at a time over SSH, persisting each result so the wizard is resumable and auditable.
Continuous Monitor	Every 90 seconds it evaluates every started miner: GPU health (temperature, utilization, process liveness, daemon silence) and subnet drift (hyperparameter changes, UID moves, economics regime shifts). Findings become deduplicated, approval-gated events.
Node Daemon	A tiny zero-dependency Python agent installed on the host. Every 60 seconds it HMAC-signs and uploads GPU telemetry (utilization, memory, temperature, process liveness) and pulls its command queue. It can only pull — the control plane never pushes into your machine, and every command was approved by a human.
Trigger & Strategy Layer	The recommendations inbox: GPU health, subnet drift, deregistration risk, immunity runway, upstream repo drift, benchmark regressions, validator-traffic probes and Miner-Mindset arbitrage/runtime suggestions — each with evidence and a one-click approved action.

Two design rules matter to you as an operator. First, **gates everywhere**: the engine will not spend your money or start a workload without an explicit approval from you, and it will never create, move or copy your wallet keys — the wallet step is a manual checkpoint where you confirm the files are in place. Second, **honest states**: when a daemon is silent, when a provider key is missing, or when a host fails to connect, the UI shows the failure and its remediation instead of hiding it behind a simulated green light. If you ever see a green check, a real command really ran.

CHAPTER 2

The Miner Lifecycle, End to End

Before you can monitor anything, the engine has to walk your machine through a fixed lifecycle. Knowing where your miner is in this lifecycle tells you exactly which checks apply and which alarms are meaningful. The journey has four stages, and each one ends in a state the engine records in its database — nothing is left implicit.

Stage 1 — Register the host

You give the engine a name, an IP/hostname, an SSH port, a user and one credential (password or private key). The secret is encrypted at rest and never leaves the server. Registration alone touches nothing on the machine — it only creates the inventory record, marked `PENDING` .

Stage 2 — Validate the environment (the 10-step pipeline)

The engine SSHes in and runs its inspection: **1** connect → **2** detect OS → **3** detect NVIDIA GPU (model + VRAM) → **4** validate driver and its CUDA capability → **5** validate Docker and Compose → **6** validate the NVIDIA container toolkit → **7** check CUDA ⩾ 12.0 → **8** prove GPU passthrough with a real `docker run --gpus all` test → **9** install or fix what is missing (safe fixes can run automatically) → **10** write the environment file. Each step keeps its raw output, duration and remediation hint; a fix that passes its re-check is marked FIXED .

Stage 3 — Stage the install plan

You pick a subnet and name your wallet and hotkey. The engine pulls that subnet's live requirements profile and builds a 9-step plan tailored to the repo — a venv pipeline for pip repos, a Docker pipeline for repos that ship a Dockerfile, and honest manual gates for Kubernetes-class subnets that must be provisioned with their own tooling. Staging runs nothing; the plan sits in STAGED until you execute steps.

Stage 4 — Execute, launch and monitor

You run the steps in order. Two of them are deliberately yours to control: the wallet checkpoint and the launch. When the final verify step passes, the install is DEPLOYED , the miner appears on the DevOps cockpit, and the 90-second monitor pass starts watching it for the rest of its life. You can stop the miner at any time (a safety stop disables the service and marks the install STOPPED), and relaunching is a one-step re-run.

The three gate types you will meet in the plan are worth memorizing, because they define who is allowed to do what:

Gate	Meaning	Which steps
AUTO	Runs unattended when you click the step; output is recorded verbatim.	OS packages, venv/Docker setup, repo clone, dependencies, env file, verify.
MANUAL	The engine can check, but only you can truthfully confirm — it refuses to pretend.	Wallet files present on the host (your coldkey + hotkey, copied by you).
APPROVAL	Starts burning compute; requires an explicit human approval click.	Launch the miner (systemd unit or GPU container).

CHAPTER 3

What to Check Once the GPU Miner Is Started

The moment the launch step completes, your job changes from building to watching. The engine watches with you — every 90 seconds, automatically — but the division of labor is specific: the engine detects and recommends, **you** confirm and approve. This chapter gives you the exact check sequence in four time windows, from the first ten minutes to the ongoing weekly rhythm. Every check names where to look (the DevOps cockpit or the host itself) and what a healthy result looks like, so a green pass is never a matter of opinion.

WINDOW 1 · FIRST 10 MINUTES

Is it actually up?

- **Service state.** The verify step ran `systemctl is-active infranex-miner-sn<netuid>` and expected **active**. Re-run it yourself from the host; a unit that flaps (crash-looping under `Restart=always`) shows as activating for a while before it shows as failed.
- **Axon port listening.** The engine checks `ss -tlnp | grep :<axon-port>` (8091 for most subnets). If the port is not listening, validators cannot reach you and no incentive can flow, no matter how healthy the process looks.
- **First log lines are clean.** The engine tails the log (systemd path: `/var/log/infranex-miner-sn<netuid>.log`; Docker path: `docker logs --tail 20 infranex-miner-sn<netuid>`). Look for the wallet being loaded, the network (finney) being connected and the axon serving — not for tracebacks, wallet-not-found or chain-sync errors.
- **Daemon telemetry arrives.** On the DevOps cockpit your miner card should flip the daemon badge to online within a minute or two (the daemon posts every 60 seconds). If it stays offline, the platform has no eyes on the machine and every later alarm is blind.
- **The monitor pass sees you.** The cockpit footer shows the last monitor pass; within one or two passes (⩽ 3 minutes) your miner should be counted in it.

```
# on the host — the same evidence the verify step collects
systemctl is-active infranex-miner-sn<netuid>
ss -tlnp | grep ':<axon-port> '
tail -n 20 /var/log/infranex-miner-sn<netuid>.log
# docker-installed repos instead:
docker ps --filter name=infranex-miner-sn<netuid>
docker logs --tail 20 infranex-miner-sn<netuid>
```

WINDOW 2 · FIRST HOUR

Is it healthy and reachable?

- **GPU vitals.** The miner card's sparkline should show real samples: temperature comfortably below the 78 °C warning line, utilization following the workload, and VRAM allocated against capacity. A flat zero utilisation on a warm GPU means the process is alive but not working.
- **Chain registration.** The engine reads your UID, incentive and emission straight from the metagraph. A fresh miner shows its assigned UID; if the UID column stays empty your hotkey is not registered on this subnet and nothing you fix on the host will change that.
- **Axon probe.** The service-health pass synthetically probes your axon from the platform side. A `PROBE FAIL` here with a healthy local service usually means the port is published but not reachable (firewall, NAT, provider security group).
- **Recommendations inbox is quiet.** One or two passes can surface early problems (thermal, daemon silence). A critical event in the first hour is normal to see once — it is not normal to ignore.

WINDOW 3 · FIRST 24 HOURS

Is it earning and stable?

- **Incentive is flowing.** After validators score you, the incentive/emission figures on the miner card should move off zero. Emissions lag registration by some hours — that is chain economics, not a platform fault.
- **No repeated thermal or idle warnings.** The engine only alarms after repeated confirmations (two passes for temperature, three for idle GPU), so any warning that survives a full day is a sustained condition, not noise.
- **Registration is stable.** Watch for `DEREG-RISK` or `RUNWAY` events: hotkey UID moves or immunity-period countdowns mean your slot on the subnet is contested.
- **No unexplained drift.** `SUBNET DRIFT` events in the first day usually mean the subnet owner changed hyperparameters or the metagraph moved — the event carries the evidence; approve the suggested re-sync or restart when it looks right.

WINDOW 4 · ONGOING (DAILY / WEEKLY)

Keep it earning

- **Daily — triage the inbox.** Every open event carries evidence and a suggested action (restart miner, re-sync config, recycle compute, fail over). Approve what is right, decline what is not; either way the event history is your ops log.
- **Daily — glance at the sparklines.** Temperature creep (dust, fan failure) and utilisation collapse show up hours before earnings stop.
- **Weekly — upstream watch.** The engine diffs the subnet's repo; `UPSTREAM` events mean the code you are running is no longer what the subnet ships. `BENCH REGRESS` events mean your measured performance fell against the rolling baseline.
- **Weekly — housekeeping.** The systemd unit appends to `/var/log/infranex-miner-sn<netuid>.log` forever — check disk space. The requirements profile re-scrapes automatically every 6 hours, so a subnet that changes its README re-plans without your involvement.
- **Weekly — provider keys.** Provisioning actions that spin up new machines need live provider API keys (RunPod / Hetzner / DigitalOcean). A missing key surfaces as an honest configuration error, not a silent failure — re-add keys in Settings when prompted.

The what-and-when matrix

When	Check	Healthy looks like	Where
First 10 min	Service active + axon port listening	**active** + port in LISTEN	Host / verify step
	First log lines	Wallet loaded, axon serving, no tracebacks	Host log tail
	Daemon telemetry + monitor pass	Daemon online; miner counted in pass	DevOps cockpit
First hour	GPU temp / util / VRAM samples	Temp < 78 °C; util follows workload	Miner card sparkline
	UID / registration on chain	UID assigned	Miner card chain facts
	Axon reachability probe	No PROBE FAIL event	Recommendations inbox
First 24 h	Incentive / emission flow	Moves off zero	Miner card economics

First 24 h	Sustained alarms	No thermal / idle / dereg events	Recommendations inbox
Daily	Open events triage	All events approved or declined	Recommendations inbox
	Sparkline trends	No temp creep, no util collapse	Miner Ops board
Weekly	Upstream repo / benchmark drift	No unresolved UPSTREAM / BENCH events	Change feed
	Log disk usage + provider keys	Disk under control; keys valid	Host + Settings

CHAPTER 4

Alarm Thresholds & Event Reference

The monitor enforces one fixed set of thresholds, published on the cockpit so the UI always draws the same lines the engine enforces. Two properties of these alarms matter for interpreting them. First, **debounce by design**: a condition must hold for several consecutive 90-second passes before it becomes an event, so single-sample spikes never page you. Second, **evidence in every event**: each finding stores the readings that triggered it, and recovery auto-resolves the event when the condition clears, so the change feed is a faithful history rather than a pile of stale tickets.

Threshold	Value	Meaning
GPU temperature warning	**78 °C** × 2 passes	Sustained hot GPU — check cooling before it throttles.
GPU temperature critical	**85 °C** × 2 passes	Danger zone — expect throttling and possible hardware damage.
Idle GPU	util < **30%** × 3 passes	Process alive but not doing subnet work — typically a stuck or mis-configured miner.
Miner process dead	**2 passes**	Critical event; the suggested action is a daemon `restart_miner` command.
Daemon silence	**10 minutes**	No telemetry for 10 min — daemon down, host down, or network path broken.

Economics regime shift	**±50%**	Median rewarded incentive on the subnet moved by half — re-check profitability.
Sample retention	**240 samples**	≈ 6 hours of GPU history at the 90-second cadence (sparkline window).

Event kinds you will meet in the inbox

Event	What it tells you
GPU HEALTH	Thermal, idle or process-down condition on a specific miner, with the readings attached.
SUBNET DRIFT	The subnet changed under you: hyperparameters (tempo, immunity, UID cap), metagraph capacity, economics regime, or your hotkey's UID moved.
DEREG-RISK	Your registration slot is at risk — typically UID churn; the action is usually failover to a fallback profile.
RUNWAY	Immunity-period countdown entered the watch or at-risk band: deregistration becomes possible at T-zero.
PROBE FAIL	The synthetic axon probe could not reach your miner from the platform — network/port problem, not process health.
SERVICE LATENCY	Your axon responds but too slowly — validators may be timing you out.
QUERY DROUGHT	Validator traffic to your miner dried up — you are registered but not being queried.
ESCALATION	The automatic repair ladder is exhausted; approving executes the recorded rollback (or the next step is a stop).
UPSTREAM	The subnet's repo moved after you deployed — your running code is stale.
BENCH REGRESS	A benchmark run regressed against the rolling baseline — performance, not uptime.
ARBITRAGE	Miner-Mindset finding: another subnet pays better for your GPU class; the action recycles compute.
RUNTIME OPT	Miner-Mindset finding: a serving profile (accelerated runtime recipe) fits your workload and would raise yield.

CHAPTER 5

Step-by-Step Operator Routine

This is the distilled daily loop, written as you would actually perform it. The first three steps happen in the web UI; the rest only apply when something looks wrong or when you prefer to confirm on the machine itself. Done faithfully, the whole routine takes a few minutes a day per miner — the engine has already done the watching; you are doing the judging.

1. **Open the DevOps cockpit and read the summary bar.** It states in one line how many miners are monitored and how many are healthy, warning or critical, whether daemons are online, and the current infra cost versus revenue per day. This bar is the fastest honest answer to "is everything fine right now?"

2. **Triage the recommendations inbox.** Work top-down: critical events first (GPU health, escalations, probe failures), then warnings (drift, deregistration risk, runway), then strategy findings (arbitrage, runtime optimizations). Expand each event, read its evidence, and either apply the suggested action to the GPU or decline it — never leave criticals sitting.

3. **Walk the per-miner ops board.** For each started miner, check the daemon badge (online, last seen within a couple of minutes), the GPU sparklines (temperature and utilization trends over the last ~6 hours), and the chain facts (UID, incentive, emission, registration state). Anything flatlined or empty gets investigated in steps 4–6.

4. **Confirm the process is really serving.** On the host, the unit and its log answer in seconds. The systemd unit is named `infranex-miner-sn<netuid>` and the Docker container reuses the same name, so one set of commands covers both install paths.

```
# process + port + recent log, one shot
systemctl status infranex-miner-sn<netuid> --no-pager -l
journalctl -u infranex-miner-sn<netuid> -n 50 --no-pager
tail -n 50 /var/log/infranex-miner-sn<netuid>.log
nvidia-smi
```

5. **If the daemon is unreachable, check the agent, not just the miner.** The daemon is a small Python service posting every 60 seconds. Silence beyond 10 minutes triggers the platform warning; the usual culprits are the daemon process itself, the host's outbound network, a platform URL change, or clock skew (signatures are only valid within a ±5-minute window, so a wrong clock silently kills telemetry). Restarting the daemon process is safe and requires no re-registration.

6 **When an action fires, verify its effect.** Approved commands (restart miner, apply config) reach the daemon's queue on its next poll and the result is recorded back to the event. After any restart or re-sync, re-check the miner card: telemetry resumes, the process-alive flag flips back, and the event auto-resolves when the underlying condition clears. If nothing changes within a few passes, the problem is on the host — go to step 4.

7 **Close the loop weekly.** Review the change feed for what auto-resolved while you were away, check log disk usage on long-running hosts, confirm provider keys in Settings are still valid for machines the engine may provision, and skim upstream/benchmark events to decide whether a re-deploy to newer subnet code is warranted.

CHAPTER 6

Troubleshooting Map

When an install step fails, the engine writes a remediation hint next to the raw output — the table below collects the ones you will actually meet, so you can act without deciphering raw logs first. The pattern to remember: install failures are almost always environment problems (network, apt sources, missing prerequisites) on the host, and they are re-runnable — fix the cause, press the step again, and the plan resumes where it stopped. Nothing ever continues past a failed step on its own.

Failed step	Typical cause	What to do
Install OS packages	Broken apt sources or no outbound network on the host.	Fix distro sources / connectivity, re-run the step.
Install Docker + NVIDIA runtime	curl or the NVIDIA repo unreachable from the host.	Install docker-ce and nvidia-container-toolkit manually, then retry.
Create Python venv	Python version mismatch or python3-venv missing.	Install the required python3.x (deadsnakes) — the OS-packages step normally adds venv.
Clone the subnet repo	Repo not public or host has no egress.	Check the URL is reachable from the host; retry.
Install Python dependencies	Heavy CUDA wheels; RAM or disk pressure.	Read the pip output, free resources if needed, retry.
Wallet files present	Coldkey/hotkey not copied to the host.	Copy `~/.bittensor/wallets/<wallet>` (coldkey + hotkeys/<hotkey>) to the host via scp, then confirm.

Launch the miner	Bad wallet path, port conflict, chain error.	The verify step's log tail usually shows the reason; fix and re-run the launch step.
Verify the miner is live	Service inactive or axon port not listening.	`journalctl -u infranex-miner-sn<netuid>` — common causes: wallet path, port conflict, chain sync.

Two platform-side conditions round out the map. **Mock transport refused**: if you ever import or inherit a host record marked as a mock, every step will fail loudly with a message telling you to delete and re-register the machine over real SSH — that is the honesty rule working as designed, not a bug. **Missing provider keys**: flows that create machines on your behalf (deployments migrating into DevOps, autopilot recycling compute) will fail with a clear configuration error until the RunPod / Hetzner / DigitalOcean keys are re-added in Settings; the engine will never fall back to a simulated machine to cover the gap.

Real machines.
Real telemetry.
Nothing fabricated.

The DevOps Engine does the watching so you can do the judging: it registers, validates, provisions and monitors every miner you start — but it never spends your compute, moves your keys, or hides a failure. When you see green, a real command really ran.

POCKET REFERENCE

unit name	infranex-miner-sn<netuid>
log file	/var/log/infranex-miner-sn<netuid>.log
monitor cadence	every 90 s
daemon heartbeat	every 60 s (silence > 10 min = warning)
thermal alarms	78 °C warn / 85 °C crit (×2 passes)
idle alarm	util < 30% × 3 passes
requirements cache	re-scraped every 6 h